

LEVEL 7 PROJECT REPORT

AWS Infrastructure as Code with Terraform for Java Web Application Deployment

using Terraform, Maven, Docker, Amazon ECR, AWS EC2AWS ASG

Technology Stack: Terraform • Maven • Docker • Amazon ECR • Amazon ALB • Amazon ASG• Amazon EC2
• Tomcat

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Prepared by Manimaran

Implementation Images: 90 screenshots + 1 system architecture diagram

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives
- 4. Technologies and AWS Services
- 5. Prerequisites
- 6. System Architecture
- 7. Complete Infrastructure Workflow
- 8. Project Components and Terraform Structure
- 9. Complete Implementation Procedure – 90 Screenshot Actions
- 10. Terraform Configuration Overview
- 11. AWS Infrastructure Layers
- 12. Docker, Tomcat and Amazon ECR Integration
- 13. Application Request Flow
- 14. Validation and Verification Checklist
- 15. Troubleshooting and Operational Notes
- 16. Security Notes
- 17. Key Lessons Learned
- 18. Interview / Viva Explanation
- Appendix – Complete Project Flow

1. Project Overview

This Level 7 project demonstrates Infrastructure as Code (IaC) using Terraform to provision and manage a complete AWS application environment. The implementation defines the network, routing, security, IAM permissions, Amazon ECR, EC2 Launch Template, Application Load Balancer, target group and Auto Scaling Group as Terraform-managed resources.

The Java web application is containerized with Docker and Tomcat and is delivered through Amazon ECR. EC2 instances are initialized through Launch Template user data, while the Application Load Balancer provides the public entry point and forwards traffic to healthy application targets on port 8080.

The supplied screenshots cover the lifecycle from local Terraform/AWS CLI setup through variables, VPC/routing, security, IAM, ECR, Docker/Tomcat, Launch Template, ALB, Target Group, Auto Scaling, application validation and Terraform cleanup.

2. Problem Statement

Manual creation of AWS networking, security, compute and load-balancing resources is repetitive and difficult to reproduce consistently. This project addresses that workflow by defining the environment as Terraform code, validating and planning the configuration, provisioning it through terraform apply, validating the application and finally cleaning up with terraform destroy.

3. Objectives

- Use Terraform to define AWS infrastructure as code.
- Create public/private VPC subnets across Availability Zones.
- Configure Internet Gateway, NAT Gateway and route tables.
- Secure ALB and EC2 tiers with separate security groups.
- Create EC2 IAM role, instance profile and ECR pull permissions.
- Build/test the Java/Tomcat Docker image and publish it to ECR.
- Create an EC2 Launch Template with automated Docker/Tomcat startup.
- Expose the application through an Application Load Balancer on port 80.
- Use a target group on application port 8080 and Auto Scaling.
- Validate healthy targets and end-to-end application access.
- Demonstrate Terraform lifecycle cleanup with terraform destroy.

4. Technologies and AWS Services

Technology / Service	Purpose / Role
Terraform	Infrastructure as Code and lifecycle management
AWS VPC	Network foundation
Public/Private Subnets	Network separation
Internet Gateway / NAT Gateway	Internet and private outbound connectivity
Security Groups	ALB/EC2 traffic control
IAM	EC2 and ECR permissions
Amazon ECR	Private Docker registry
Docker / Tomcat	Container runtime and Java web server
Amazon EC2	Application compute
Launch Template	Repeatable EC2 bootstrap
Application Load Balancer	Public HTTP entry point
Target Group	Target registration and health checks
Auto Scaling Group	Application capacity management
Java Web Application	SampleApp

5. Prerequisites

- AWS account with required VPC, EC2, IAM, ECR, ELB and Auto Scaling permissions.
- Terraform installed.
- AWS CLI installed and configured.
- AWS identity verified with `aws sts get-caller-identity`.
- VS Code or another editor.
- Docker installed.
- Java/Tomcat SampleApp and Dockerfile.

6. System Architecture

The project starts with the Terraform project and provisions the AWS network, security, IAM, ECR, Launch Template, ALB, target group and Auto Scaling Group. Application EC2 instances run behind the public ALB and serve the containerized Java application on port 8080.

Architecture flow: Developer → GitHub/Terraform project → Terraform → AWS VPC → ALB :80 → Target Group → EC2 :8080 → Docker/Tomcat → Java Web Application.

AWS DevOps System Architecture

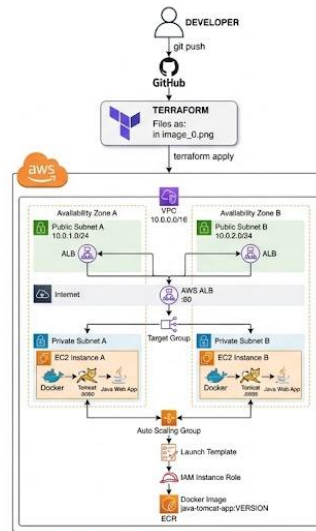


Figure 1. AWS DevOps System Architecture

7. Complete Infrastructure Workflow

Step	Flow	Purpose
1	Terraform project	Define desired AWS state
2	init / validate / plan	Initialize provider, validate and preview changes
3	VPC and routing	Provision VPC, subnets, IGW, NAT and routes
4	Security	Apply ALB and EC2 security controls
5	IAM	Create role, policy and instance profile
6	ECR / Docker	Build, test and publish application image
7	Launch Template	Automate EC2 Docker/Tomcat startup
8	ALB / Target Group	Forward port 80 traffic to port 8080
9	Auto Scaling	Maintain application instances and target registration
10	Validation / Cleanup	Verify app and run terraform destroy

8. Project Components and Terraform Structure

- provider.tf — AWS provider configuration.
- variables.tf — reusable variables.
- terraform.tfvars — environment values and CIDRs.
- vpc.tf — VPC, subnets, Internet Gateway, NAT Gateway and routing.
- security-groups.tf — ALB and EC2 security rules.
- iam.tf — role, policy and instance profile.
- ecr.tf — ECR repository.
- ec2.tf — Launch Template and user data.
- alb.tf — ALB, target group and listener.
- autoscaling.tf — Auto Scaling Group.
- outputs.tf — important IDs/endpoints.
- Dockerfile / SampleApp — Java/Tomcat application.

9. Implementation Procedure

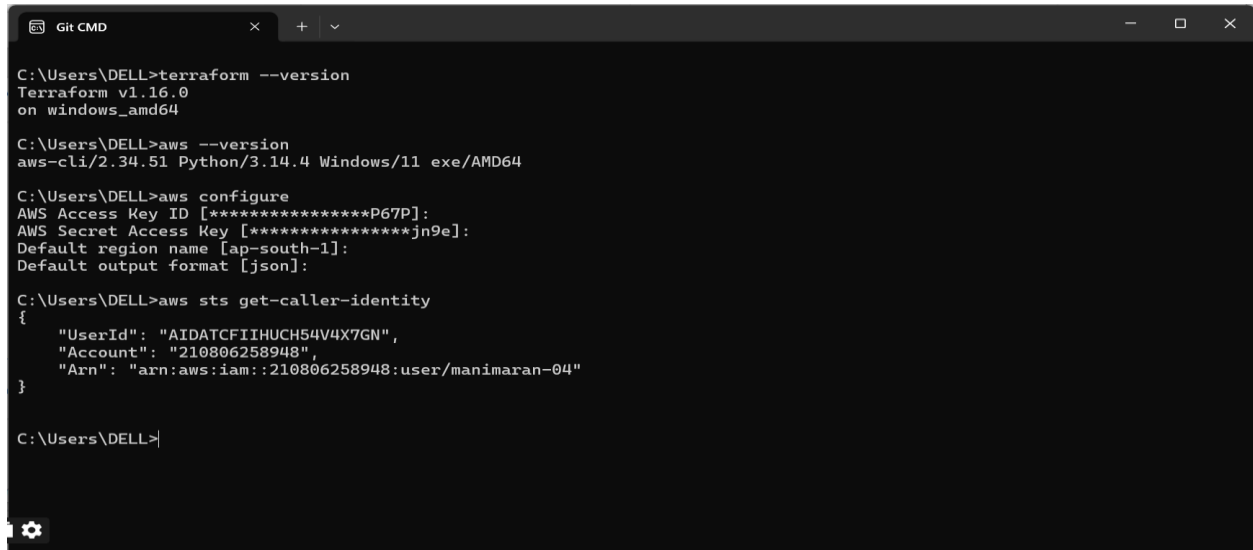
The following 90 steps are presented in the chronological order. Each instruction/action is placed before its corresponding image.

Phase 1 – Terraform Setup and Configuration

Step 1 : Verify Terraform and AWS CLI

Implementation Action / What I did: Verify Terraform/AWS CLI versions and the configured AWS identity.

What the screenshot proves: Terraform, AWS CLI and account context are available.



```
Git CMD
C:\Users\DELL>terraform --version
Terraform v1.16.0
on windows_amd64

C:\Users\DELL>aws --version
aws-cli/2.34.51 Python/3.14.4 Windows/11 exe/AMD64

C:\Users\DELL>aws configure
AWS Access Key ID [*****P67P]:
AWS Secret Access Key [*****jn9e]:
Default region name [ap-south-1]:
Default output format [json]:

C:\Users\DELL>aws sts get-caller-identity
{
  "UserId": "AIDATCFIIHUCH54V4X7GN",
  "Account": "210806258948",
  "Arn": "arn:aws:iam::210806258948:user/manimaran-04"
}

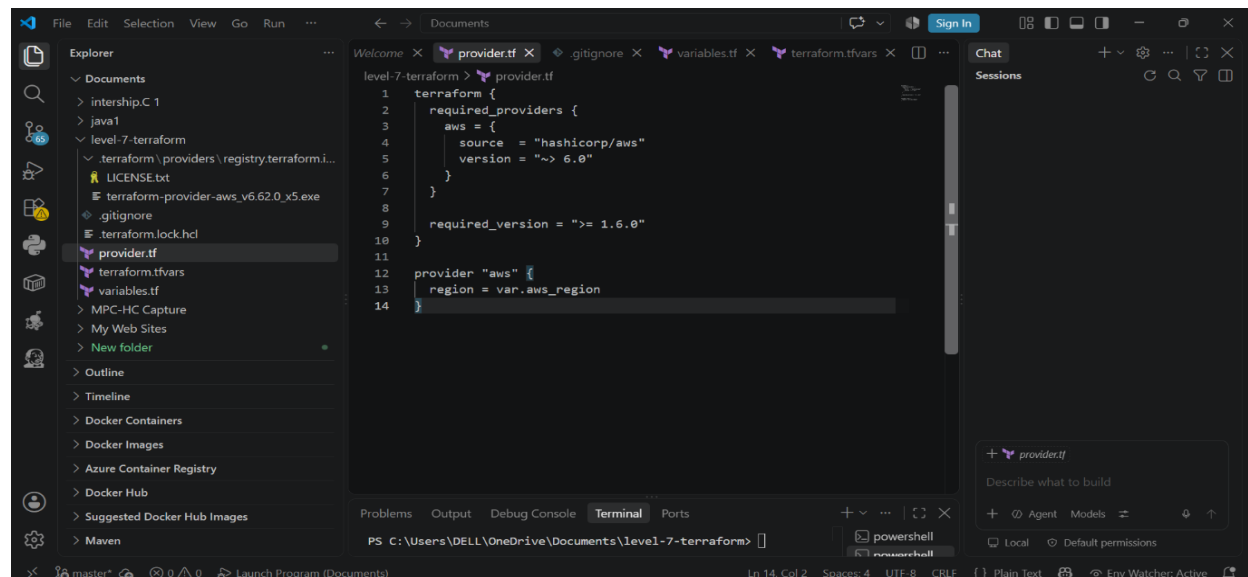
C:\Users\DELL>
```

Figure 2. Verify Terraform and AWS CLI.

Step 2 : Create provider.tf

Implementation Action / What I did: Define Terraform and AWS provider requirements.

What the screenshot proves: The provider foundation is defined.



```
level-7-terraform > provider.tf
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7   }
8   required_version = ">= 1.6.0"
9 }
10
11
12 provider "aws" {
13   region = var.aws_region
14 }
```

Figure 3. Create provider.tf.

Step 3 : Configure the AWS provider

Implementation Action / What I did: Review the AWS provider block and required version.

What the screenshot proves: Terraform is configured to use AWS.

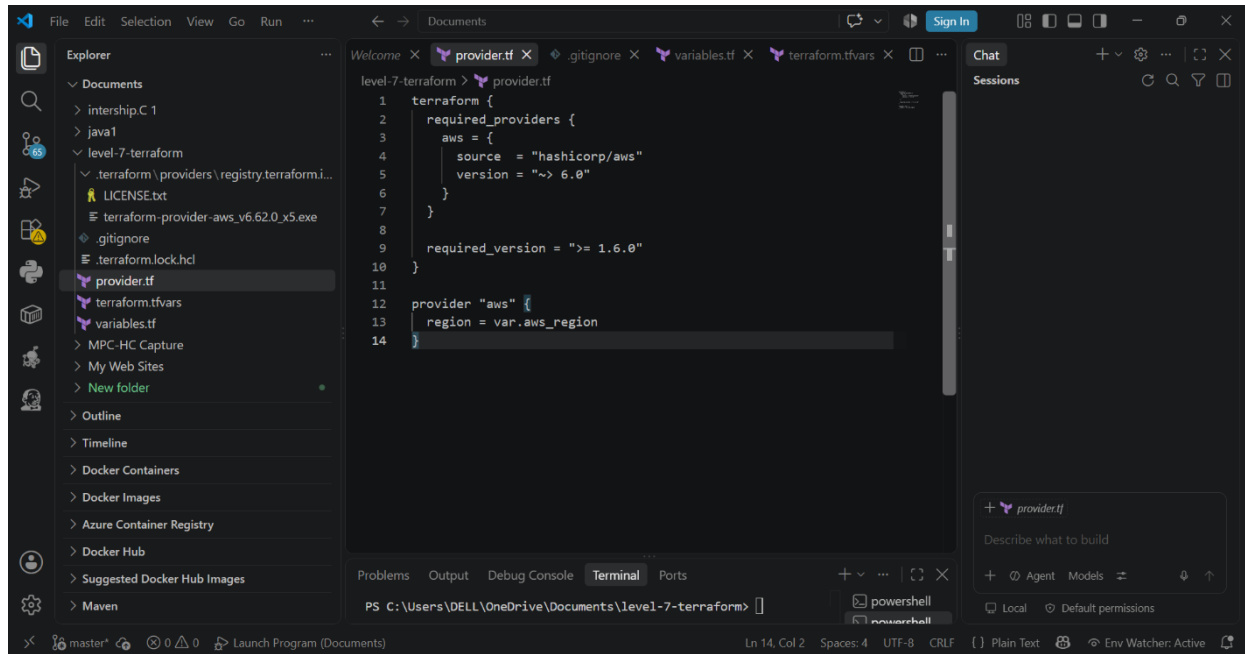


Figure 4. Configure the AWS provider.

Step 4 : Define reusable variables

Implementation Action / What I did: Create region, project, environment and instance variables.

What the screenshot proves: Reusable Terraform inputs are defined.

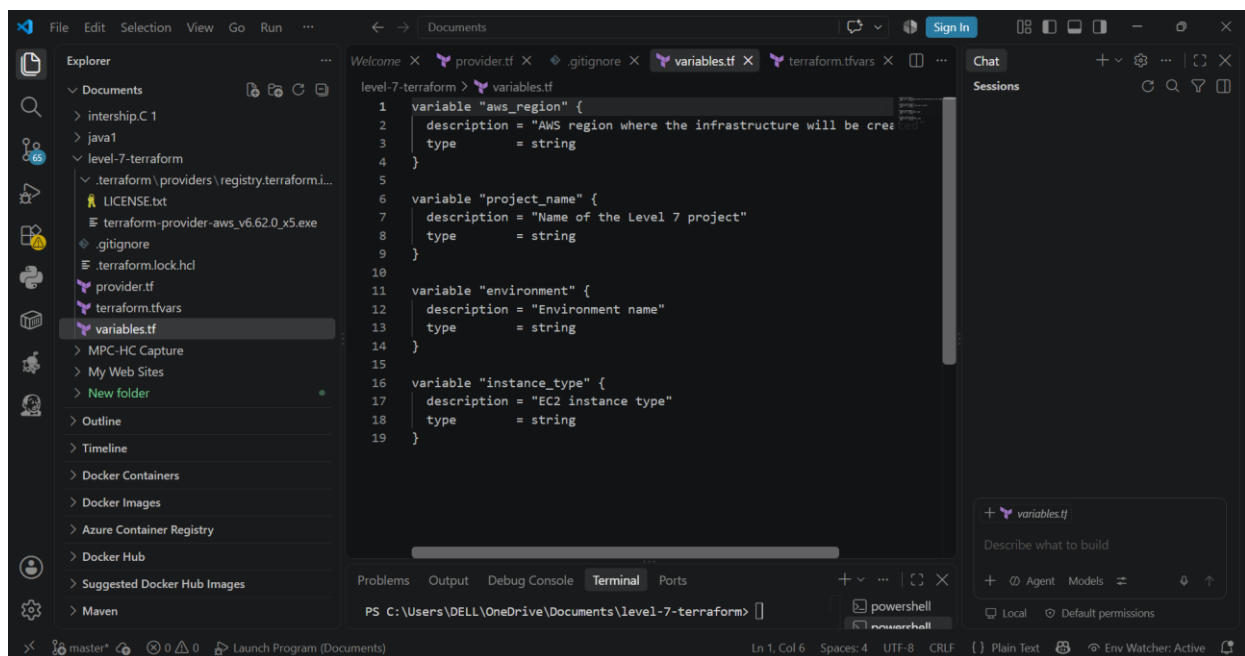


Figure 5. Define reusable variables.

Step 5 : Configure terraform.tfvars

Implementation Action / What I did: Set environment-specific region, project, instance and network values.

What the screenshot proves: Environment values are separated from resources.

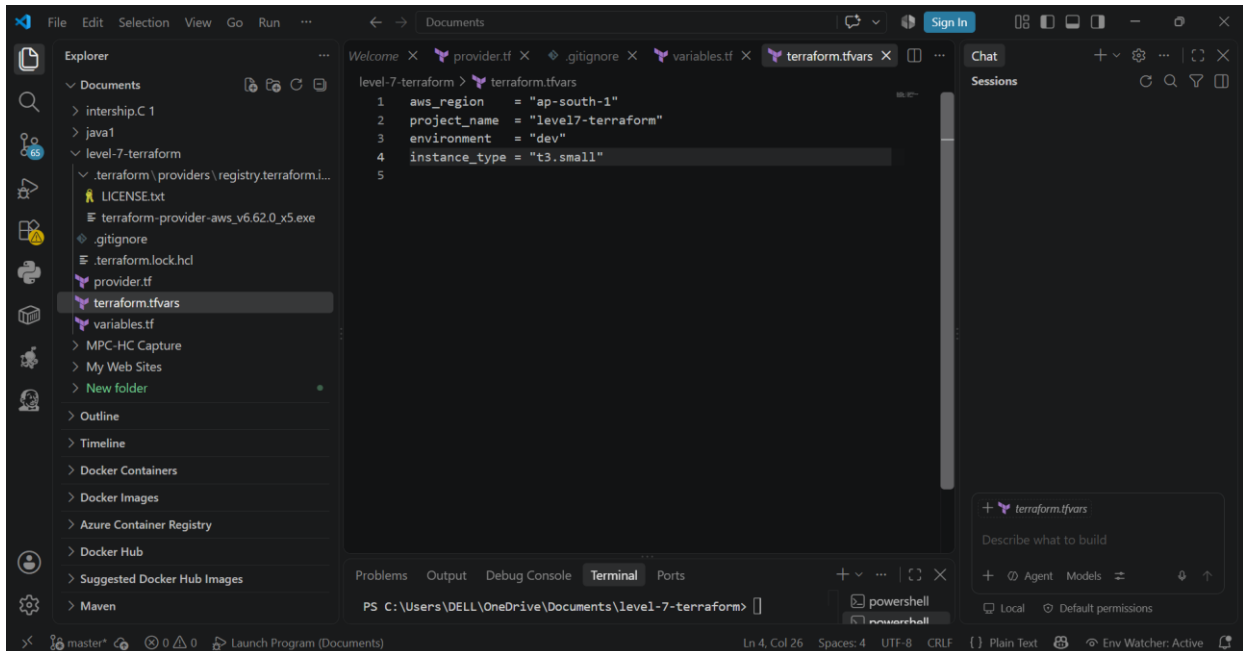


Figure 6. Configure terraform.tfvars.

Step 6 : Initialize, validate and plan Terraform

Implementation Action / What I did: Run terraform init, validate and plan.

What the screenshot proves: Initialization, validation and planning succeed.

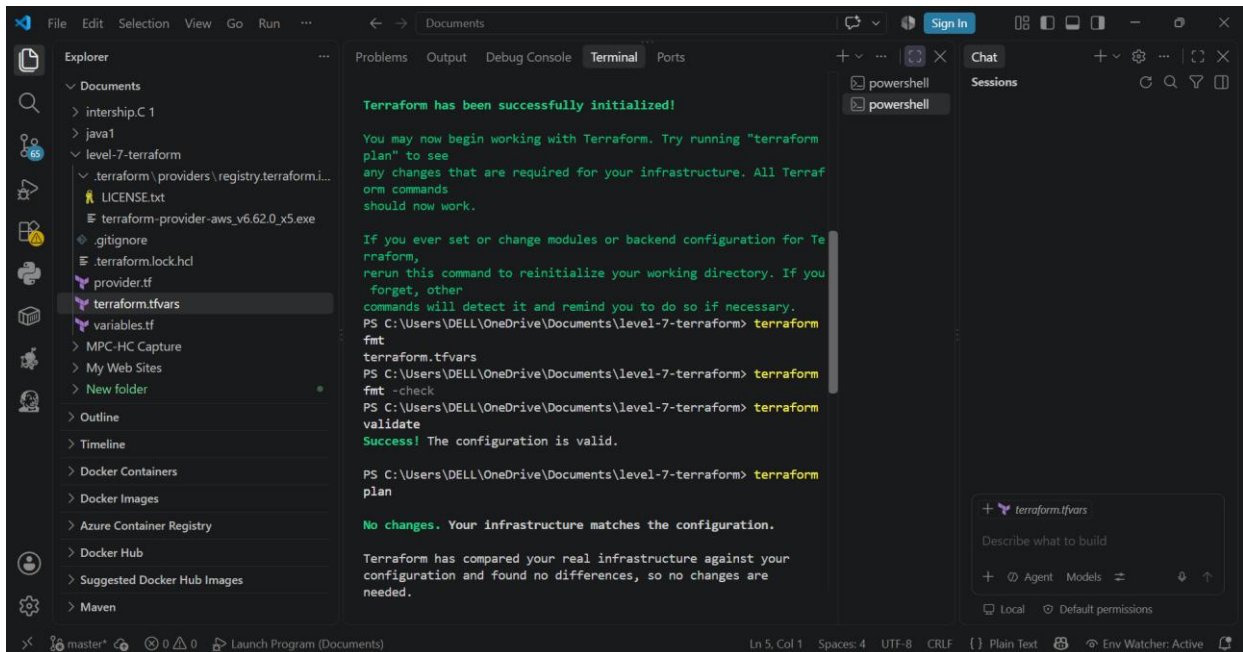


Figure 7. Initialize, validate and plan Terraform.

Step 7 : Define network CIDR variables

Implementation Action / What I did: Add VPC and public/private subnet CIDR variables.

What the screenshot proves: Network inputs are parameterized.

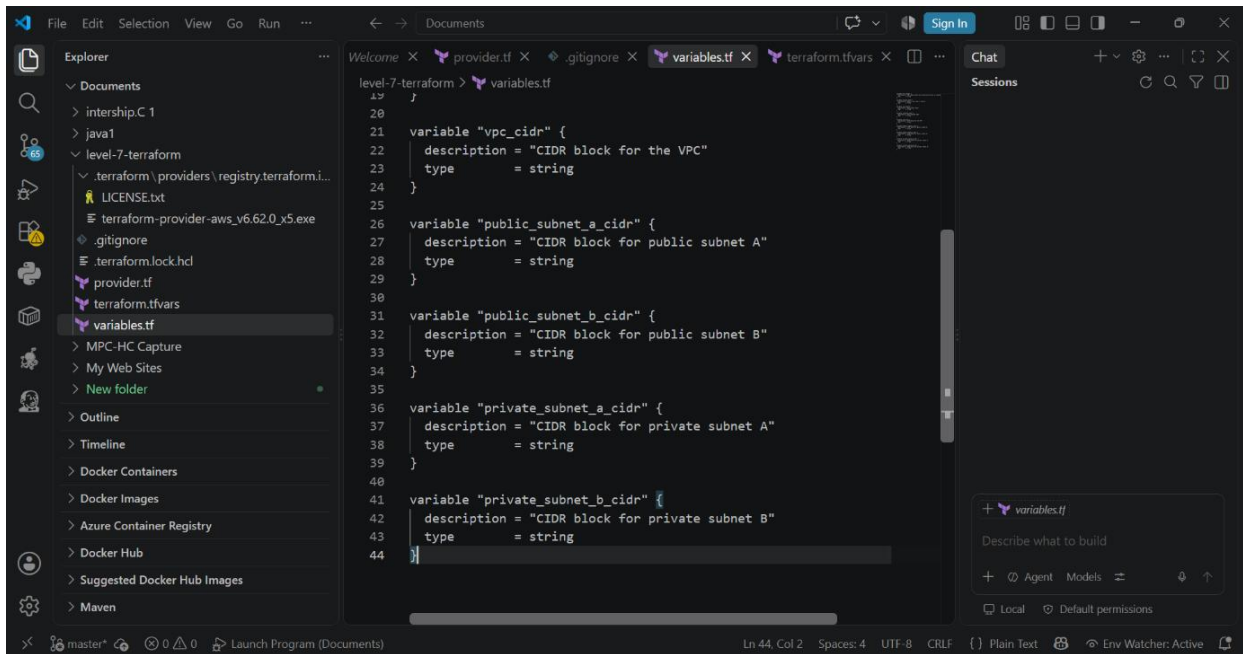


Figure 8. Define network CIDR variables.

Step 8 : Review network environment values

Implementation Action / What I did: Confirm region and CIDR values before provisioning.

What the screenshot proves: The intended addressing plan is visible.

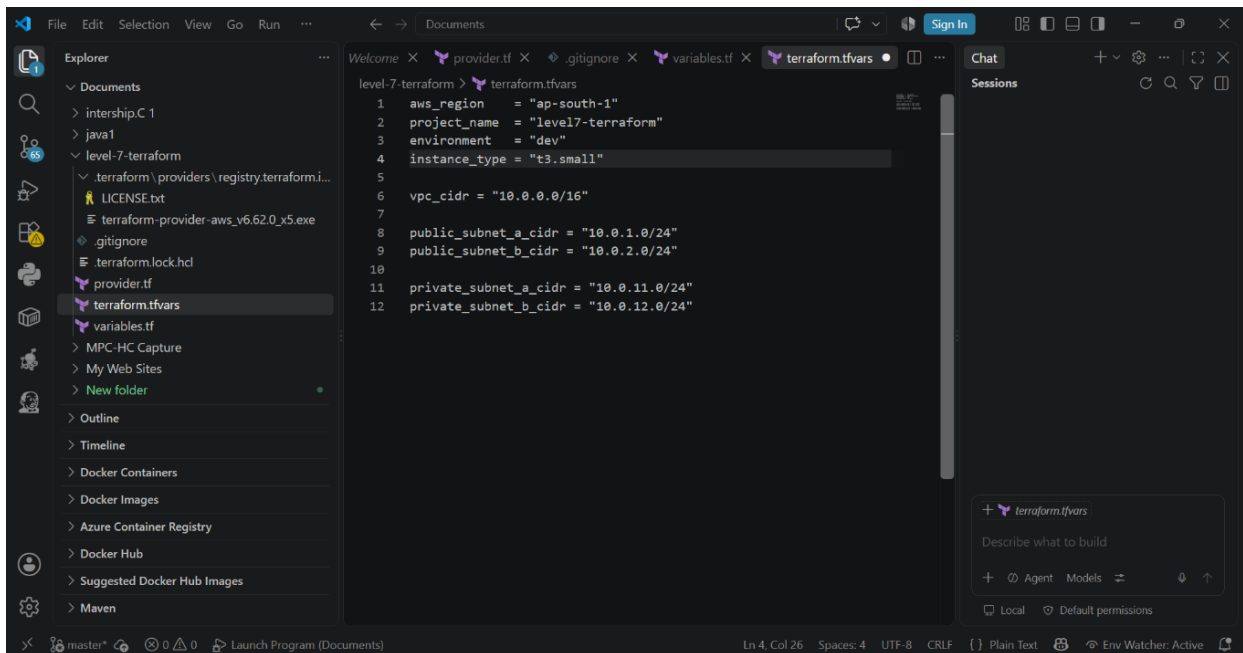


Figure 9. Review network environment values.

Phase 2 – VPC, Subnets, Routing and Apply

Step 9 : Define the VPC resource

Implementation Action / What I did: Implement the define the vpc resource in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.

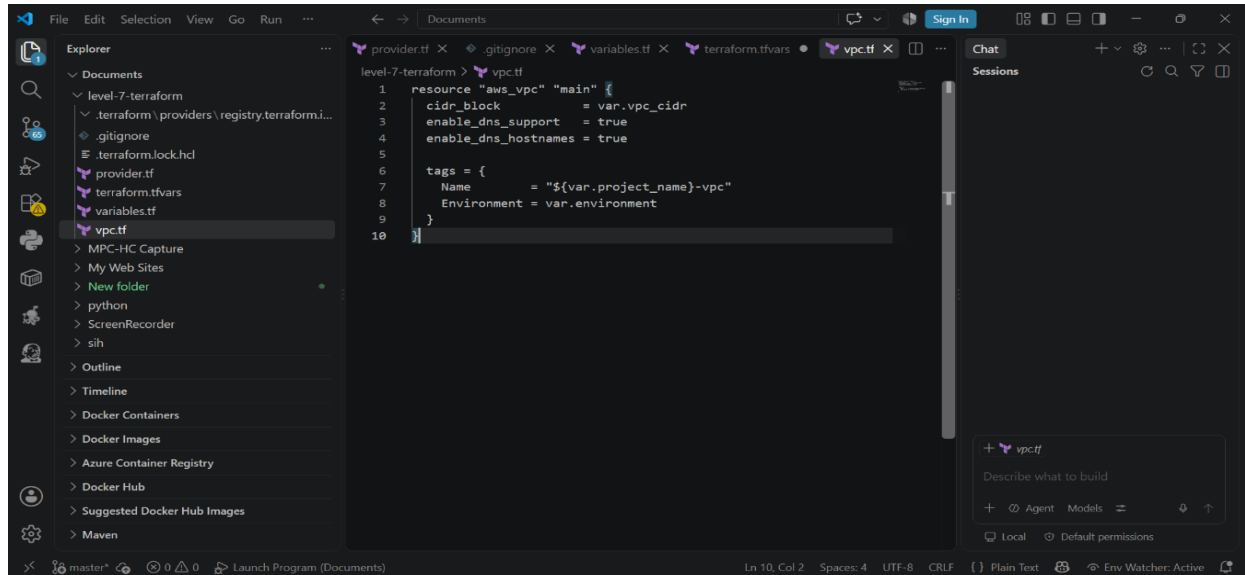


Figure 10. Define the VPC resource.

Step 10 : Configure VPC attributes and tags

Implementation Action / What I did: Implement the configure vpc attributes and tags in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.

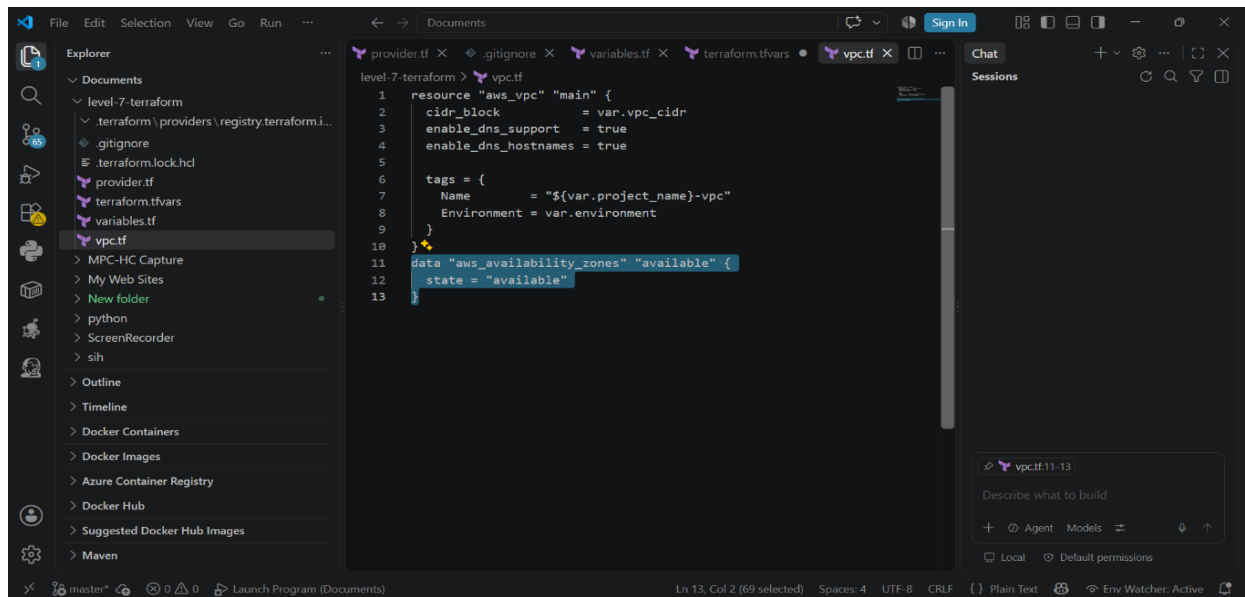
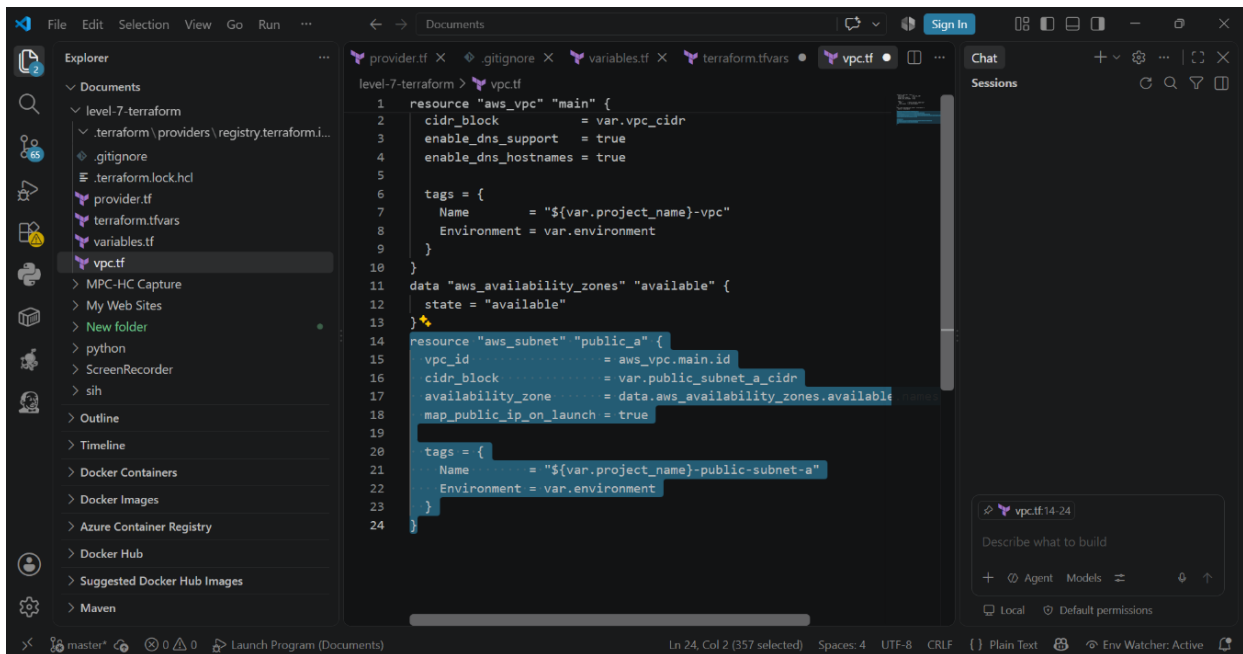


Figure 11. Configure VPC attributes and tags.

Step 11 : Define public subnet A

Implementation Action / What I did: Implement the define public subnet a in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows a Visual Studio Code editor with a Terraform project. The Explorer pane on the left shows the file structure, with 'vpctf' selected. The main editor displays the 'vpctf.tf' file, which contains the following Terraform configuration:

```
1 resource "aws_vpc" "main" {
2   cidr_block      = var.vpc_cidr
3   enable_dns_support = true
4   enable_dns_hostnames = true
5
6   tags = {
7     Name        = "${var.project_name}-vpc"
8     Environment = var.environment
9   }
10 }
11 data "aws_availability_zones" "available" {
12   state = "available"
13 }
14 resource "aws_subnet" "public_a" {
15   vpc_id            = aws_vpc.main.id
16   cidr_block        = var.public_subnet_a_cidr
17   availability_zone  = data.aws_availability_zones.available
18   map_public_ip_on_launch = true
19
20   tags = {
21     Name        = "${var.project_name}-public-subnet-a"
22     Environment = var.environment
23   }
24 }
```

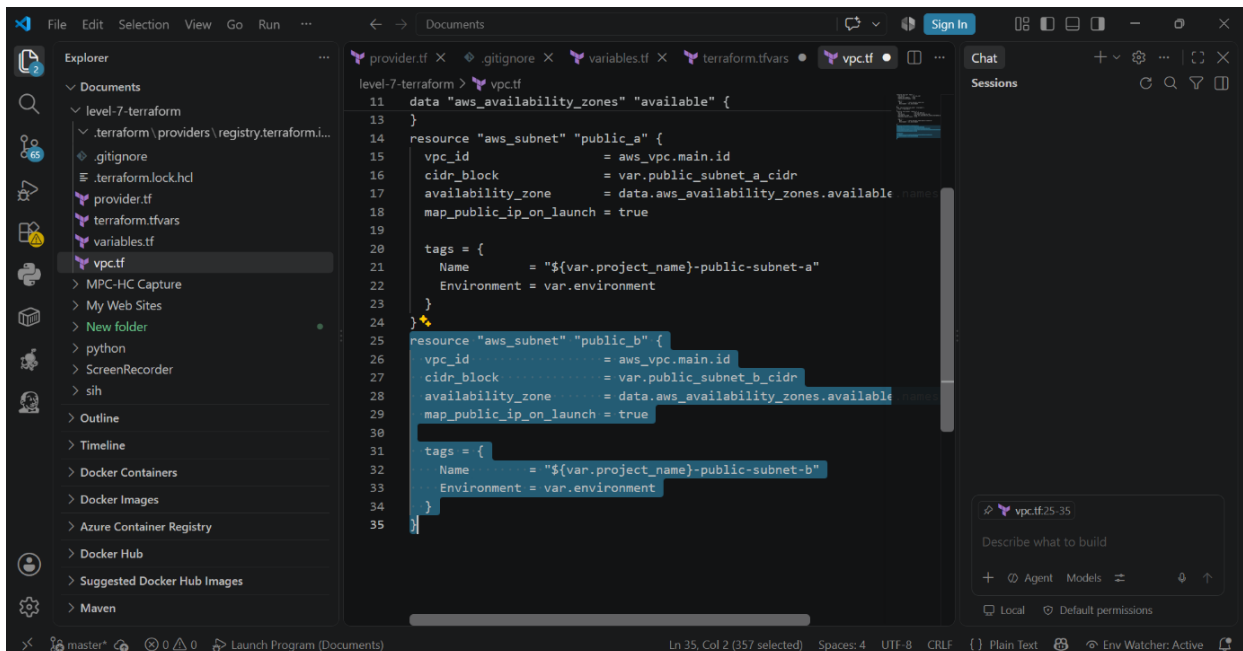
The status bar at the bottom indicates 'Ln 24, Col 2 (357 selected) Spaces: 4 UTF-8 CRLF {} Plain Text Env Watcher: Active'.

Figure 12. Level 7 implementation evidence – Define public subnet A.

Step 12 : Define public subnet B

Implementation Action / What I did: Implement the define public subnet b in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows a Visual Studio Code editor with a Terraform project. The Explorer pane on the left shows the file structure, with 'vpctf' selected. The main editor displays the 'vpctf.tf' file, which contains the following Terraform configuration:

```
11 data "aws_availability_zones" "available" {
12   state = "available"
13 }
14 resource "aws_subnet" "public_a" {
15   vpc_id            = aws_vpc.main.id
16   cidr_block        = var.public_subnet_a_cidr
17   availability_zone  = data.aws_availability_zones.available
18   map_public_ip_on_launch = true
19
20   tags = {
21     Name        = "${var.project_name}-public-subnet-a"
22     Environment = var.environment
23   }
24 }
25 resource "aws_subnet" "public_b" {
26   vpc_id            = aws_vpc.main.id
27   cidr_block        = var.public_subnet_b_cidr
28   availability_zone  = data.aws_availability_zones.available
29   map_public_ip_on_launch = true
30
31   tags = {
32     Name        = "${var.project_name}-public-subnet-b"
33     Environment = var.environment
34   }
35 }
```

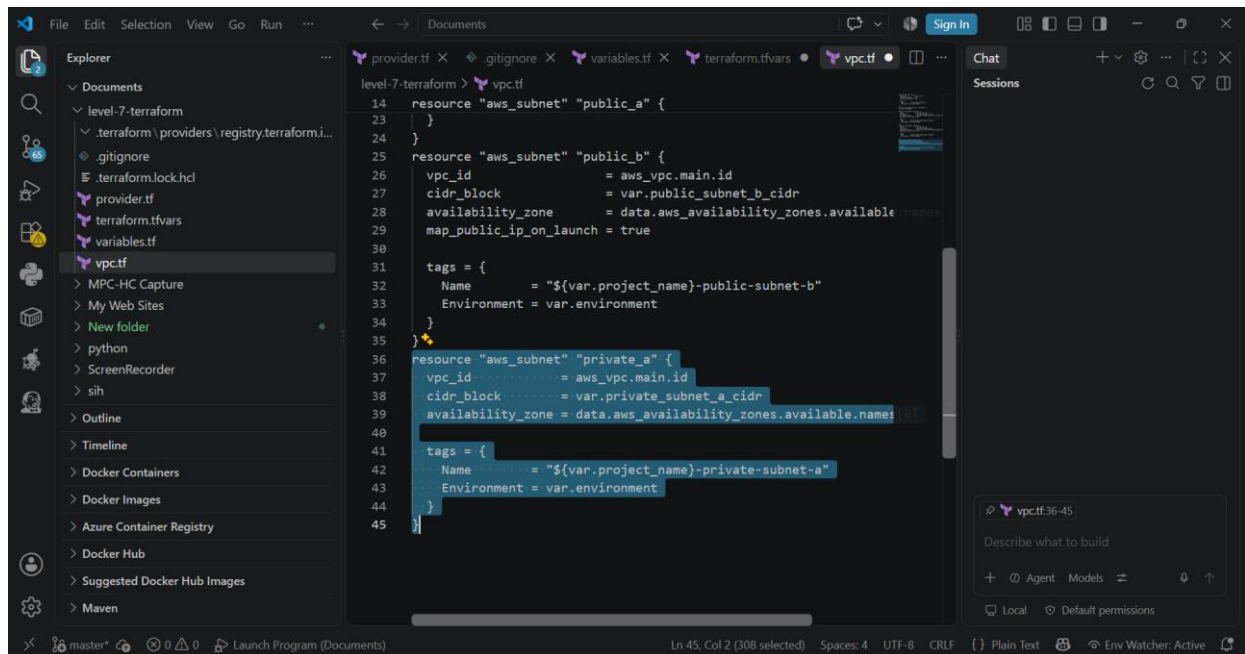
The status bar at the bottom indicates 'Ln 35, Col 2 (357 selected) Spaces: 4 UTF-8 CRLF {} Plain Text Env Watcher: Active'.

Figure 13. Level 7 implementation evidence – Define public subnet B.

Step 13 : Define private subnet A

Implementation Action / What I did: Implement the define private subnet a in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the Visual Studio Code editor with a Terraform configuration file named `vpctf` open. The file is located in the `Documents` directory. The configuration defines two AWS subnets: `public_b` and `private_a`. The `private_a` resource is highlighted in blue. The configuration includes the following code:

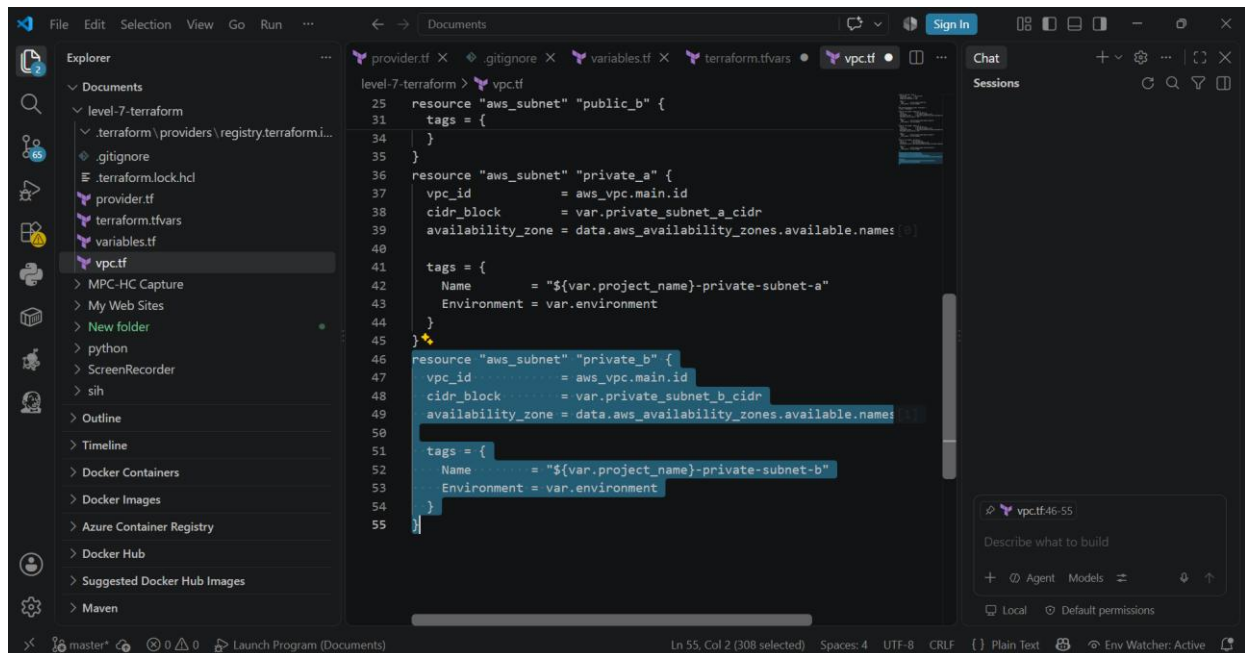
```
14 resource "aws_subnet" "public_a" {
23 }
24
25 resource "aws_subnet" "public_b" {
26   vpc_id           = aws_vpc.main.id
27   cidr_block       = var.public_subnet_b_cidr
28   availability_zone = data.aws_availability_zones.available.names[0]
29   map_public_ip_on_launch = true
30
31   tags = {
32     Name        = "${var.project_name}-public-subnet-b"
33     Environment = var.environment
34   }
35 }
36
37 resource "aws_subnet" "private_a" {
38   vpc_id           = aws_vpc.main.id
39   cidr_block       = var.private_subnet_a_cidr
40   availability_zone = data.aws_availability_zones.available.names[0]
41
42   tags = {
43     Name        = "${var.project_name}-private-subnet-a"
44     Environment = var.environment
45   }
46 }
```

Figure 14. Define private subnet A.

Step 14 : Define private subnet B

Implementation Action / What I did: Implement the define private subnet b in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the Visual Studio Code editor with the same Terraform configuration file named `vpctf` open. The file is located in the `Documents` directory. The configuration defines two AWS subnets: `private_a` and `private_b`. The `private_b` resource is highlighted in blue. The configuration includes the following code:

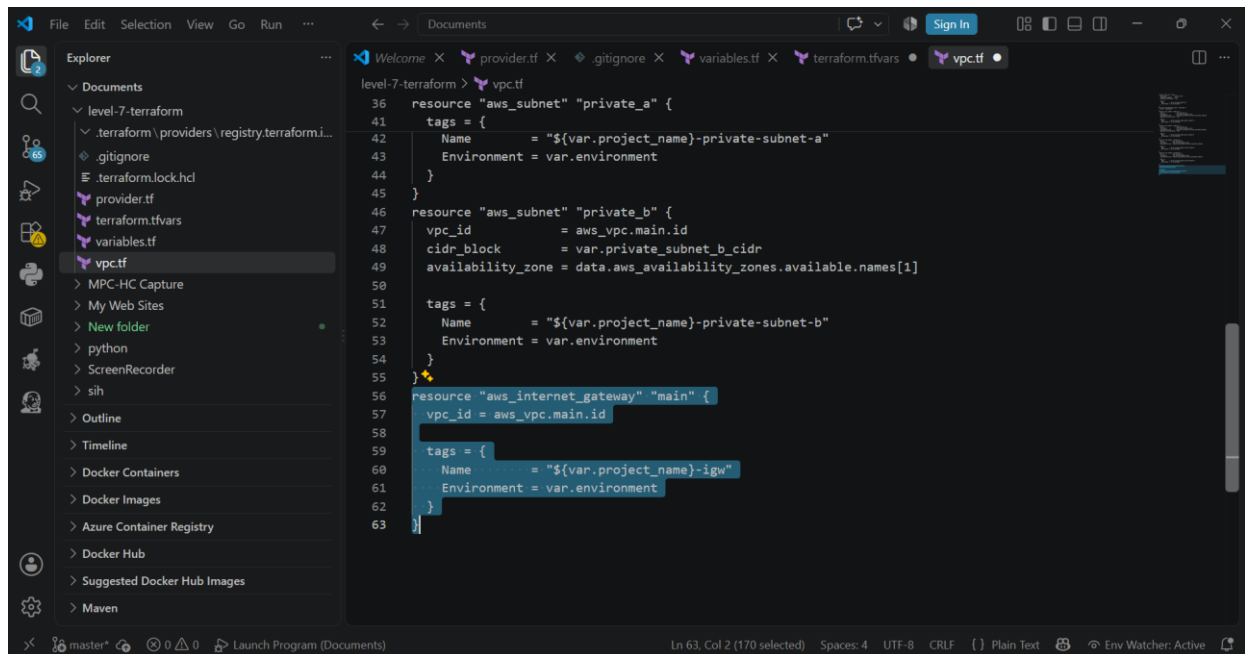
```
25 resource "aws_subnet" "public_b" {
31   tags = {
32     Name        = "${var.project_name}-public-subnet-b"
33     Environment = var.environment
34   }
35 }
36
37 resource "aws_subnet" "private_a" {
38   vpc_id           = aws_vpc.main.id
39   cidr_block       = var.private_subnet_a_cidr
40   availability_zone = data.aws_availability_zones.available.names[0]
41
42   tags = {
43     Name        = "${var.project_name}-private-subnet-a"
44     Environment = var.environment
45   }
46 }
47
48 resource "aws_subnet" "private_b" {
49   vpc_id           = aws_vpc.main.id
50   cidr_block       = var.private_subnet_b_cidr
51   availability_zone = data.aws_availability_zones.available.names[0]
52
53   tags = {
54     Name        = "${var.project_name}-private-subnet-b"
55     Environment = var.environment
56   }
57 }
```

Figure 15. Define private subnet B.

Step 15 : Complete private subnet definitions

Implementation Action / What I did: Implement the complete private subnet definitions in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



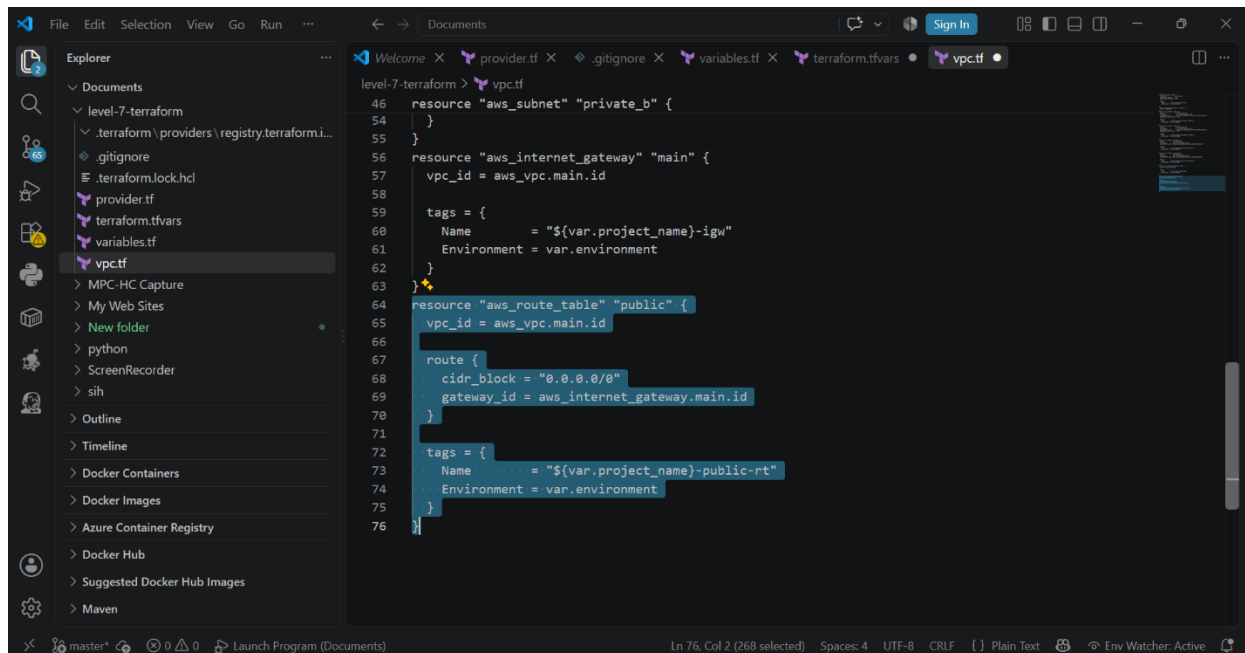
```
level-7-terraform > vpc.tf
36 resource "aws_subnet" "private_a" {
41   tags = {
42     Name      = "${var.project_name}-private-subnet-a"
43     Environment = var.environment
44   }
45 }
46 resource "aws_subnet" "private_b" {
47   vpc_id      = aws_vpc.main.id
48   cidr_block  = var.private_subnet_b_cidr
49   availability_zone = data.aws_availability_zones.available.names[1]
50 }
51 tags = {
52   Name      = "${var.project_name}-private-subnet-b"
53   Environment = var.environment
54 }
55 }
56 resource "aws_internet_gateway" "main" {
57   vpc_id = aws_vpc.main.id
58 }
59 tags = {
60   Name      = "${var.project_name}-igw"
61   Environment = var.environment
62 }
63 }
```

Figure 16. Complete private subnet definitions.

Step 16 : Continue subnet/network resources

Implementation Action / What I did: Implement the continue subnet/network resources in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding main Terraform network resource definition.



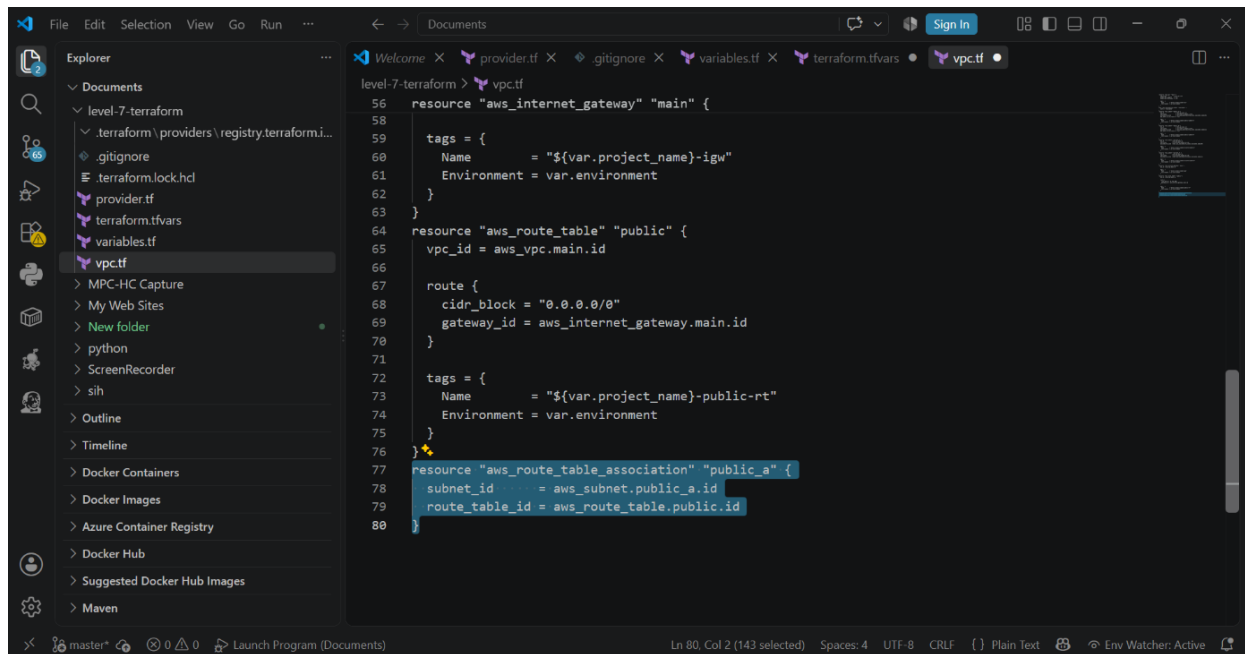
```
level-7-terraform > vpc.tf
46 resource "aws_subnet" "private_b" {
54 }
55 }
56 resource "aws_internet_gateway" "main" {
57   vpc_id = aws_vpc.main.id
58 }
59 tags = {
60   Name      = "${var.project_name}-igw"
61   Environment = var.environment
62 }
63 }
64 resource "aws_route_table" "public" {
65   vpc_id = aws_vpc.main.id
66 }
67 route {
68   cidr_block = "0.0.0.0/0"
69   gateway_id = aws_internet_gateway.main.id
70 }
71 }
72 tags = {
73   Name      = "${var.project_name}-public-rt"
74   Environment = var.environment
75 }
76 }
```

Figure 17. Continue subnet/network resources.

Step 17 : Define the Internet Gateway

Implementation Action / What I did: Implement the define the internet gateway in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



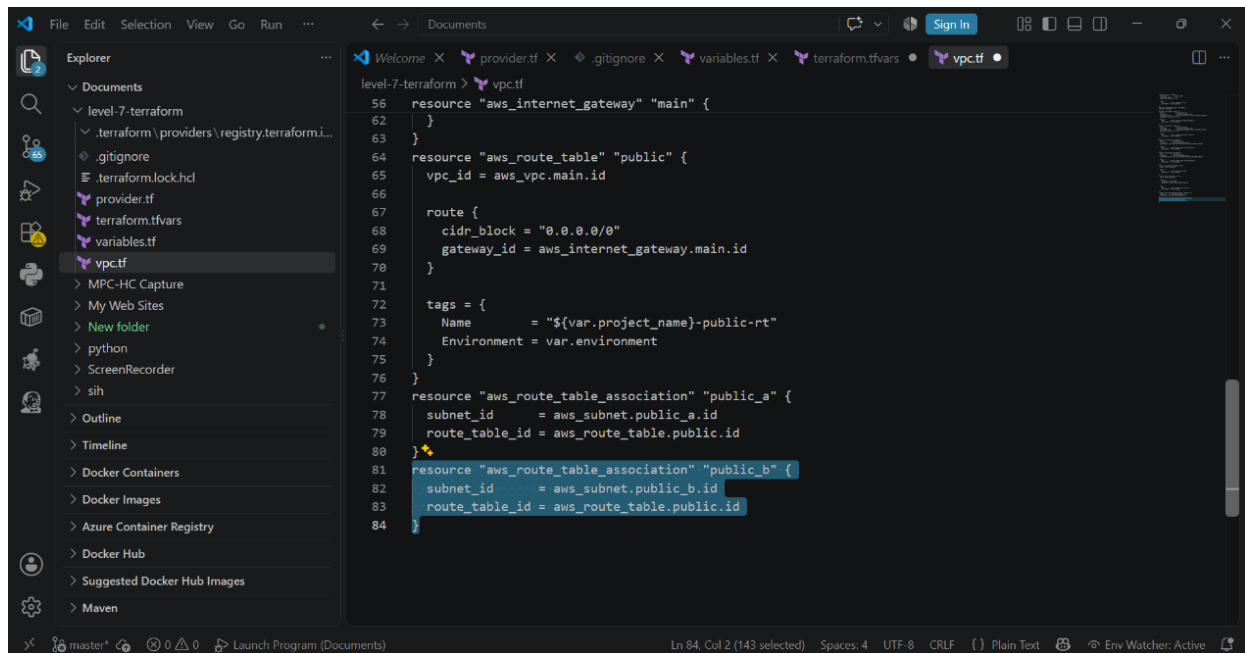
```
level-7-terraform > vpctf
56 resource "aws_internet_gateway" "main" {
57
58   tags = {
59     Name      = "${var.project_name}-igw"
60     Environment = var.environment
61   }
62 }
63
64 resource "aws_route_table" "public" {
65   vpc_id = aws_vpc.main.id
66
67   route {
68     cidr_block = "0.0.0.0/0"
69     gateway_id = aws_internet_gateway.main.id
70   }
71
72   tags = {
73     Name      = "${var.project_name}-public-rt"
74     Environment = var.environment
75   }
76 }
77 resource "aws_route_table_association" "public_a" {
78   subnet_id      = aws_subnet.public_a.id
79   route_table_id = aws_route_table.public.id
80 }
```

Figure 18. Define the Internet Gateway.

Step 18 : Configure Internet Gateway association/tags

Implementation Action / What I did: Implement the configure internet gateway association/tags in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



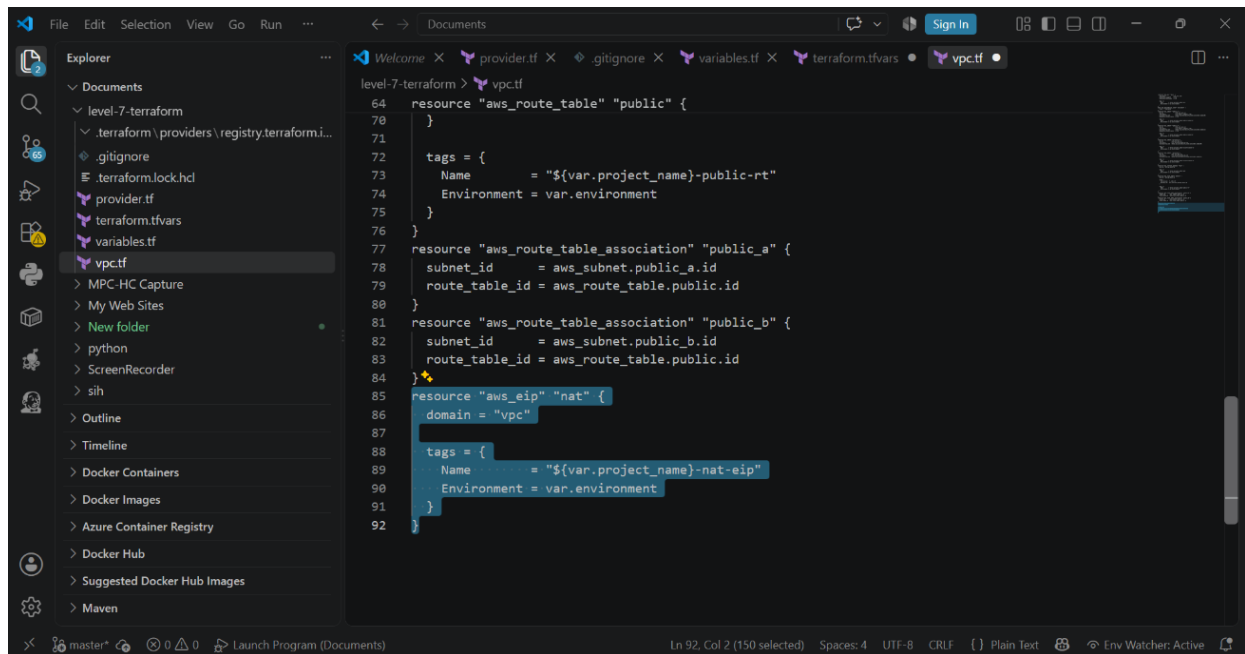
```
level-7-terraform > vpctf
56 resource "aws_internet_gateway" "main" {
57
58   tags = {
59     Name      = "${var.project_name}-igw"
60     Environment = var.environment
61   }
62 }
63
64 resource "aws_route_table" "public" {
65   vpc_id = aws_vpc.main.id
66
67   route {
68     cidr_block = "0.0.0.0/0"
69     gateway_id = aws_internet_gateway.main.id
70   }
71
72   tags = {
73     Name      = "${var.project_name}-public-rt"
74     Environment = var.environment
75   }
76 }
77 resource "aws_route_table_association" "public_a" {
78   subnet_id      = aws_subnet.public_a.id
79   route_table_id = aws_route_table.public.id
80 }
81 resource "aws_route_table_association" "public_b" {
82   subnet_id      = aws_subnet.public_b.id
83   route_table_id = aws_route_table.public.id
84 }
```

Figure 19. Configure Internet Gateway association/tags.

Step 19 : Define public route table

Implementation Action / What I did: Implement the define public route table in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the Visual Studio Code editor with a Terraform project. The Explorer pane on the left shows the file structure, with 'vpc.tf' selected. The main editor displays the content of 'vpc.tf', which defines an AWS route table and its associations. The code is as follows:

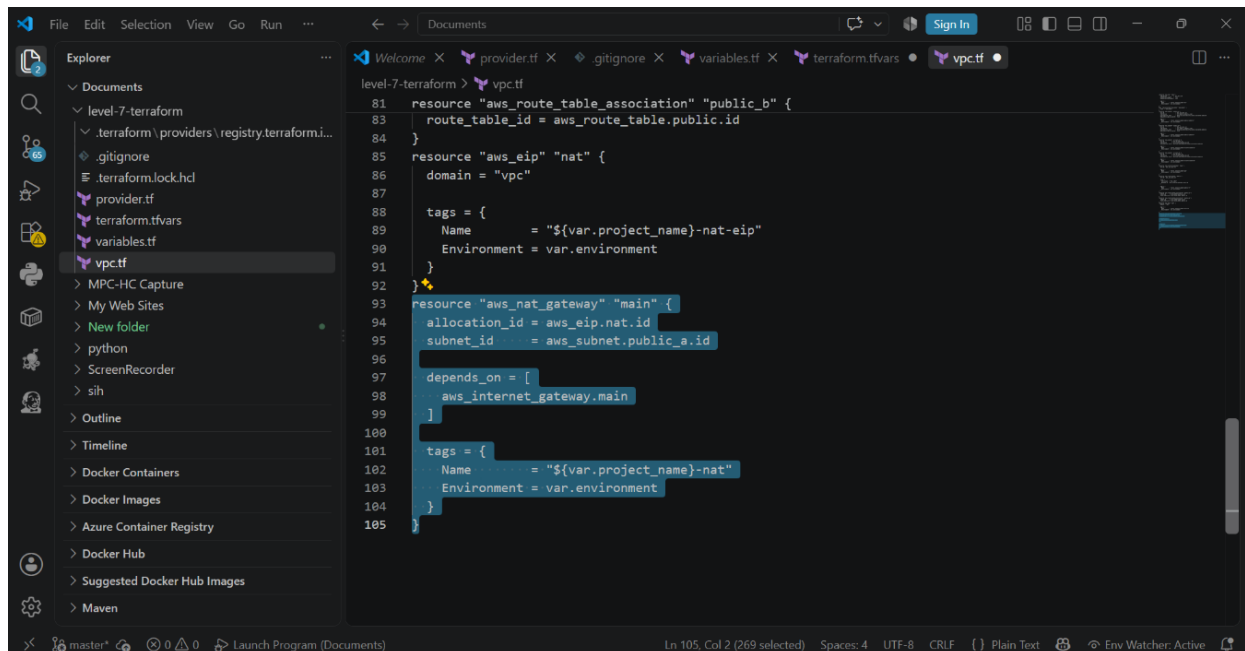
```
64 resource "aws_route_table" "public" {
70 }
71
72 tags = {
73   Name = "${var.project_name}-public-rt"
74   Environment = var.environment
75 }
76 }
77 resource "aws_route_table_association" "public_a" {
78   subnet_id = aws_subnet.public_a.id
79   route_table_id = aws_route_table.public.id
80 }
81 resource "aws_route_table_association" "public_b" {
82   subnet_id = aws_subnet.public_b.id
83   route_table_id = aws_route_table.public.id
84 }
85 resource "aws_eip" "nat" {
86   domain = "vpc"
87
88   tags = {
89     Name = "${var.project_name}-nat-eip"
90     Environment = var.environment
91   }
92 }
```

Figure 20. Define public route table.

Step 20 : Associate public subnets with routes

Implementation Action / What I did: Implement the associate public subnets with routes in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the Visual Studio Code editor with a Terraform project. The Explorer pane on the left shows the file structure, with 'vpc.tf' selected. The main editor displays the content of 'vpc.tf', which defines an AWS NAT gateway and its dependencies. The code is as follows:

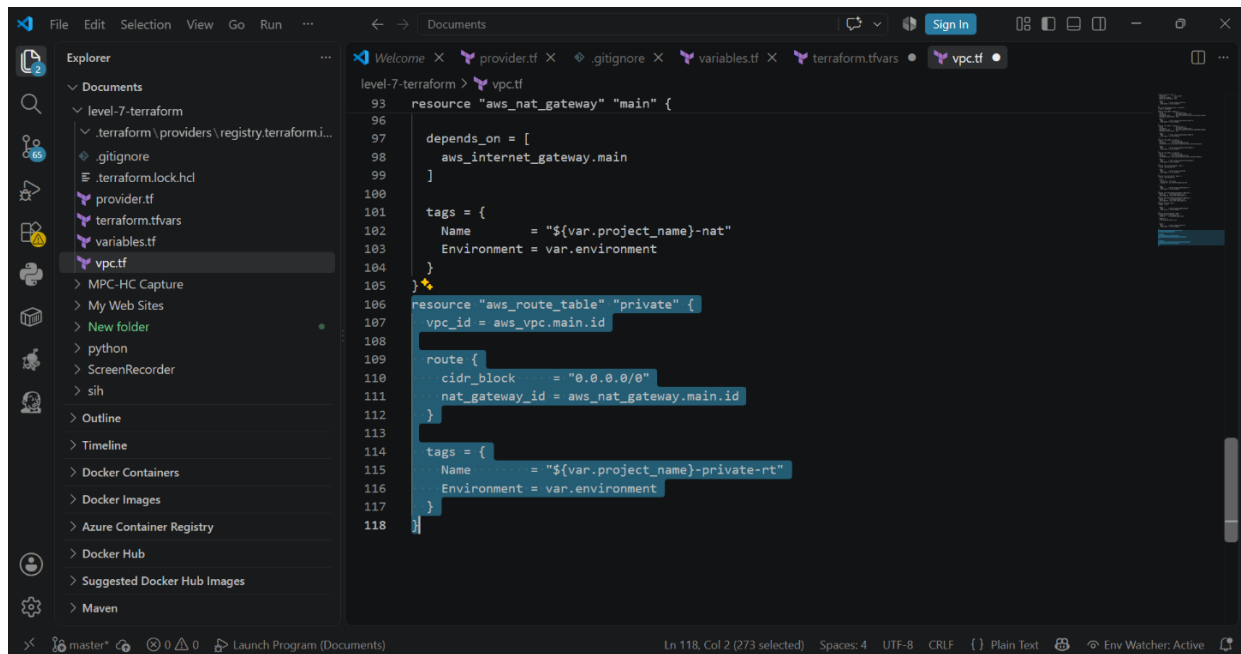
```
81 resource "aws_route_table_association" "public_b" {
83   route_table_id = aws_route_table.public.id
84 }
85 resource "aws_eip" "nat" {
86   domain = "vpc"
87
88   tags = {
89     Name = "${var.project_name}-nat-eip"
90     Environment = var.environment
91   }
92 }
93 resource "aws_nat_gateway" "main" {
94   allocation_id = aws_eip.nat.id
95   subnet_id = aws_subnet.public_a.id
96
97   depends_on = [
98     aws_internet_gateway.main
99   ]
100
101   tags = {
102     Name = "${var.project_name}-nat"
103     Environment = var.environment
104   }
105 }
```

Figure 21. Associate public subnets with routes.

Step 21 : Define private route table

Implementation Action / What I did: Implement the define private route table in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows a Visual Studio Code editor with a Terraform project. The Explorer pane on the left shows the file structure, with 'vpc.tf' selected. The main editor displays the content of 'vpc.tf', which defines an AWS NAT gateway and a private route table. The NAT gateway is named 'main' and depends on 'aws_internet_gateway.main'. The private route table is named 'private' and is associated with 'aws_vpc.main.id'. It has a single route with a CIDR block of '0.0.0.0/0' and is associated with 'aws_nat_gateway.main.id'. Both resources have tags for project name and environment.

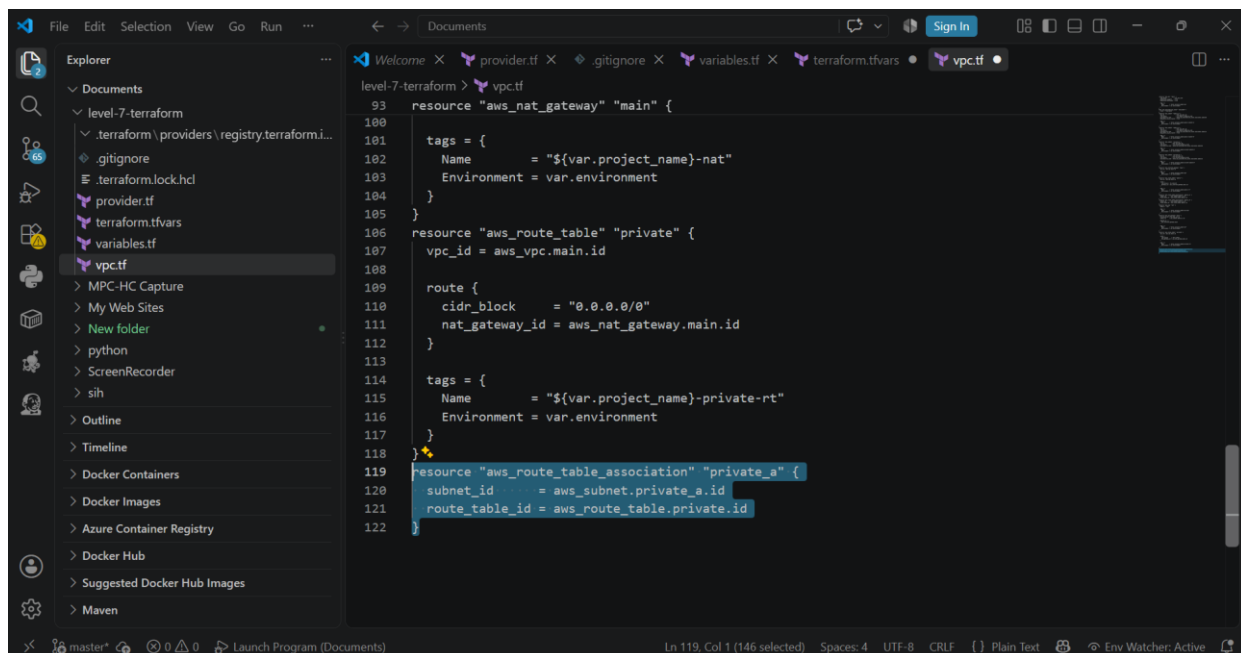
```
93 resource "aws_nat_gateway" "main" {
94
95     depends_on = [
96         aws_internet_gateway.main
97     ]
98
99     tags = {
100         Name      = "${var.project_name}-nat"
101         Environment = var.environment
102     }
103 }
104
105 resource "aws_route_table" "private" {
106     vpc_id = aws_vpc.main.id
107
108     route {
109         cidr_block      = "0.0.0.0/0"
110         nat_gateway_id = aws_nat_gateway.main.id
111     }
112
113     tags = {
114         Name      = "${var.project_name}-private-rt"
115         Environment = var.environment
116     }
117 }
118 }
```

Figure 22. Define private route table.

Step 22 : Associate private subnets with private routes

Implementation Action / What I did: Implement the associate private subnets with private routes in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the same Visual Studio Code editor with the 'vpc.tf' file. The configuration now includes an additional resource, 'aws_route_table_association', named 'private_a'. This resource associates the 'private' route table with 'aws_subnet.private_a.id'. The route table ID is referenced as 'aws_route_table.private.id'.

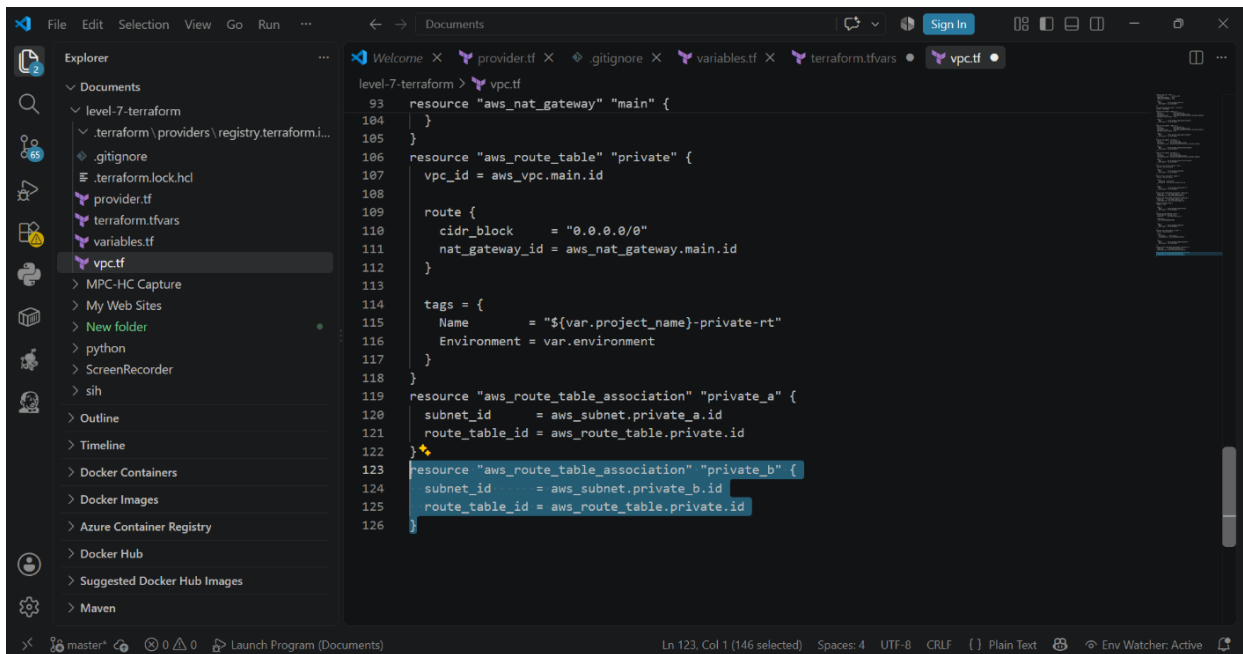
```
100     tags = {
101         Name      = "${var.project_name}-nat"
102         Environment = var.environment
103     }
104 }
105
106 resource "aws_route_table" "private" {
107     vpc_id = aws_vpc.main.id
108
109     route {
110         cidr_block      = "0.0.0.0/0"
111         nat_gateway_id = aws_nat_gateway.main.id
112     }
113
114     tags = {
115         Name      = "${var.project_name}-private-rt"
116         Environment = var.environment
117     }
118 }
119
120 resource "aws_route_table_association" "private_a" {
121     subnet_id      = aws_subnet.private_a.id
122     route_table_id = aws_route_table.private.id
123 }
```

Figure 23. Associate private subnets with private routes.

Step 23 : Review initialized Terraform project state

Implementation Action / What I did: Implement the review initialized terraform project state in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the Visual Studio Code editor with the Terraform project state. The Explorer pane on the left shows the project structure, including the `vpc.tf` file. The main editor displays the Terraform configuration for the `vpc.tf` file, which defines the `aws_nat_gateway`, `aws_route_table`, and `aws_route_table_association` resources. The code is as follows:

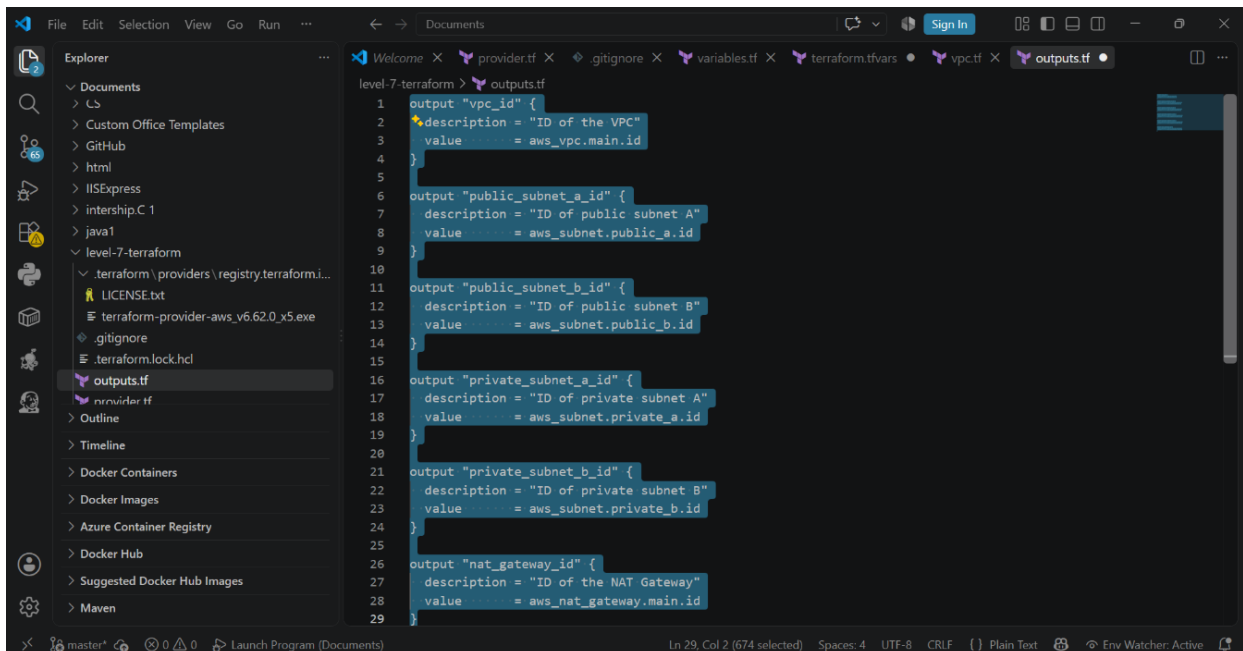
```
93 resource "aws_nat_gateway" "main" {
94 }
95
96 resource "aws_route_table" "private" {
97   vpc_id = aws_vpc.main.id
98
99   route {
100     cidr_block = "0.0.0.0/0"
101     nat_gateway_id = aws_nat_gateway.main.id
102   }
103
104   tags = {
105     Name = "${var.project_name}-private-rt"
106     Environment = var.environment
107   }
108 }
109
110 resource "aws_route_table_association" "private_a" {
111   subnet_id = aws_subnet.private_a.id
112   route_table_id = aws_route_table.private.id
113 }
114
115 resource "aws_route_table_association" "private_b" {
116   subnet_id = aws_subnet.private_b.id
117   route_table_id = aws_route_table.private.id
118 }
```

Figure 24. Review initialized Terraform project state.

Step 24 : Define Terraform outputs

Implementation Action / What I did: Implement the define terraform outputs in the Terraform project and keep dependencies connected to the VPC variables/resources.

What the screenshot proves: The screenshot shows the corresponding Terraform network resource definition.



The screenshot shows the Visual Studio Code editor with the Terraform project state. The Explorer pane on the left shows the project structure, including the `outputs.tf` file. The main editor displays the Terraform configuration for the `outputs.tf` file, which defines the `output` resources. The code is as follows:

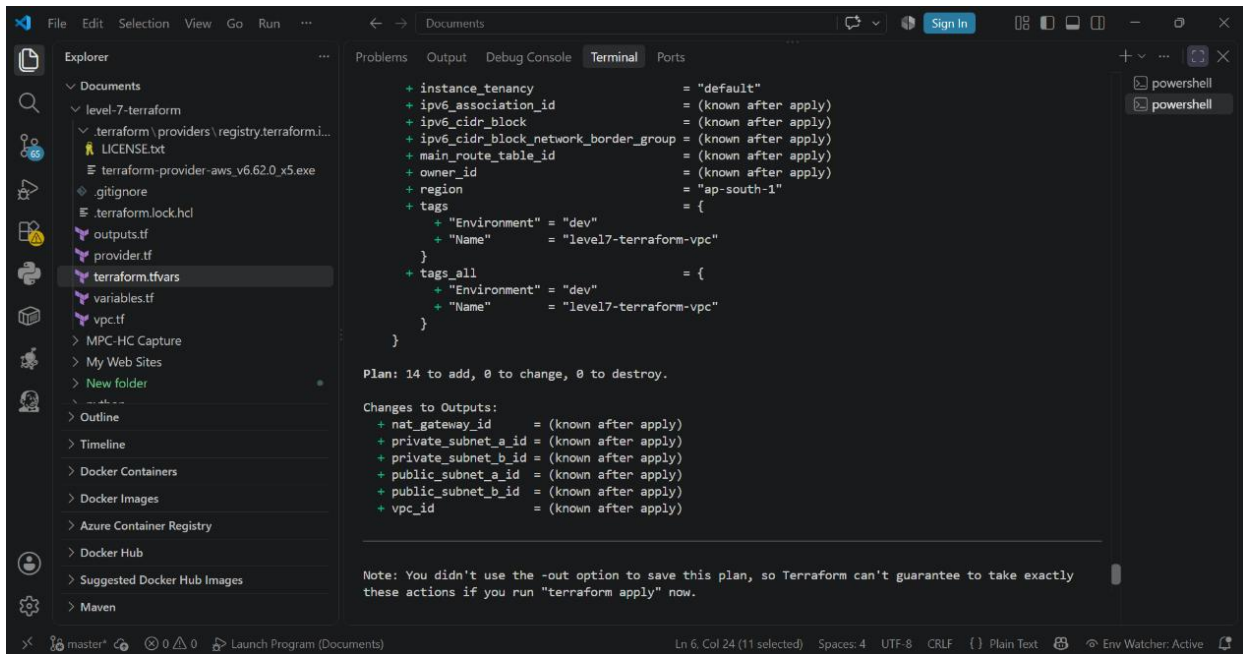
```
1 output "vpc_id" {
2   description = "ID of the VPC"
3   value = aws_vpc.main.id
4 }
5
6 output "public_subnet_a_id" {
7   description = "ID of public subnet A"
8   value = aws_subnet.public_a.id
9 }
10
11 output "public_subnet_b_id" {
12   description = "ID of public subnet B"
13   value = aws_subnet.public_b.id
14 }
15
16 output "private_subnet_a_id" {
17   description = "ID of private subnet A"
18   value = aws_subnet.private_a.id
19 }
20
21 output "private_subnet_b_id" {
22   description = "ID of private subnet B"
23   value = aws_subnet.private_b.id
24 }
25
26 output "nat_gateway_id" {
27   description = "ID of the NAT Gateway"
28   value = aws_nat_gateway.main.id
29 }
```

Figure 25. Define Terraform outputs.

Step 25 : Review Terraform plan

Implementation Action / What I did: Inspect the planned resource changes before provisioning.

What the screenshot proves: Terraform has generated the planned infrastructure changes.



```
File Edit Selection View Go Run ...
Documents
level-7-terraform
  terraform\providers\registry.terraform.i...
  LICENSE.txt
  terraform-provider-aws_v6.62.0_x5.exe
  .gitignore
  .terraform.lock.hcl
  outputs.tf
  provider.tf
  terraform.tfvars
  variables.tf
  vpc.tf
  MPC-HC Capture
  My Web Sites
  New folder
  Outline
  Timeline
  Docker Containers
  Docker Images
  Azure Container Registry
  Docker Hub
  Suggested Docker Hub Images
  Maven

Problems Output Debug Console Terminal Ports
+ instance_tenancy = "default"
+ ipv6_association_id = (known after apply)
+ ipv6_cidr_block = (known after apply)
+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id = (known after apply)
+ owner_id = (known after apply)
+ region = "ap-south-1"
+ tags = {
+   "Environment" = "dev"
+   "Name" = "level7-terraform-vpc"
+ }
+ tags_all = {
+   "Environment" = "dev"
+   "Name" = "level7-terraform-vpc"
+ }
}

Plan: 14 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ nat_gateway_id = (known after apply)
+ private_subnet_a_id = (known after apply)
+ private_subnet_b_id = (known after apply)
+ public_subnet_a_id = (known after apply)
+ public_subnet_b_id = (known after apply)
+ vpc_id = (known after apply)

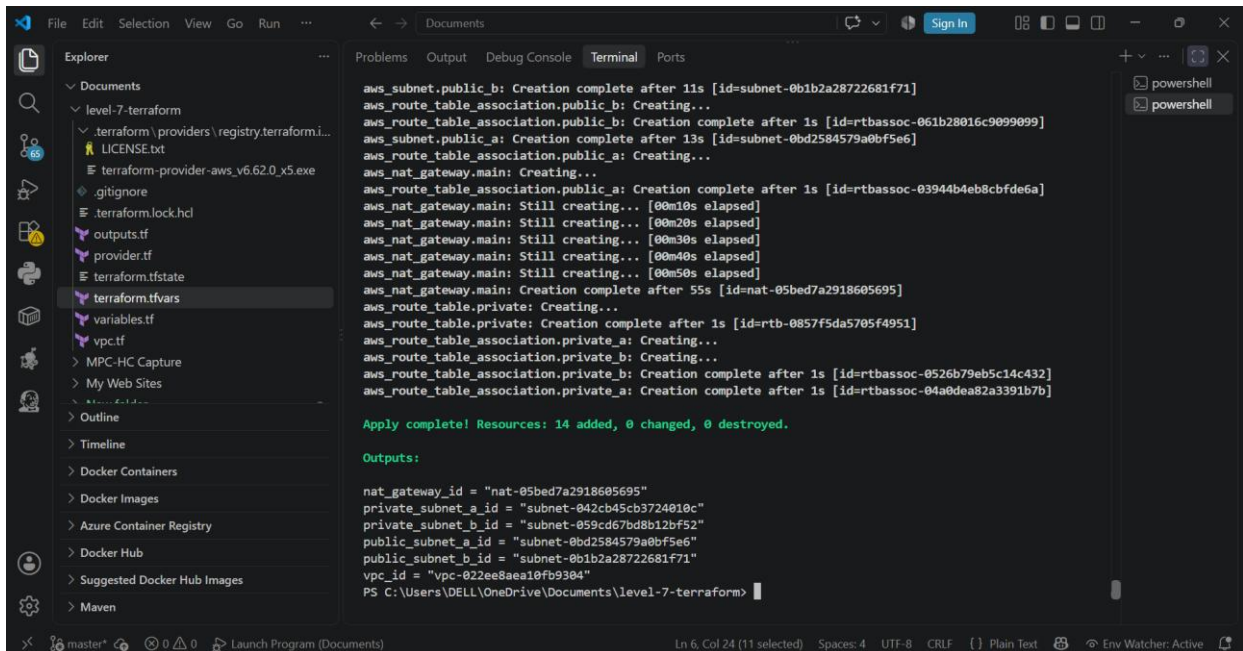
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

Figure 26. Review Terraform plan.

Step 26 : Apply the VPC infrastructure

Implementation Action / What I did: Run terraform apply and confirm resource creation.

What the screenshot proves: Terraform creation output is shown.



```
File Edit Selection View Go Run ...
Documents
level-7-terraform
  terraform\providers\registry.terraform.i...
  LICENSE.txt
  terraform-provider-aws_v6.62.0_x5.exe
  .gitignore
  .terraform.lock.hcl
  outputs.tf
  provider.tf
  terraform.tfstate
  terraform.tfvars
  variables.tf
  vpc.tf
  MPC-HC Capture
  My Web Sites
  Outline
  Timeline
  Docker Containers
  Docker Images
  Azure Container Registry
  Docker Hub
  Suggested Docker Hub Images
  Maven

Problems Output Debug Console Terminal Ports
aws_subnet.public_b: Creation complete after 11s [id=subnet-0b1b2a28722681f71]
aws_route_table_association.public_b: Creating...
aws_route_table_association.public_b: Creation complete after 1s [id=rtbassoc-061b28016c9099099]
aws_subnet.public_a: Creation complete after 13s [id=subnet-0bd2584579a0bf5e6]
aws_route_table_association.public_a: Creating...
aws_nat_gateway.main: Creating...
aws_route_table_association.public_a: Creation complete after 1s [id=rtbassoc-03944b4eb8cbfde6a]
aws_nat_gateway.main: Still creating... [00m10s elapsed]
aws_nat_gateway.main: Still creating... [00m20s elapsed]
aws_nat_gateway.main: Still creating... [00m30s elapsed]
aws_nat_gateway.main: Still creating... [00m40s elapsed]
aws_nat_gateway.main: Still creating... [00m50s elapsed]
aws_nat_gateway.main: Creation complete after 55s [id=nat-05bed7a2918605695]
aws_route_table.private: Creating...
aws_route_table.private: Creation complete after 1s [id=rtb-0857f5da5705f4951]
aws_route_table_association.private_a: Creating...
aws_route_table_association.private_b: Creating...
aws_route_table_association.private_b: Creation complete after 1s [id=rtbassoc-0526b79eb5c14c432]
aws_route_table_association.private_a: Creation complete after 1s [id=rtbassoc-04a0dea82a3391b7b]

Apply complete! Resources: 14 added, 0 changed, 0 destroyed.

Outputs:
nat_gateway_id = "nat-05bed7a2918605695"
private_subnet_a_id = "subnet-042cb45cb3724010c"
private_subnet_b_id = "subnet-059cd67bd8b12bf52"
public_subnet_a_id = "subnet-0bd2584579a0bf5e6"
public_subnet_b_id = "subnet-0b1b2a28722681f71"
vpc_id = "vpc-022ee8aea10fb9304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure 27. Apply the VPC infrastructure.

Phase 3 – AWS Network Verification

Step 27 : Verify the VPC in AWS

Implementation Action / What I did: Open the AWS VPC console and verify the verify the vpc in aws created by Terraform.

What the screenshot proves: The AWS console provides evidence that the network resource exists.

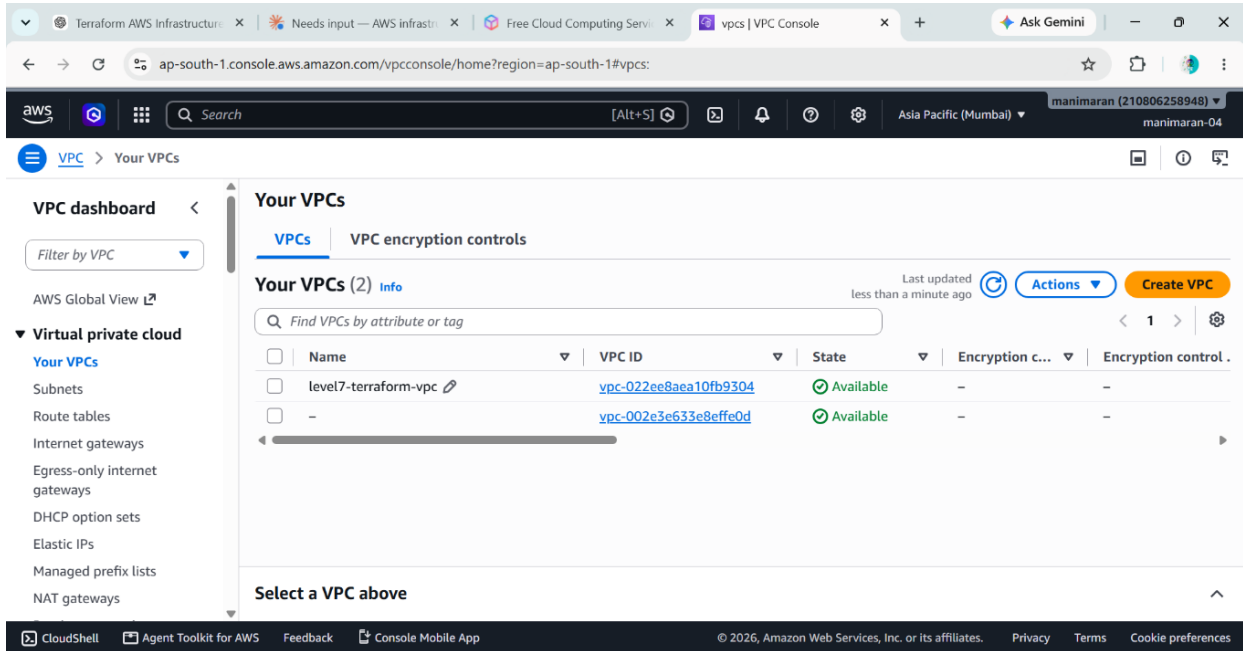


Figure 28. Verify the VPC in AWS.

Step 28 : Verify public and private subnets

Implementation Action / What I did: Open the AWS VPC console and verify the verify public and private subnets created by Terraform.

What the screenshot proves: The AWS console provides evidence that the network resource exists.

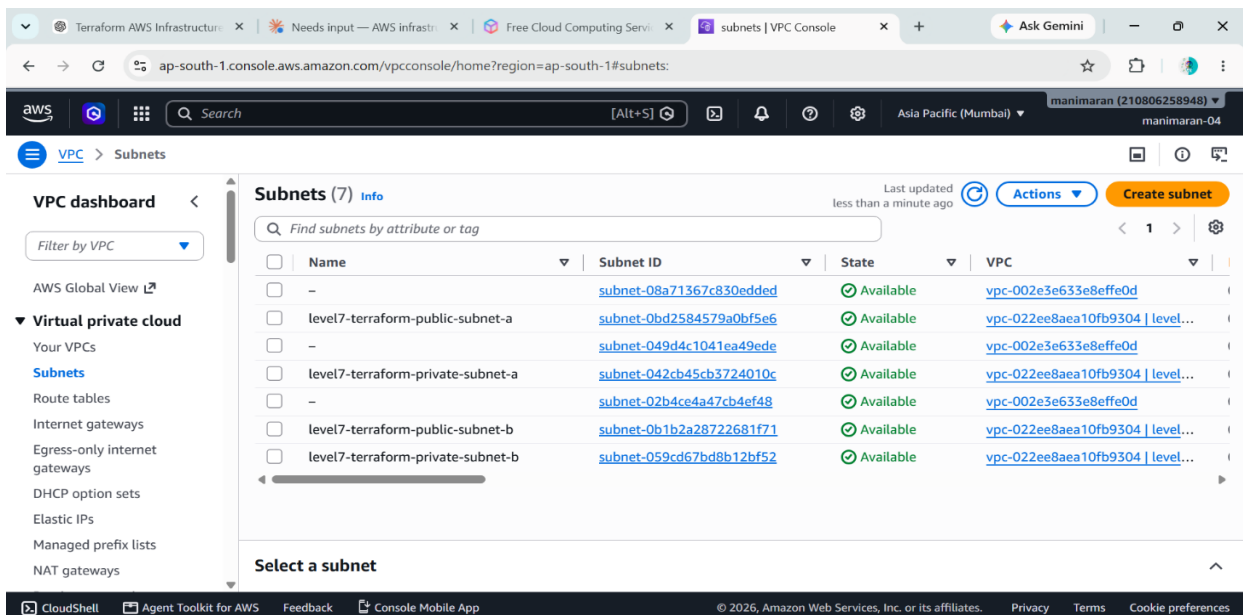


Figure 29. Verify public and private subnets.

Step 29 : Verify route tables

Implementation Action / What I did: Open the AWS VPC console and verify the verify route tables created by Terraform.

What the screenshot proves: The AWS console provides evidence that the network resource exists.

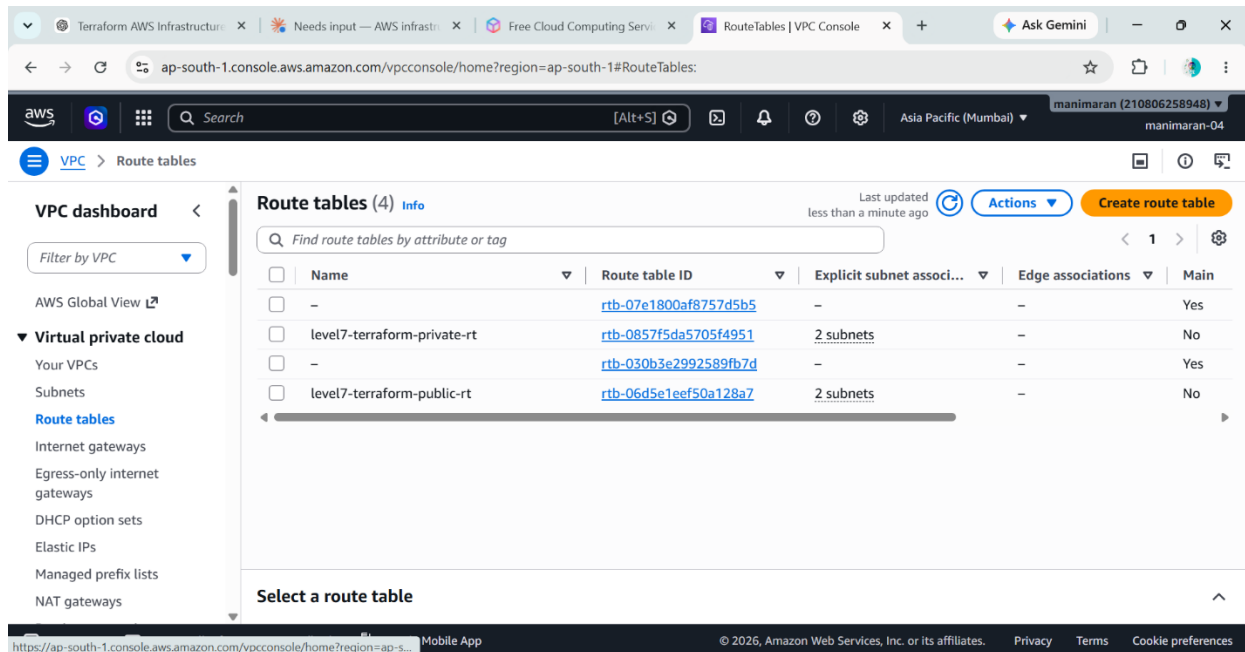


Figure 30. Verify route tables.

Step 30 : Verify the Internet Gateway

Implementation Action / What I did: Open the AWS VPC console and verify the verify the internet gateway created by Terraform.

What the screenshot proves: The AWS console provides evidence that the network resource exists.

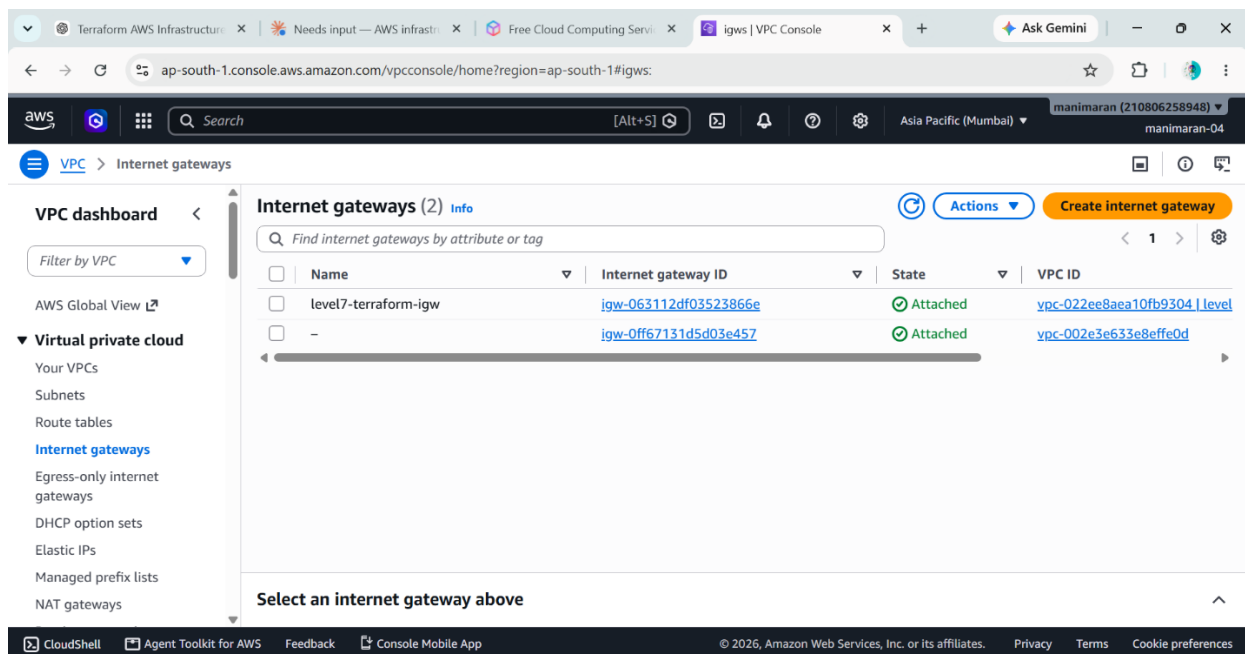


Figure 31. Verify the Internet Gateway.

Step 31 : Verify the NAT Elastic IP

Implementation Action / What I did: Open the AWS VPC console and verify the verify the nat elastic ip created by Terraform.

What the screenshot proves: The AWS console provides evidence that the network resource exists.

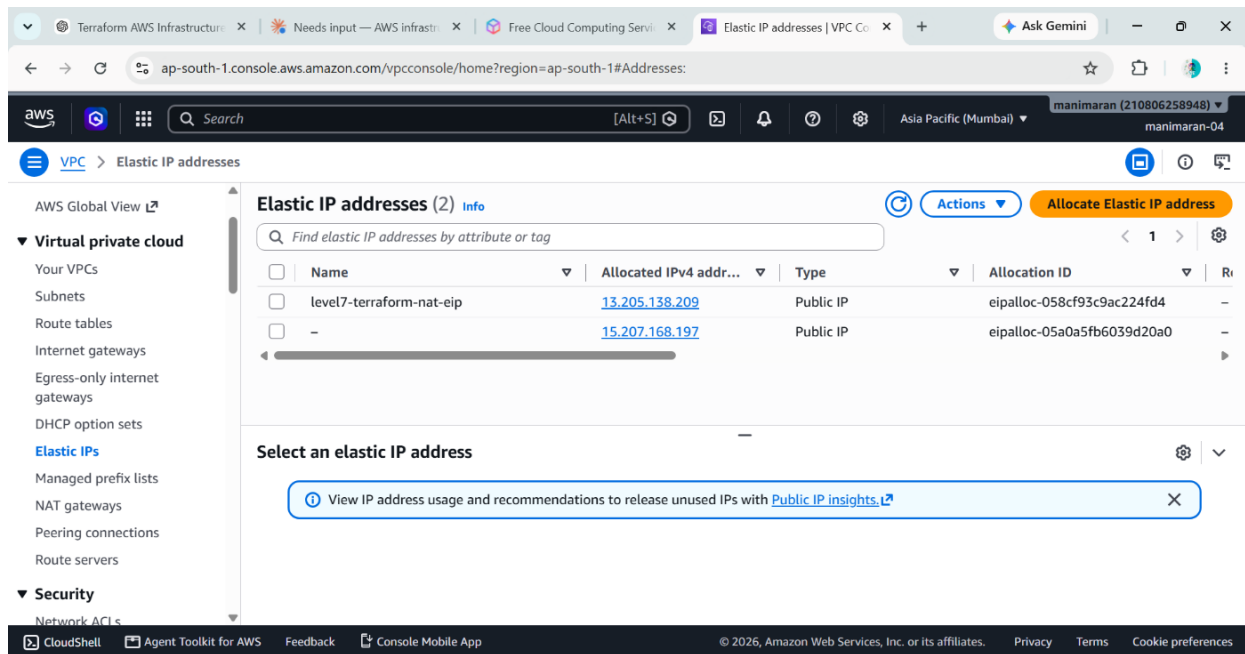


Figure 32. Verify the NAT Elastic IP.

Step 32 : Verify the NAT Gateway

Implementation Action / What I did: Open the AWS VPC console and verify the verify the nat gateway created by Terraform.

What the screenshot proves: The AWS console provides evidence that the network resource exists.

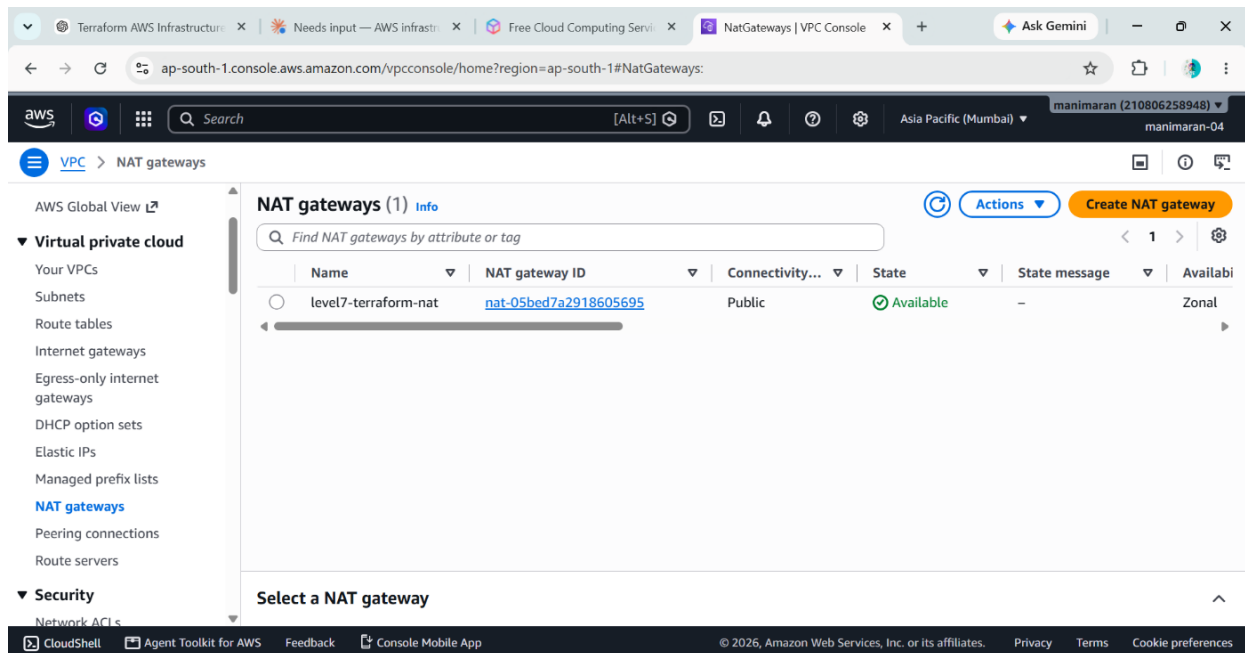


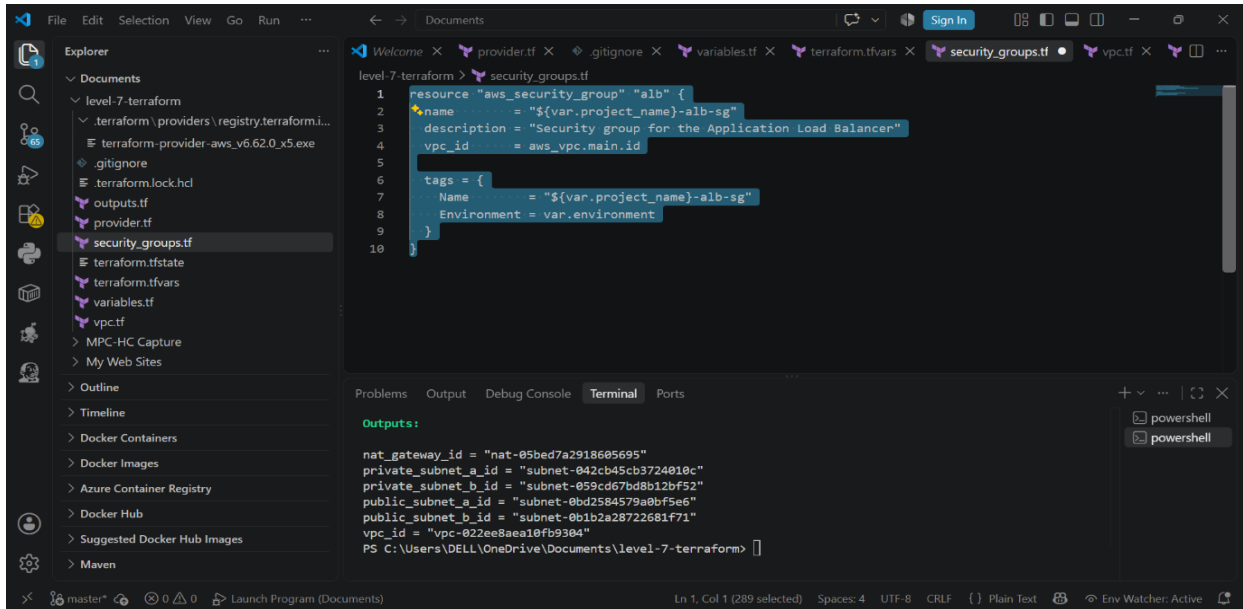
Figure 33. Verify the NAT Gateway.

Phase 4 – Security Groups

Step 33 : Create the ALB security group

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.



The screenshot shows the VS Code editor with the Terraform configuration file `security_groups.tf` open. The configuration defines an `aws_security_group` resource named `alb`. The `name` is set to `"${var.project_name}-alb-sg"`, the `description` is `"Security group for the Application Load Balancer"`, and the `vpc_id` is `aws_vpc.main.id`. The `tags` block includes `Name = "${var.project_name}-alb-sg"` and `Environment = var.environment`. The `Outputs` section shows the results of the `terraform apply` command, including `nat_gateway_id`, `private_subnet_a_id`, `private_subnet_b_id`, `public_subnet_a_id`, `public_subnet_b_id`, and `vpc_id`.

```
1 resource "aws_security_group" "alb" {
2   name           = "${var.project_name}-alb-sg"
3   description    = "Security group for the Application Load Balancer"
4   vpc_id         = aws_vpc.main.id
5
6   tags = {
7     Name       = "${var.project_name}-alb-sg"
8     Environment = var.environment
9   }
10 }
```

Outputs:

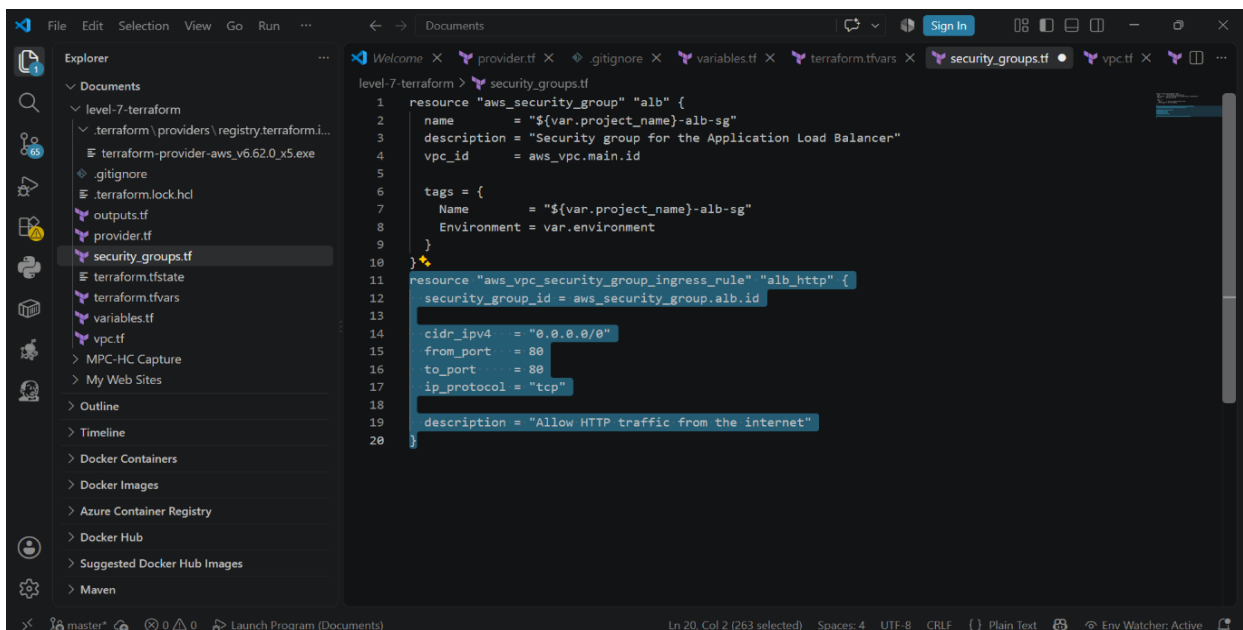
```
nat_gateway_id = "nat-05bed7a2918605695"
private_subnet_a_id = "subnet-042cb45cb3724010c"
private_subnet_b_id = "subnet-059cd67bd8b12bf52"
public_subnet_a_id = "subnet-0bd2584579a0bf5e6"
public_subnet_b_id = "subnet-0b1b2a28722681f71"
vpc_id = "vpc-022ee8aea10fb9304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure 34. Create the ALB security group.

Step 34 : Allow HTTP traffic to the ALB

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.



The screenshot shows the VS Code editor with the Terraform configuration file `security_groups.tf` open. The configuration defines an `aws_vpc_security_group_ingress_rule` resource named `alb_http`. The `security_group_id` is `aws_security_group.alb.id`, the `cidr_ipv4` is `"0.0.0.0/0"`, the `from_port` is `80`, the `to_port` is `80`, and the `ip_protocol` is `"tcp"`. The `description` is `"Allow HTTP traffic from the internet"`.

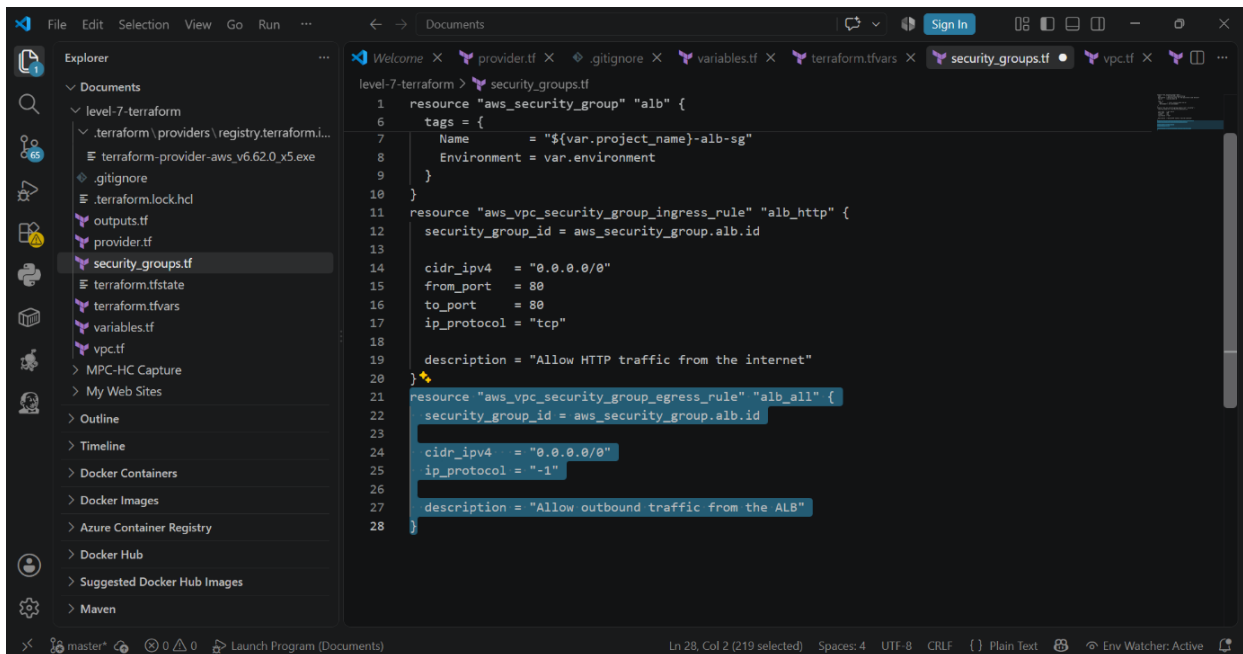
```
1 resource "aws_security_group" "alb" {
2   name           = "${var.project_name}-alb-sg"
3   description    = "Security group for the Application Load Balancer"
4   vpc_id         = aws_vpc.main.id
5
6   tags = {
7     Name       = "${var.project_name}-alb-sg"
8     Environment = var.environment
9   }
10 }
11 resource "aws_vpc_security_group_ingress_rule" "alb_http" {
12   security_group_id = aws_security_group.alb.id
13
14   cidr_ipv4 = "0.0.0.0/0"
15   from_port = 80
16   to_port   = 80
17   ip_protocol = "tcp"
18
19   description = "Allow HTTP traffic from the internet"
20 }
```

Figure 35. Allow HTTP traffic to the ALB.

Step 35 : Configure ALB egress

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.



The screenshot shows a VS Code editor with a Terraform configuration file named `security_groups.tf`. The file is part of a project named `level-7-terraform`. The configuration defines an `aws_security_group` resource named `alb` and two `aws_vpc_security_group_ingress_rule` resources: `alb_http` and `alb_all`. The `alb` resource is configured with the following properties:

```
resource "aws_security_group" "alb" {
  tags = {
    Name = "${var.project_name}-alb-sg"
    Environment = var.environment
  }
}
```

The `alb_http` resource is configured with the following properties:

```
resource "aws_vpc_security_group_ingress_rule" "alb_http" {
  security_group_id = aws_security_group.alb.id

  cidr_ipv4 = "0.0.0.0/0"
  from_port = 80
  to_port = 80
  ip_protocol = "tcp"

  description = "Allow HTTP traffic from the internet"
}
```

The `alb_all` resource is configured with the following properties:

```
resource "aws_vpc_security_group_egress_rule" "alb_all" {
  security_group_id = aws_security_group.alb.id

  cidr_ipv4 = "0.0.0.0/0"
  ip_protocol = "-1"

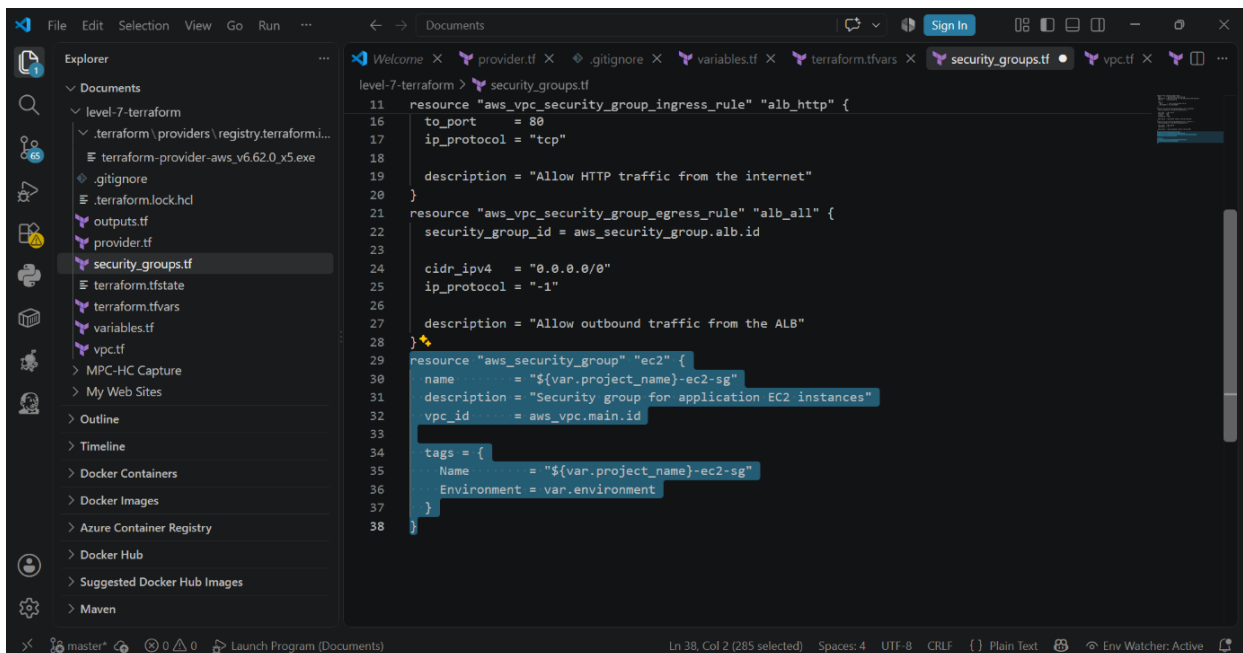
  description = "Allow outbound traffic from the ALB"
}
```

Figure 36. Configure ALB egress.

Step 36 : Create the EC2 application security group

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.



The screenshot shows a VS Code editor with a Terraform configuration file named `security_groups.tf`. The file is part of a project named `level-7-terraform`. The configuration defines an `aws_vpc_security_group_ingress_rule` resource named `alb_http` and an `aws_vpc_security_group_egress_rule` resource named `alb_all`. The `alb_http` resource is configured with the following properties:

```
resource "aws_vpc_security_group_ingress_rule" "alb_http" {
  to_port = 80
  ip_protocol = "tcp"

  description = "Allow HTTP traffic from the internet"
}
```

The `alb_all` resource is configured with the following properties:

```
resource "aws_vpc_security_group_egress_rule" "alb_all" {
  security_group_id = aws_security_group.alb.id

  cidr_ipv4 = "0.0.0.0/0"
  ip_protocol = "-1"

  description = "Allow outbound traffic from the ALB"
}
```

The `ec2` resource is configured with the following properties:

```
resource "aws_security_group" "ec2" {
  name = "${var.project_name}-ec2-sg"
  description = "Security group for application EC2 instances"
  vpc_id = aws_vpc.main.id

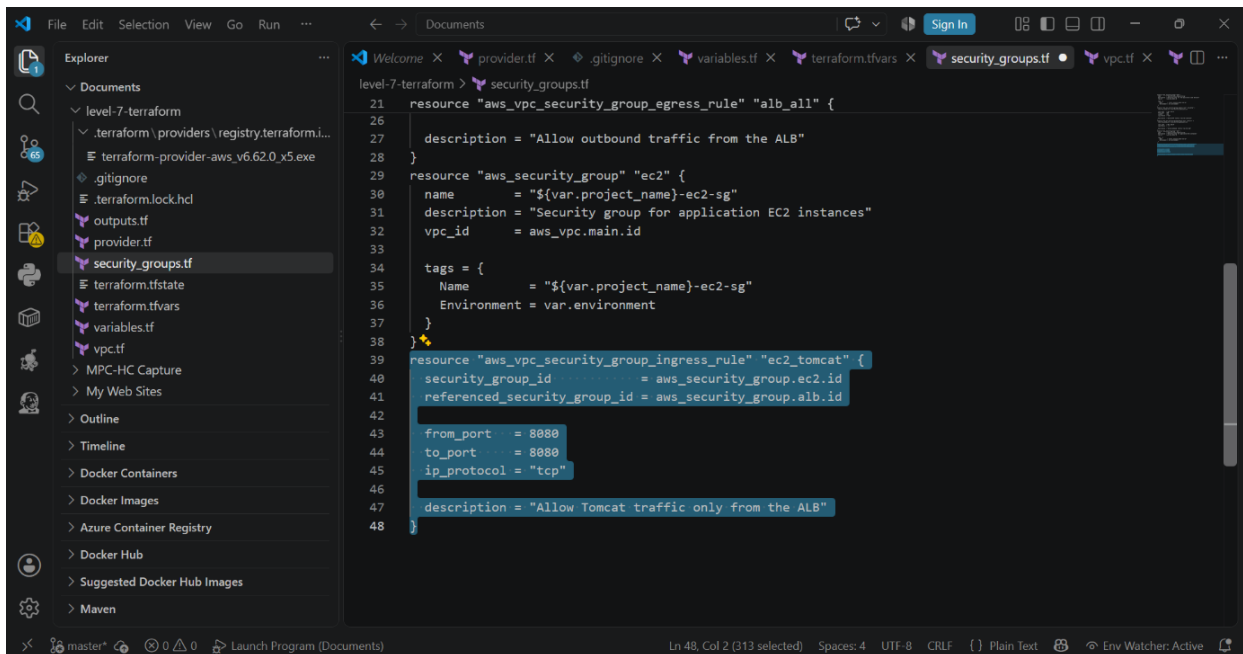
  tags = {
    Name = "${var.project_name}-ec2-sg"
    Environment = var.environment
  }
}
```

Figure 37. Create the EC2 application security group.

Step 37 : Restrict EC2 port 8080 to the ALB

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.



The screenshot shows a VS Code editor with a Terraform configuration file named `security_groups.tf`. The configuration defines an AWS security group and two rules. The `aws_vpc_security_group_egress_rule` rule, named `alb_all`, allows all outbound traffic. The `aws_security_group` resource, named `ec2`, is configured with a name that includes the project name and environment. The `aws_vpc_security_group_ingress_rule` resource, named `ec2_tomcat`, restricts inbound traffic to port 8080 from the ALB security group.

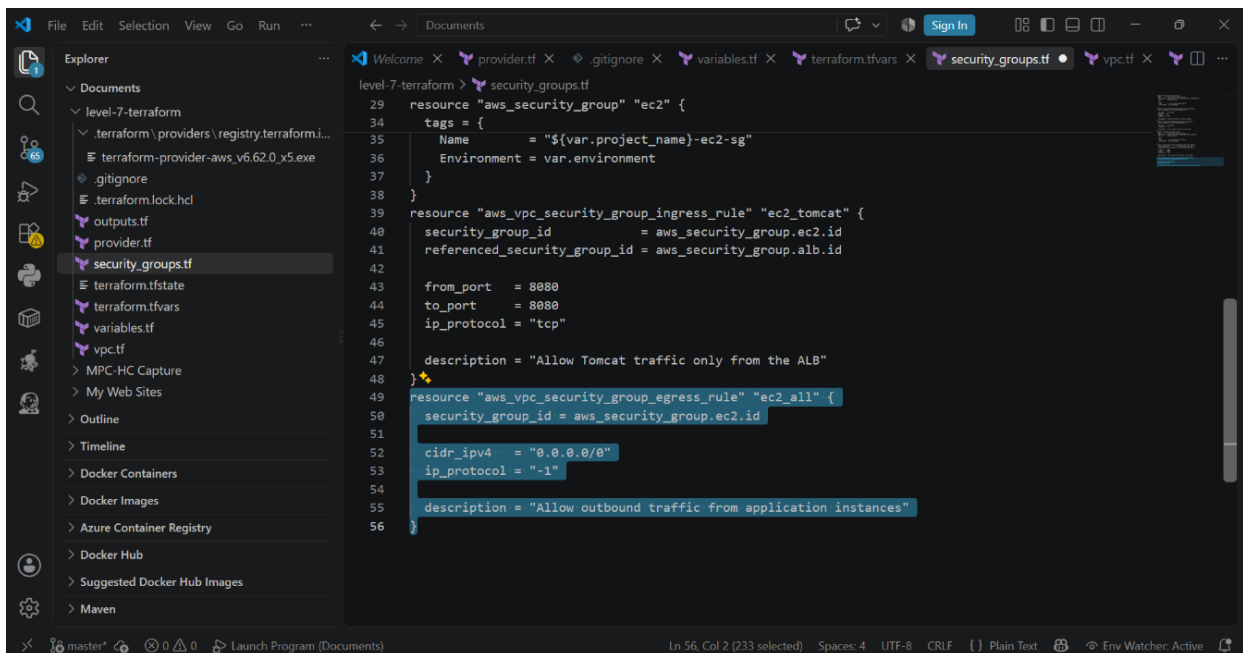
```
level-7-terraform > security_groups.tf
21 resource "aws_vpc_security_group_egress_rule" "alb_all" {
22   description = "Allow outbound traffic from the ALB"
23 }
24
25 resource "aws_security_group" "ec2" {
26   name           = "${var.project_name}-ec2-sg"
27   description    = "Security group for application EC2 instances"
28   vpc_id         = aws_vpc.main.id
29
30   tags = {
31     Name       = "${var.project_name}-ec2-sg"
32     Environment = var.environment
33   }
34 }
35
36 resource "aws_vpc_security_group_ingress_rule" "ec2_tomcat" {
37   security_group_id         = aws_security_group.ec2.id
38   referenced_security_group_id = aws_security_group.alb.id
39
40   from_port = 8080
41   to_port   = 8080
42   ip_protocol = "tcp"
43
44   description = "Allow Tomcat traffic only from the ALB"
45 }
```

Figure 38. Restrict EC2 port 8080 to the ALB.

Step 38 : Complete EC2 security-group rules

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.



The screenshot shows a VS Code editor with a Terraform configuration file named `security_groups.tf`. The configuration defines an AWS security group and three rules. The `aws_vpc_security_group_ingress_rule` resource, named `ec2_tomcat`, restricts inbound traffic to port 8080 from the ALB security group. The `aws_vpc_security_group_egress_rule` resource, named `ec2_all`, allows all outbound traffic from the EC2 security group.

```
level-7-terraform > security_groups.tf
29 resource "aws_security_group" "ec2" {
30   tags = {
31     Name       = "${var.project_name}-ec2-sg"
32     Environment = var.environment
33   }
34 }
35
36 resource "aws_vpc_security_group_ingress_rule" "ec2_tomcat" {
37   security_group_id         = aws_security_group.ec2.id
38   referenced_security_group_id = aws_security_group.alb.id
39
40   from_port = 8080
41   to_port   = 8080
42   ip_protocol = "tcp"
43
44   description = "Allow Tomcat traffic only from the ALB"
45 }
46
47 resource "aws_vpc_security_group_egress_rule" "ec2_all" {
48   security_group_id = aws_security_group.ec2.id
49
50   cidr_ipv4 = "0.0.0.0/0"
51   ip_protocol = "-1"
52
53   description = "Allow outbound traffic from application instances"
54 }
```

Figure 39. Complete EC2 security-group rules.

Step 39 : Review security-group configuration

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.

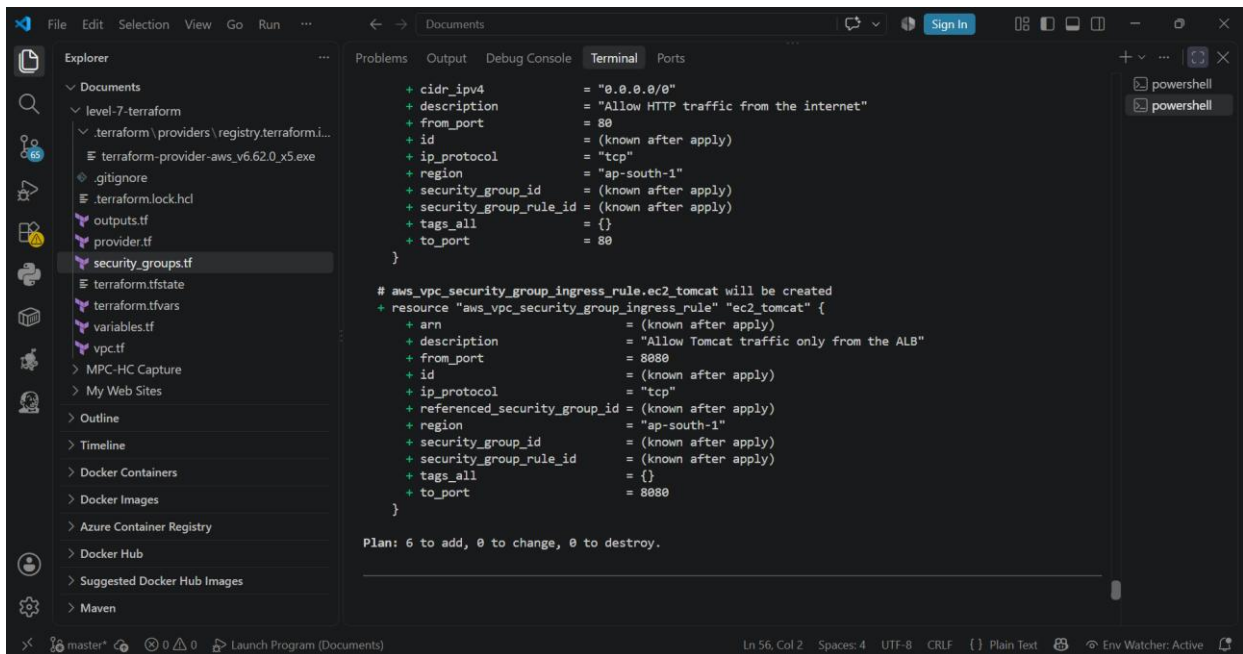


Figure 40. Review security-group configuration.

Step 40 : Apply security-group resources

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.

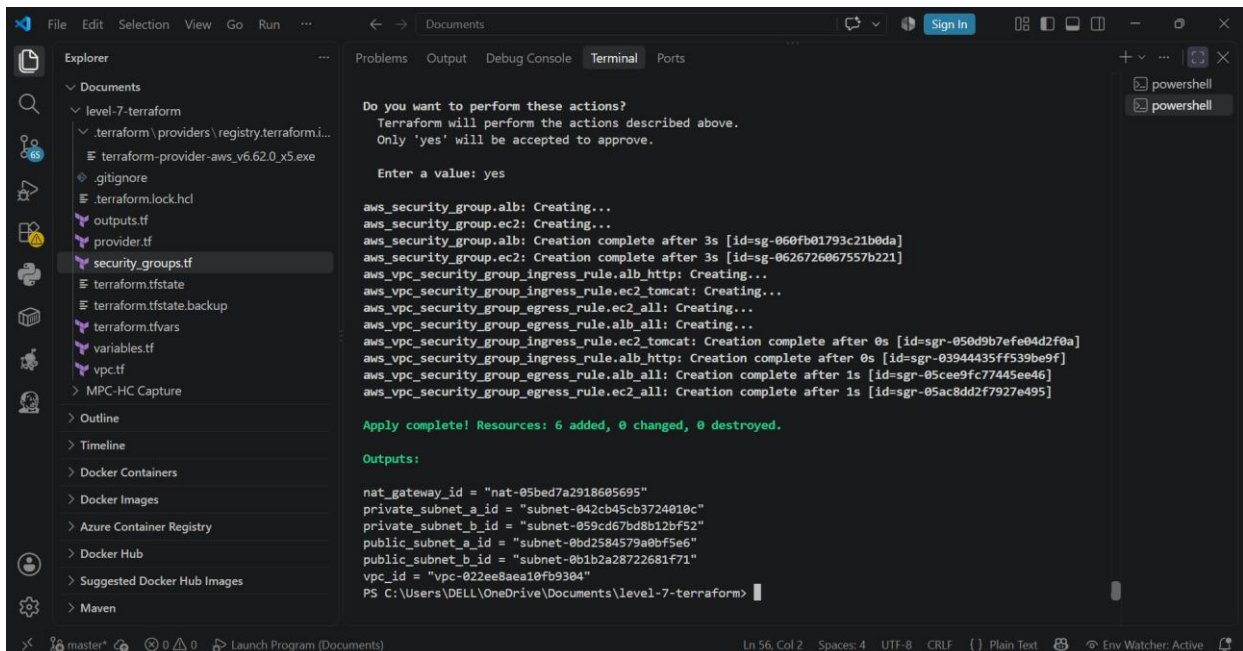


Figure 41. Apply security-group resources.

Step 41 : Verify security groups in AWS

Implementation Action / What I did: Configure and verify the security rule shown in this screenshot for the ALB/application tiers.

What the screenshot proves: The screenshot shows the security-group definition, apply output or AWS rule verification.

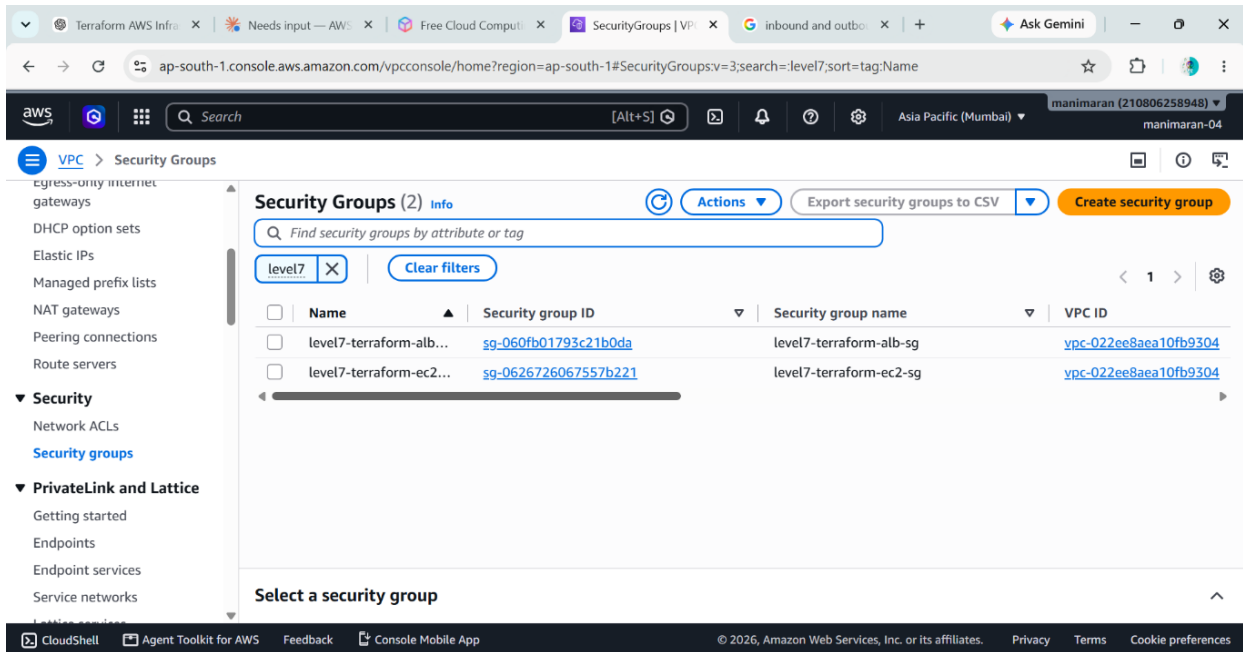


Figure 42. Verify security groups in AWS.

Phase 5 – IAM Role, Policy and Instance Profile

Step 42 : Define the EC2 IAM role

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.

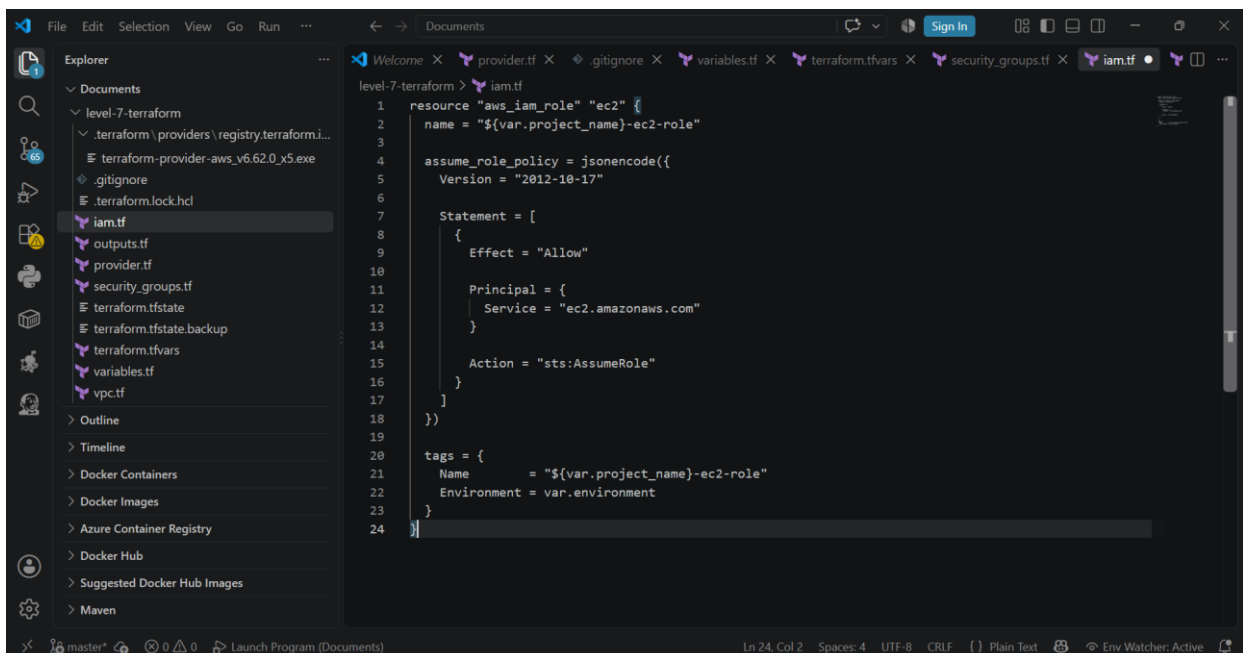
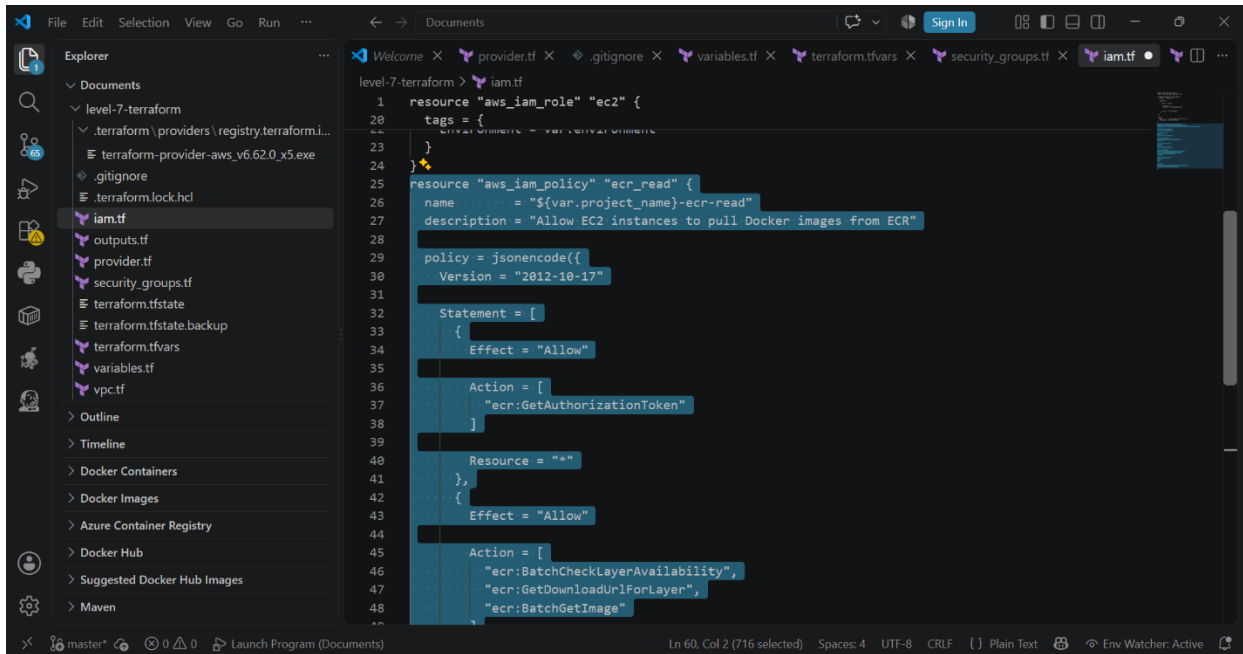


Figure 43. Define the EC2 IAM role.

Step 43 : Define the ECR pull policy

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.



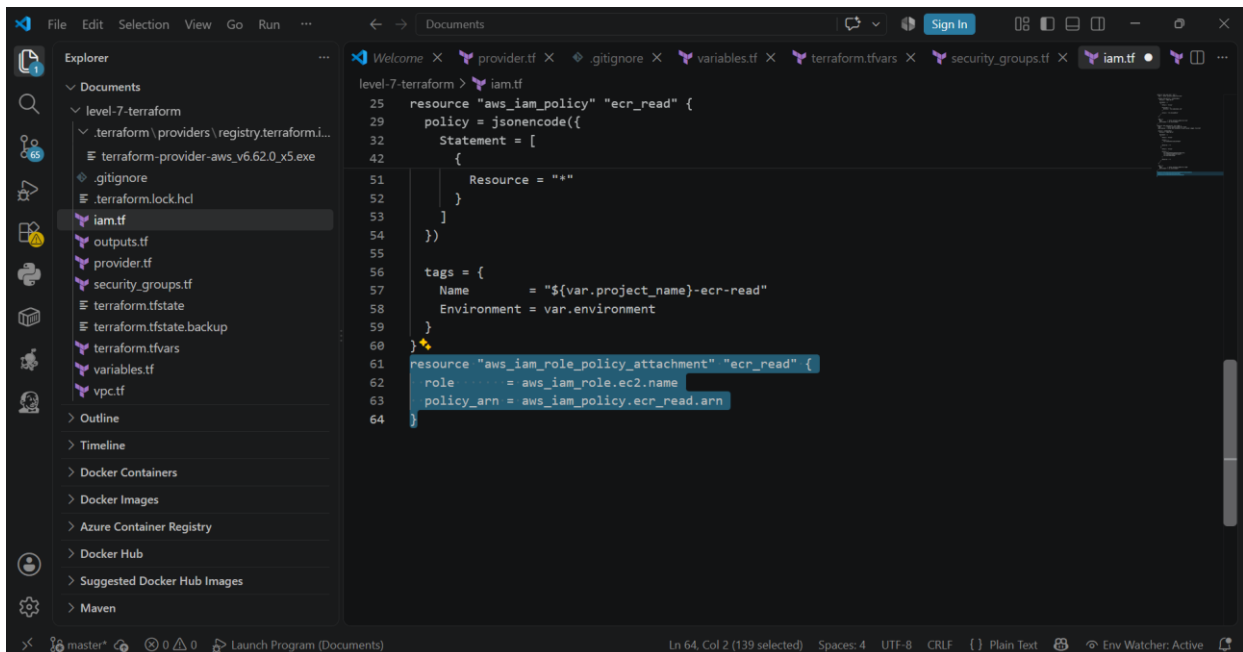
```
1 resource "aws_iam_role" "ec2" {
2   tags = {
3     "Project" = "level-7-terraform"
4   }
5 }
6
7 resource "aws_iam_policy" "ecr_read" {
8   name = "${var.project_name}-ecr-read"
9   description = "Allow EC2 instances to pull Docker images from ECR"
10
11   policy = jsonencode({
12     Version = "2012-10-17"
13
14     Statement = [
15       {
16         Effect = "Allow"
17
18         Action = [
19           "ecr:GetAuthorizationToken"
20         ]
21
22         Resource = "*"
23       },
24       {
25         Effect = "Allow"
26
27         Action = [
28           "ecr:BatchCheckLayerAvailability",
29           "ecr:GetDownloadUrlForLayer",
30           "ecr:BatchGetImage"
31         ]
32       }
33     ]
34   })
35 }
```

Figure 44. Define the ECR pull policy.

Step 44 : Attach the ECR policy

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.



```
25 resource "aws_iam_policy" "ecr_read" {
26   policy = jsonencode({
27     Statement = [
28       {
29         Resource = "*"
30       }
31     ]
32   })
33
34   tags = {
35     Name = "${var.project_name}-ecr-read"
36     Environment = var.environment
37   }
38 }
39
40 resource "aws_iam_role_policy_attachment" "ecr_read" {
41   role = aws_iam_role.ec2.name
42   policy_arn = aws_iam_policy.ecr_read.arn
43 }
```

Figure 45. Attach the ECR policy.

Step 45 : Create the EC2 instance profile

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.

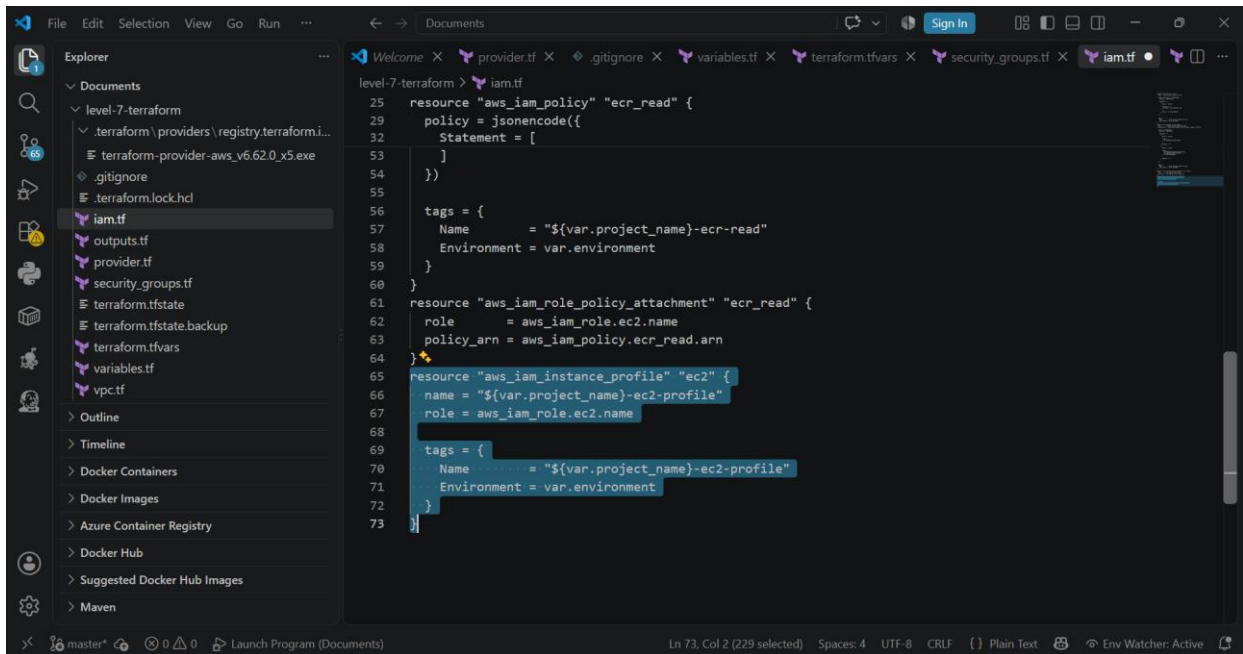


Figure 46. Create the EC2 instance profile.

Step 46 : Apply IAM resources

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.

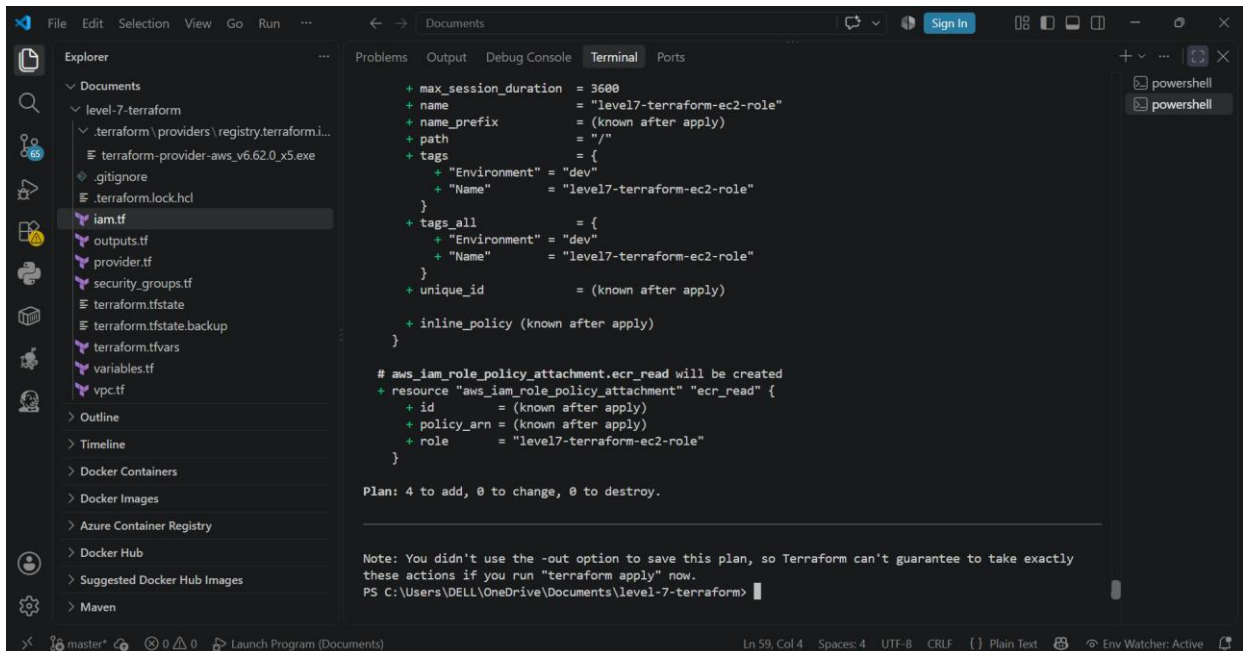
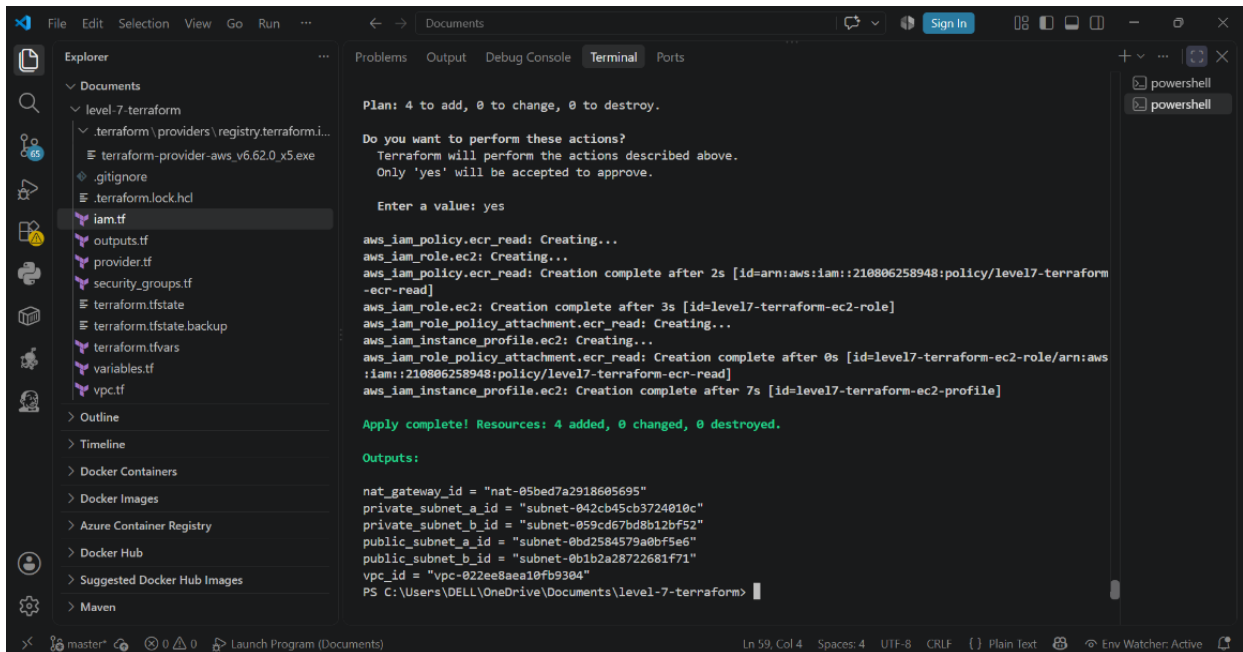


Figure 47. Apply IAM resources.

Step 47 : Verify IAM apply output

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.



```
Plan: 4 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_iam_policy.ecr_read: Creating...
aws_iam_role.ec2: Creating...
aws_iam_role_policy_attachment.ecr_read: Creation complete after 2s [id=arn:aws:iam::210806258948:policy/level7-terraform-ecr-read]
aws_iam_role.ec2: Creation complete after 3s [id=level7-terraform-ec2-role]
aws_iam_role_policy_attachment.ecr_read: Creating...
aws_iam_instance_profile.ec2: Creating...
aws_iam_role_policy_attachment.ecr_read: Creation complete after 0s [id=level7-terraform-ec2-role/arn:aws:iam::210806258948:policy/level7-terraform-ecr-read]
aws_iam_instance_profile.ec2: Creation complete after 7s [id=level7-terraform-ec2-profile]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

nat_gateway_id = "nat-05bed7a29186d5695"
private_subnet_a_id = "subnet-042cb45cb3724010c"
private_subnet_b_id = "subnet-059cd67bd8b12bf52"
public_subnet_a_id = "subnet-0bd2584579a0bf5e6"
public_subnet_b_id = "subnet-0b1b2a28722681f71"
vpc_id = "vpc-022ee8aea10fb9304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure 48. Verify IAM apply output.

Step 48 : Verify the IAM role in AWS

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.

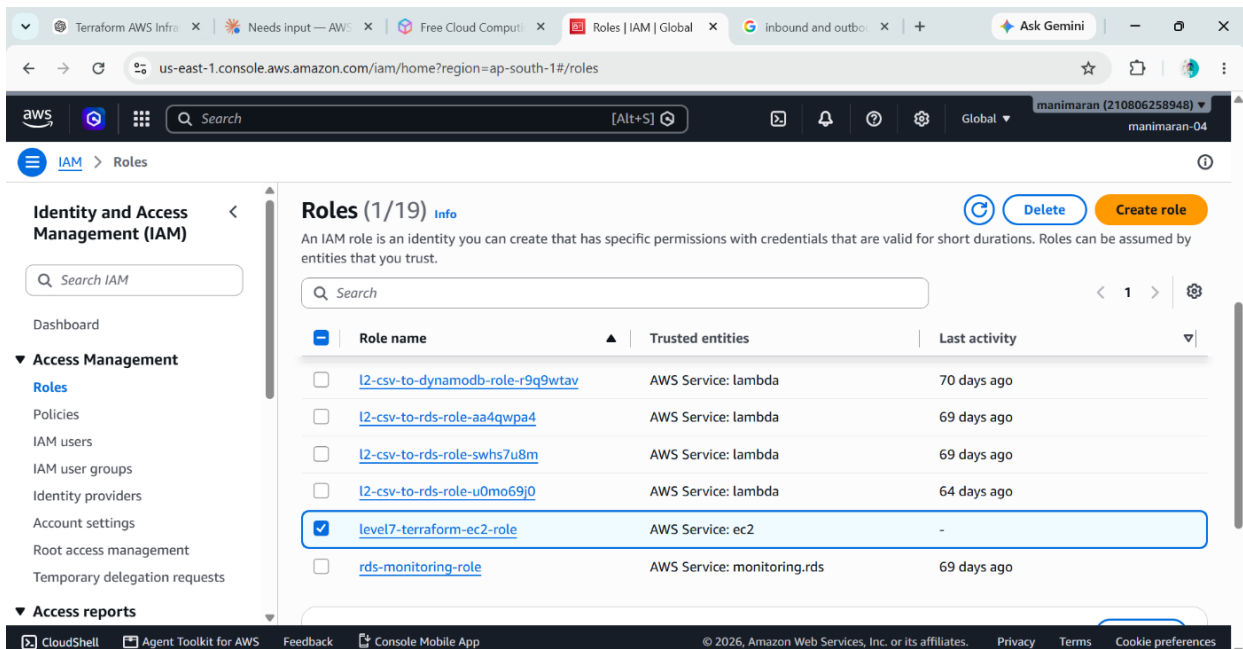


Figure 49. Verify the IAM role in AWS.

Step 49 : Verify the IAM policy in AWS

Implementation Action / What I did: Configure, apply or verify the IAM resource shown in the screenshot.

What the screenshot proves: The screenshot proves the required IAM resource/permission exists.

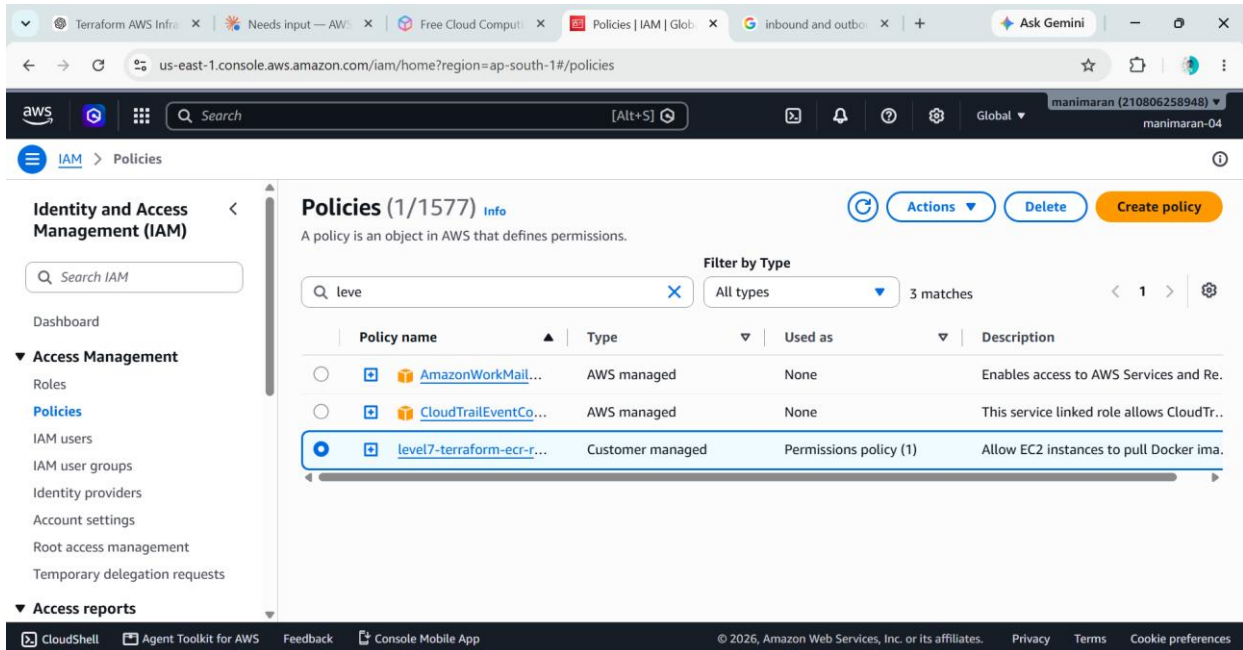


Figure 50. Verify the IAM policy in AWS.

Phase 6 – Amazon ECR and Docker/Tomcat

Step 50 : Define the ECR repository

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.

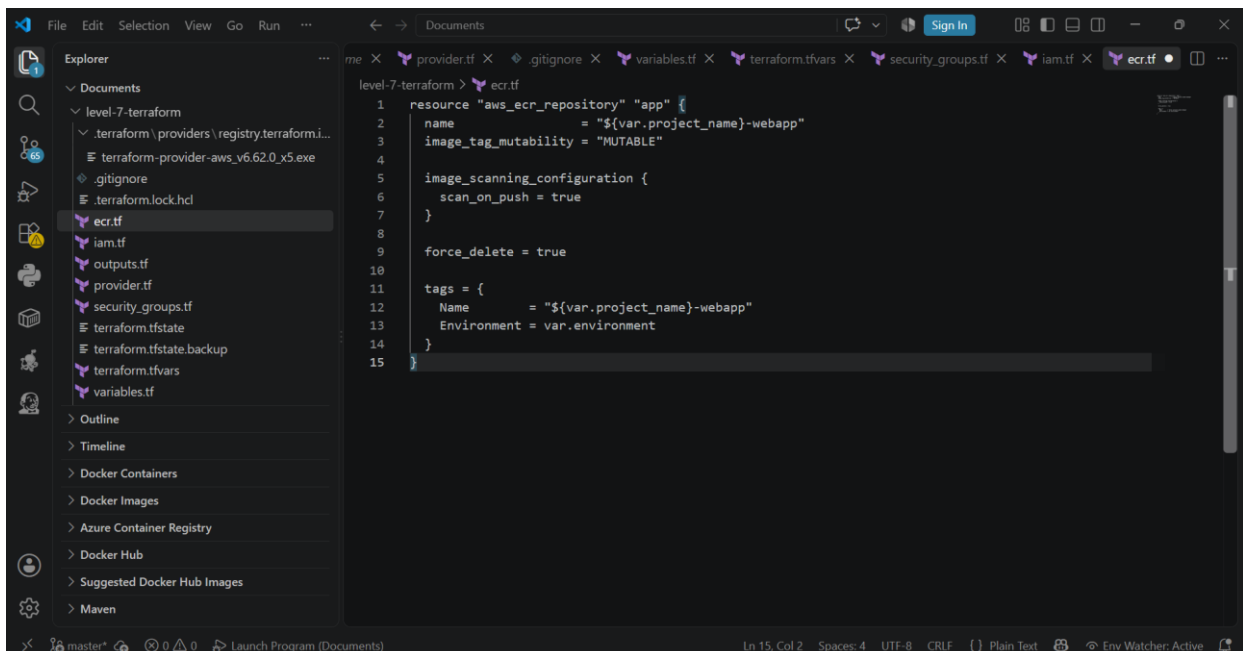
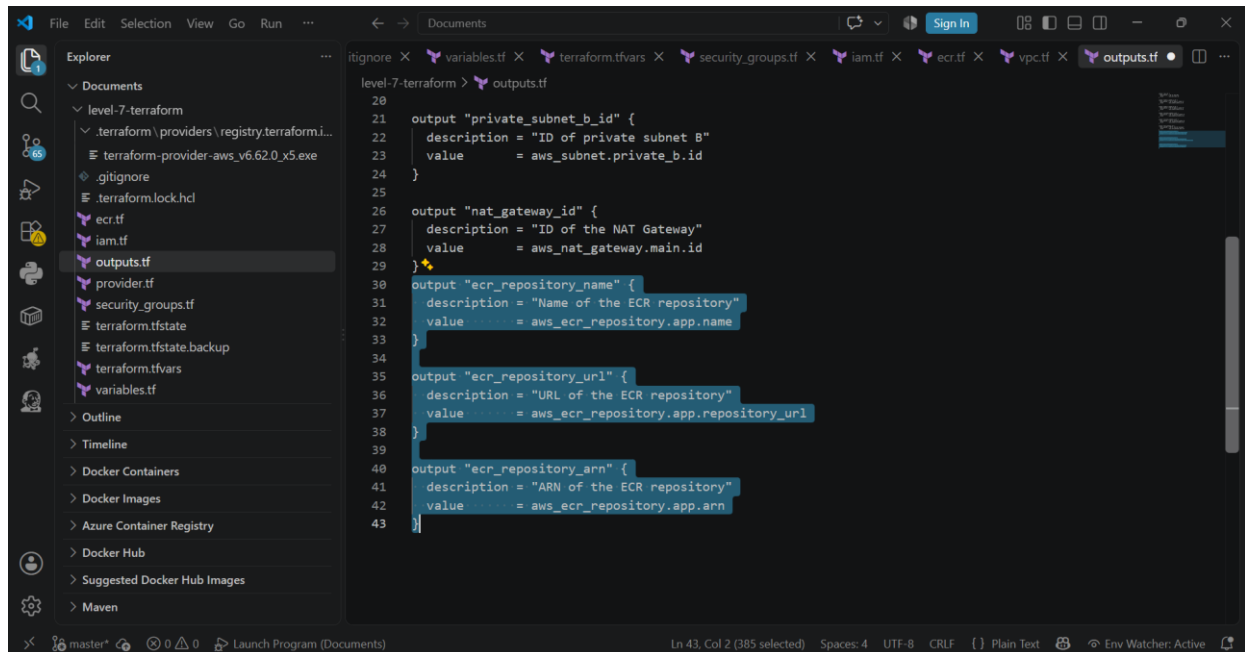


Figure 51. Define the ECR repository.

Step 51 : Configure ECR repository settings

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows a project structure with various Terraform files. The code editor displays the contents of the `outputs.tf` file, which defines several outputs for the ECR repository configuration. The outputs are:

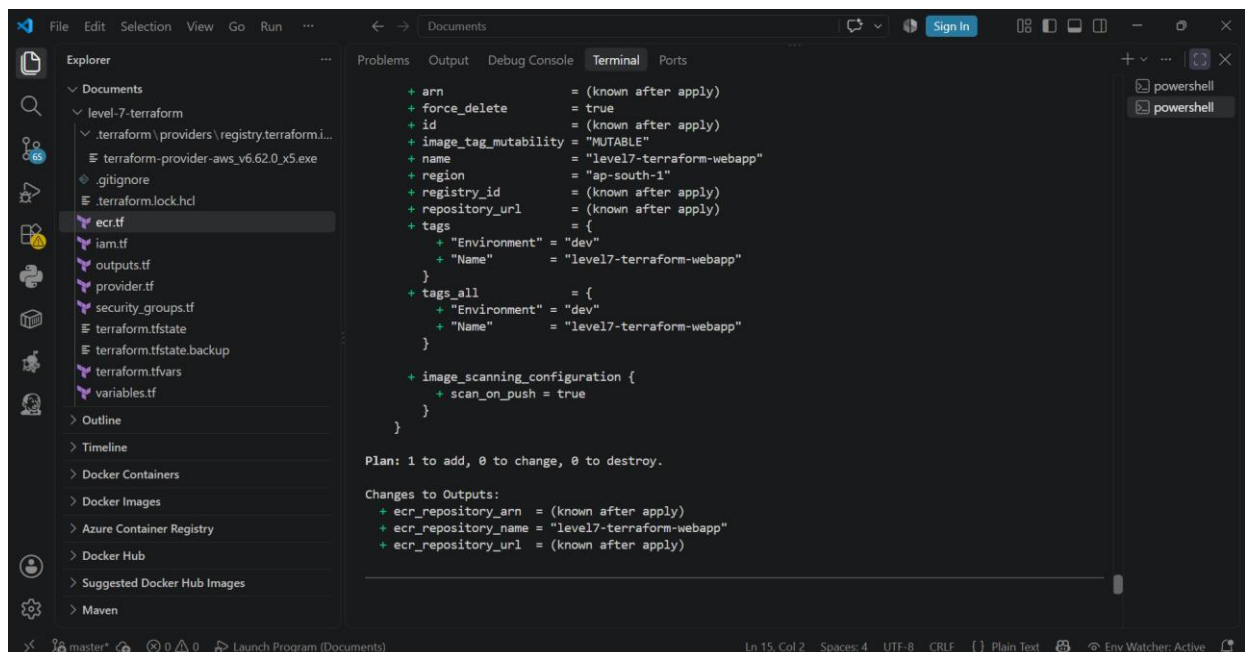
```
20
21 output "private_subnet_b_id" {
22   description = "ID of private subnet B"
23   value       = aws_subnet.private_b_id
24 }
25
26 output "nat_gateway_id" {
27   description = "ID of the NAT Gateway"
28   value       = aws_nat_gateway.main.id
29 }
30 output "ecr_repository_name" {
31   description = "Name of the ECR repository"
32   value       = aws_ecr_repository.app.name
33 }
34 output "ecr_repository_url" {
35   description = "URL of the ECR repository"
36   value       = aws_ecr_repository.app.repository_url
37 }
38
39 output "ecr_repository_arn" {
40   description = "ARN of the ECR repository"
41   value       = aws_ecr_repository.app.arn
42 }
43 }
```

Figure 52. Configure ECR repository settings.

Step 52 : Configure the ECR repository reference

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows a project structure with various Terraform files. The code editor displays the contents of the `ecr.tf` file, which defines the ECR repository resource. The resource is named `aws_ecr_repository.app` and has the following attributes:

```
+ arn                = (known after apply)
+ force_delete       = true
+ id                 = (known after apply)
+ image_tag_mutability = "MUTABLE"
+ name               = "level7-terraform-webapp"
+ region             = "ap-south-1"
+ registry_id        = (known after apply)
+ repository_url      = (known after apply)
+ tags               = {
+   "Environment" = "dev"
+   "Name"        = "level7-terraform-webapp"
+ }
+ tags_all           = {
+   "Environment" = "dev"
+   "Name"        = "level7-terraform-webapp"
+ }
+ image_scanning_configuration {
+   scan_on_push = true
+ }
```

Below the resource definition, there is a plan summary and a list of changes to outputs:

```
Plan: 1 to add, 0 to change, 0 to destroy.

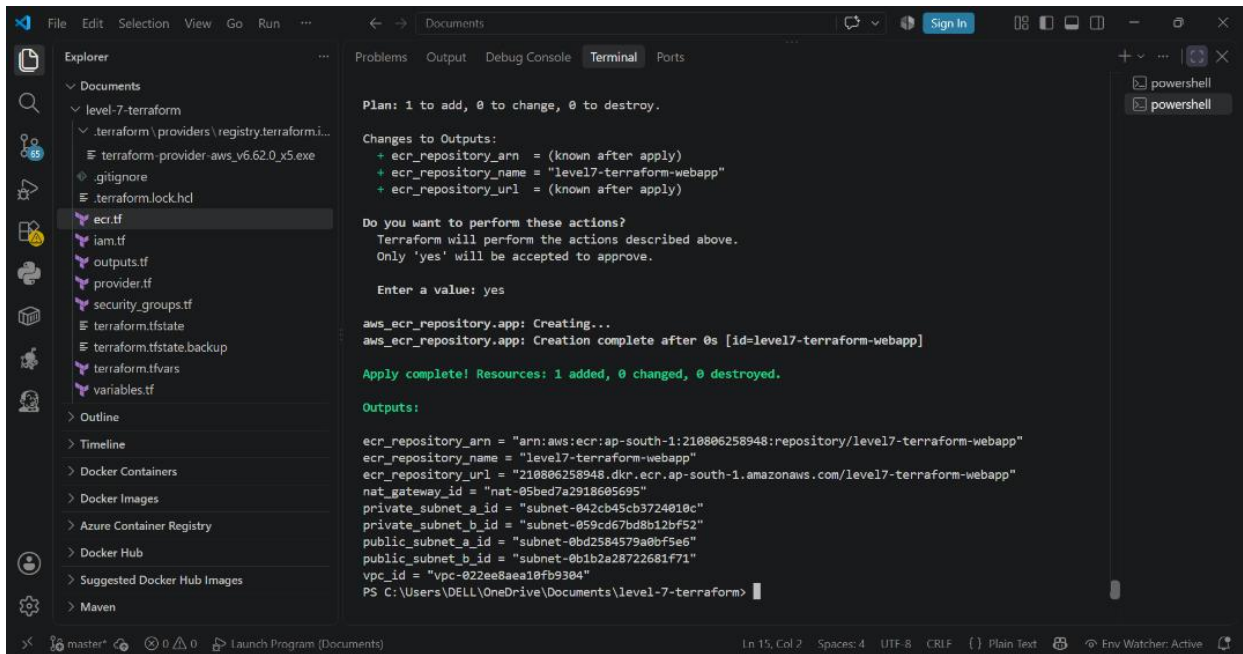
Changes to Outputs:
+ ecr_repository_arn = (known after apply)
+ ecr_repository_name = "level7-terraform-webapp"
+ ecr_repository_url = (known after apply)
```

Figure 53. Configure the ECR repository reference.

Step 53 : Apply the ECR configuration

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.



```
File Edit Selection View Go Run ... Documents Sign In 08:08 [Icons] - [Maximize] [Close]

Explorer Problems Output Debug Console Terminal Ports
Documents
  level-7-terraform
    .terraform\providers\registry.terraform.i...
    terraform-provider-aws_v6.62.0_x5.exe
    .gitignore
    terraform.lock.hcl
  ecr.tf
  iam.tf
  outputs.tf
  provider.tf
  security_groups.tf
  terraform.tfstate
  terraform.tfstate.backup
  terraform.tfvars
  variables.tf
  Outline
  Timeline
  Docker Containers
  Docker Images
  Azure Container Registry
  Docker Hub
  Suggested Docker Hub Images
  Maven

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ecr_repository_arn = (known after apply)
+ ecr_repository_name = "level7-terraform-webapp"
+ ecr_repository_url = (known after apply)

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_ecr_repository.app: Creating...
aws_ecr_repository.app: Creation complete after 0s [id=level7-terraform-webapp]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

ecr_repository_arn = "arn:aws:ecr:ap-south-1:210806258948:repository/level7-terraform-webapp"
ecr_repository_name = "level7-terraform-webapp"
ecr_repository_url = "210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp"
nat_gateway_id = "nat-05bed7a2918605695"
private_subnet_a_id = "subnet-042cb45cb3724018c"
private_subnet_b_id = "subnet-059cd67bd8b12bf52"
public_subnet_a_id = "subnet-0bd2584579a0bf5e6"
public_subnet_b_id = "subnet-0b1b2a28722681f71"
vpc_id = "vpc-022ee8aea10fb9304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure 54. Apply the ECR configuration.

Step 54 : Verify ECR Terraform state

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.

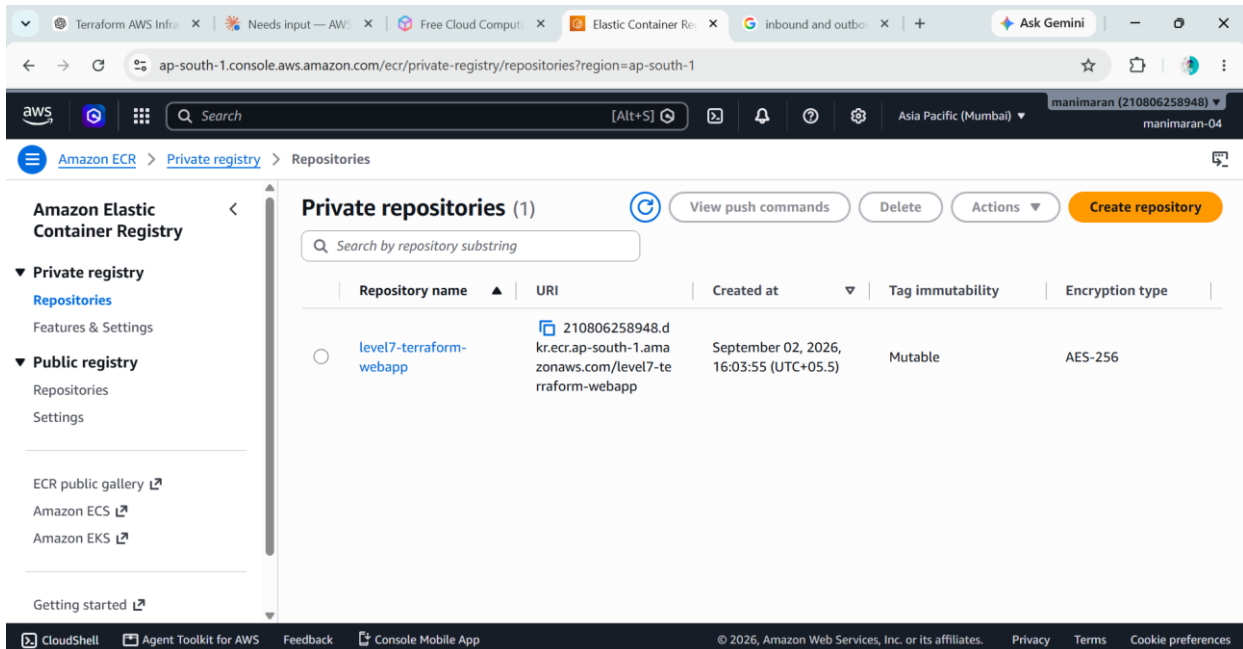


Figure 55. Verify ECR Terraform state.

Step 55 : Build and push the Docker image

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.

```
Windows PowerShell
=> => transferring context: 67B 0.1s
=> [2/3] RUN rm -rf /usr/local/tomcat/webapps/* 0.3s
=> [3/3] COPY target/*.war /usr/local/tomcat/webapps/ROOT.war 0.1s
=> exporting to image 0.6s
=> => exporting layers 0.3s
=> => exporting manifest sha256:d0cbd2e250c994b48a5052b320bbb617f293e337f53d51bc11f1d13c6904b346 0.0s
=> => exporting config sha256:45bfd1e23f10ed9766e168eb55edb43eee61c728a73b10eccc637ff24cb417c5 0.0s
=> => exporting attestation manifest sha256:fba9a8809b952e44566511202ee01754a125613a4c0887db88a6ce7e54975a08 0.0s
=> => exporting manifest list sha256:c3c9180ec5e5ec2a3da9d757aa89603176ddf01faf80f6e37f78c5e14976e2b4 0.0s
=> => naming to docker.io/library/level7-webapp:latest 0.0s
=> => unpacking to docker.io/library/level7-webapp:latest 0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qfixwwwb8h35oic5xblahsfxk
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker images

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
210806258948.dkr.ecr.ap-south-1.amazonaws.com/tomcat-app:latest adfe24233636 721MB 229MB
calculator-app:latest 7a968151385f 197MB 48.2MB
level7-webapp:latest c3c9180ec5e5 721MB 229MB
manimaran04/calculator-app:latest 7a968151385f 197MB 48.2MB
manimaran04/calculator-app:v2 6964b6a3ccd8 197MB 48.2MB
tomcat-app:latest adfe24233636 721MB 229MB
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker run -d -p 8080:8080 --name level7-webapp level7-webapp
569a06a81ba34eb2815855e18a2fad911b5b2d55713c90561c62f34d67eb53b0
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
569a06a81ba3 level7-webapp "catalina.sh run" 36 seconds ago Up 36 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master>
```

Figure 56. Build and push the Docker image.

Step 56 : Test the containerized web application

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.

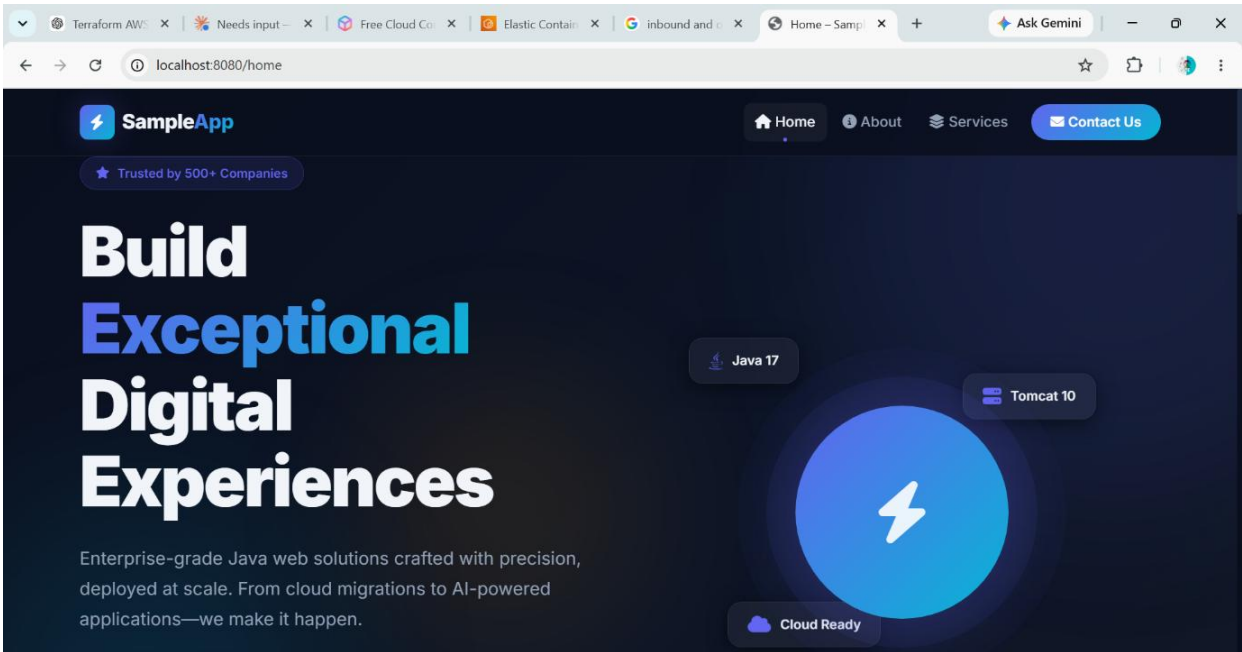
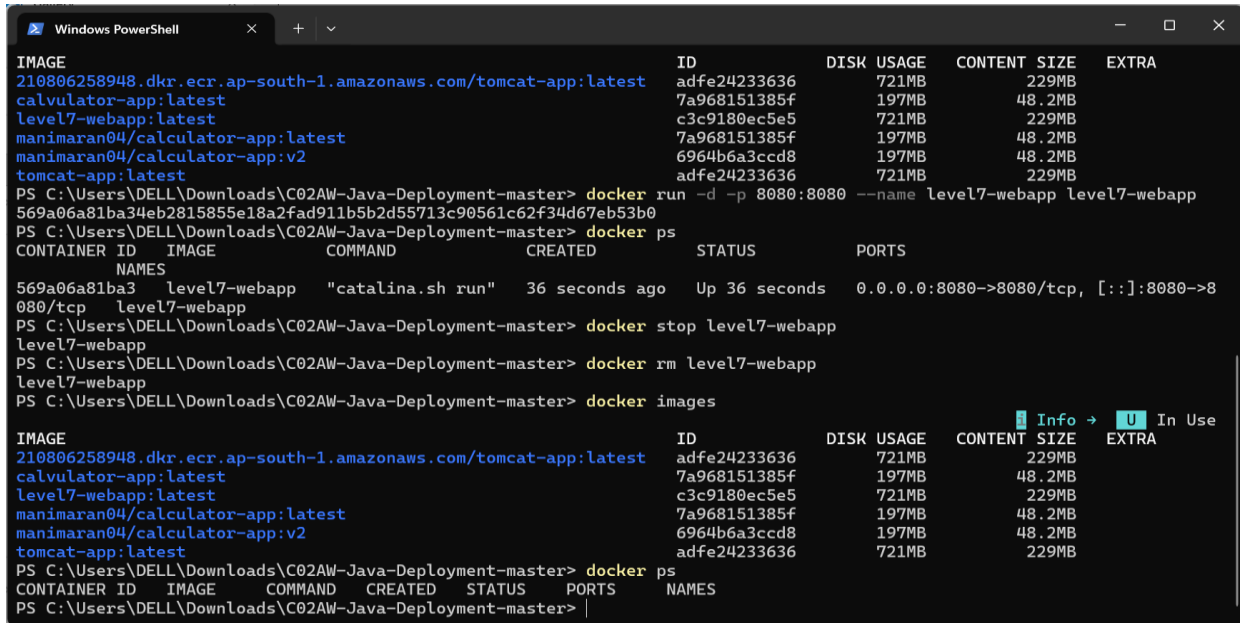


Figure 57. Test the containerized web application.

Step 57 : Verify the Docker image locally

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.



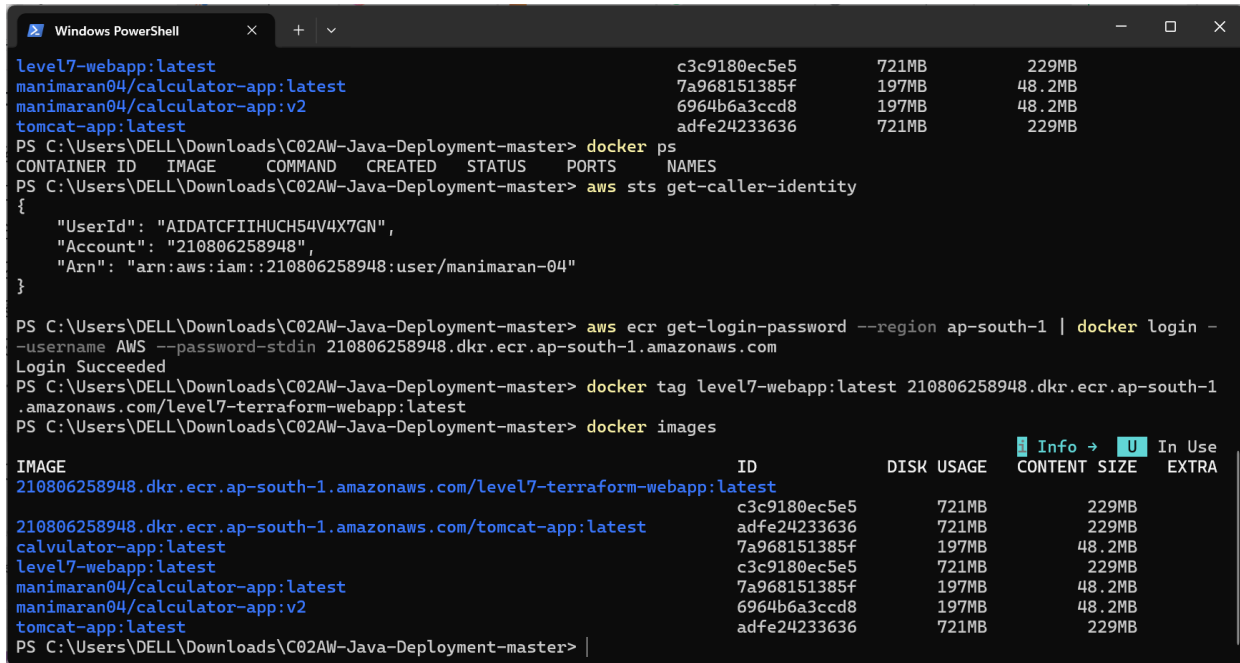
```
Windows PowerShell
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
210806258948.dkr.ecr.ap-south-1.amazonaws.com/tomcat-app:latest adfe24233636 721MB 229MB
calvulator-app:latest 7a968151385f 197MB 48.2MB
level7-webapp:latest c3c9180ec5e5 721MB 229MB
manimaran04/calculator-app:latest 7a968151385f 197MB 48.2MB
manimaran04/calculator-app:v2 6964b6a3ccd8 197MB 48.2MB
tomcat-app:latest adfe24233636 721MB 229MB
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker run -d -p 8080:8080 --name level7-webapp level7-webapp
569a06a81ba34eb2815855e18a2fad911b5b2d55713c90561c62f34d67eb53b0
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
569a06a81ba3 level7-webapp "catalina.sh run" 36 seconds ago Up 36 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker stop level7-webapp
level7-webapp
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker rm level7-webapp
level7-webapp
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
210806258948.dkr.ecr.ap-south-1.amazonaws.com/tomcat-app:latest adfe24233636 721MB 229MB
calvulator-app:latest 7a968151385f 197MB 48.2MB
level7-webapp:latest c3c9180ec5e5 721MB 229MB
manimaran04/calculator-app:latest 7a968151385f 197MB 48.2MB
manimaran04/calculator-app:v2 6964b6a3ccd8 197MB 48.2MB
tomcat-app:latest adfe24233636 721MB 229MB
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master>
```

Figure 58. Verify the Docker image locally

Step 58 : Verify Docker/ECR transfer output

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.



```
Windows PowerShell
level7-webapp:latest c3c9180ec5e5 721MB 229MB
manimaran04/calculator-app:latest 7a968151385f 197MB 48.2MB
manimaran04/calculator-app:v2 6964b6a3ccd8 197MB 48.2MB
tomcat-app:latest adfe24233636 721MB 229MB
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> aws sts get-caller-identity
{
  "UserId": "AIDATCFIHHUCH54V4X7GN",
  "Account": "210806258948",
  "Arn": "arn:aws:iam::210806258948:user/manimaran-04"
}
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 210806258948.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker tag level7-webapp:latest 210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp:latest
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp:latest c3c9180ec5e5 721MB 229MB
210806258948.dkr.ecr.ap-south-1.amazonaws.com/tomcat-app:latest adfe24233636 721MB 229MB
calvulator-app:latest 7a968151385f 197MB 48.2MB
level7-webapp:latest c3c9180ec5e5 721MB 229MB
manimaran04/calculator-app:latest 7a968151385f 197MB 48.2MB
manimaran04/calculator-app:v2 6964b6a3ccd8 197MB 48.2MB
tomcat-app:latest adfe24233636 721MB 229MB
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master>
```

Figure 59. Verify Docker/ECR transfer output.

Step 59 : Confirm image push completion

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.

```
Windows PowerShell
-username AWS --password-stdin 210806258948.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker tag level7-webapp:latest 210806258948.dkr.ecr.ap-south-1
.amazonaws.com/level7-terraform-webapp:latest
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp:latest	c3c9180ec5e5	721MB	229MB	
210806258948.dkr.ecr.ap-south-1.amazonaws.com/tomcat-app:latest	adfe24233636	721MB	229MB	
calculator-app:latest	7a968151385f	197MB	48.2MB	
level7-webapp:latest	c3c9180ec5e5	721MB	229MB	
manimaran04/calculator-app:latest	7a968151385f	197MB	48.2MB	
manimaran04/calculator-app:v2	6964b6a3ccd8	197MB	48.2MB	
tomcat-app:latest	adfe24233636	721MB	229MB	

```
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker push 210806258948.dkr.ecr.ap-south-1.amazonaws.com/level
7-terraform-webapp:latest
The push refers to repository [210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp]
a65278a6475f: Pushed
6611c962e4e8: Pushed
a8a21a570f26: Pushed
2b31ee5335cc: Pushed
4f4fb700ef54: Pushed
7e5eef01c0a7: Pushed
887642163ab8: Pushed
f3bb84430441: Pushed
48b7756512f9: Pushed
0926a8b0e60: Pushed
latest: digest: sha256:c3c9180ec5e5ec2a3da9d757aa89603176ddf01faf80f6e37f78c5e14976e2b4 size: 856
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master>
```

Figure 60. Confirm image push completion.

Step 60 : Verify the image in Amazon ECR

Implementation Action / What I did: Implement or verify the ECR/Docker step shown in the screenshot.

What the screenshot proves: The screenshot proves the ECR/Docker stage is configured, executed or verified.

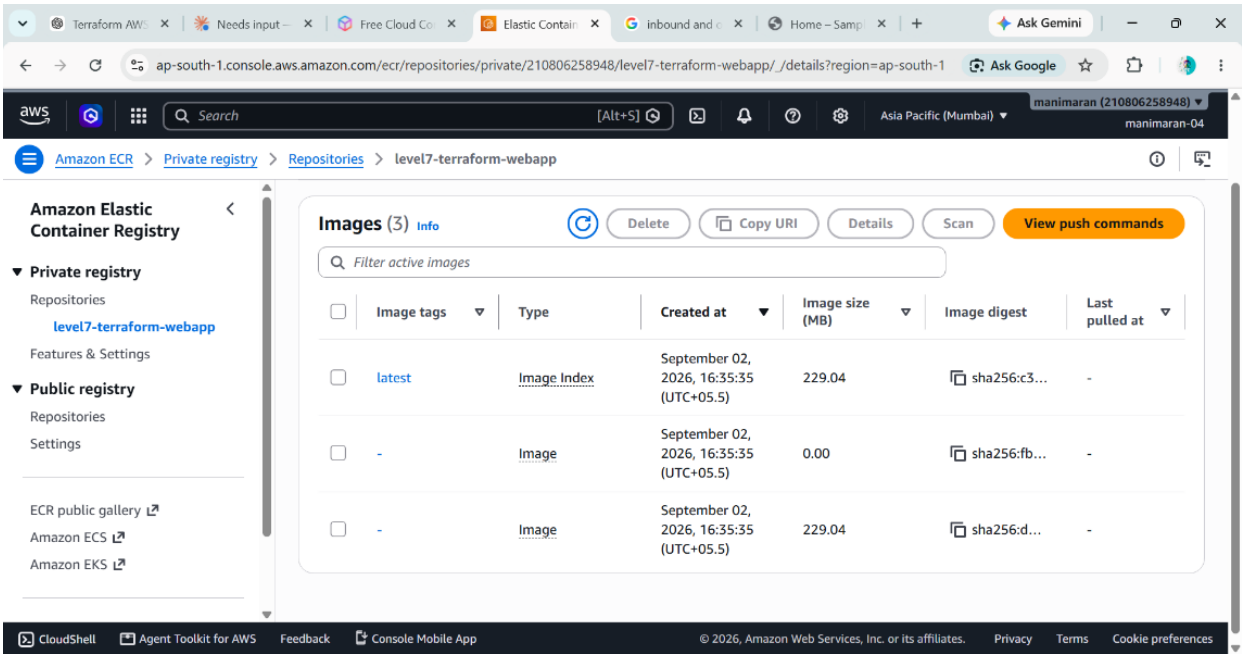


Figure 61. Verify the image in Amazon ECR.

Phase 7 – EC2 Launch Template and User Data

Step 61 : Define the EC2 Launch Template

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.

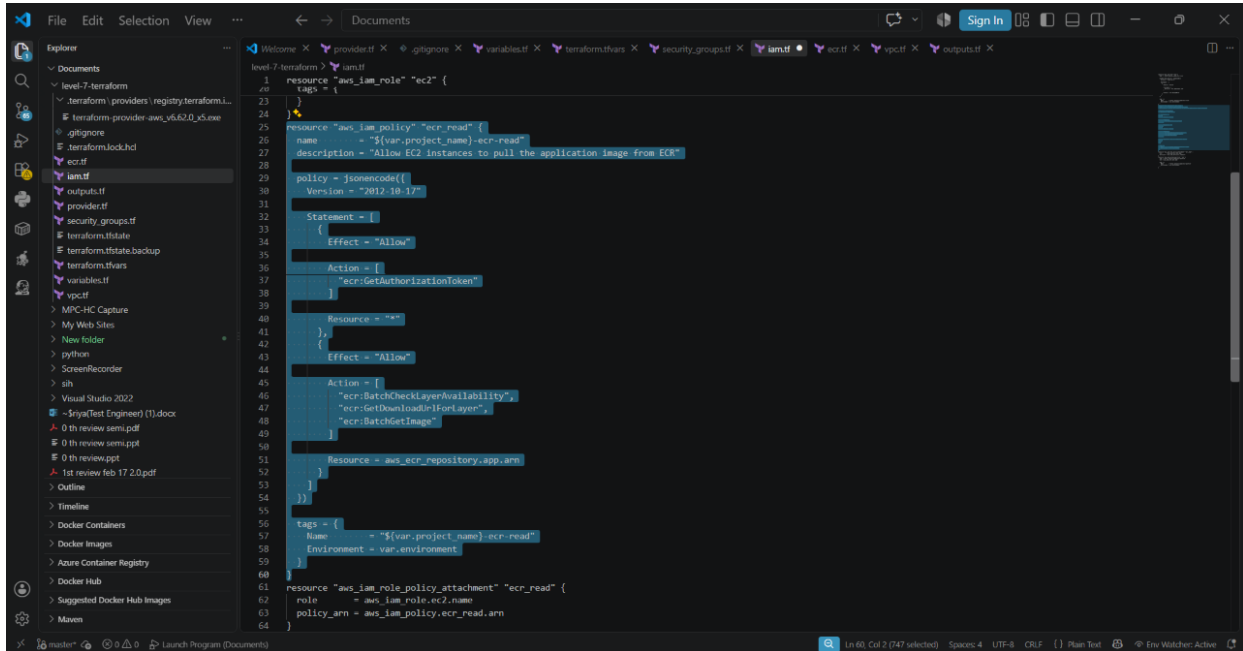


Figure 62. Define the EC2 Launch Template.

Step 62 : Configure Launch Template networking

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.

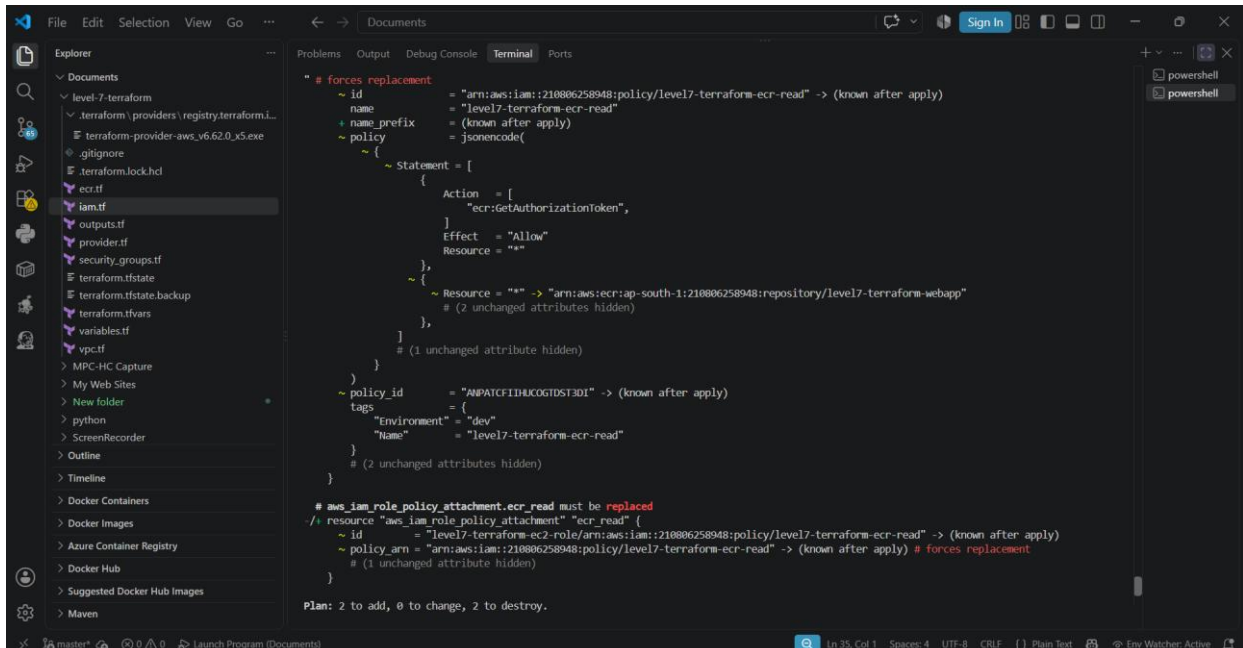


Figure 63. Configure Launch Template networking.

Step 63 : Set the application image URI

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.

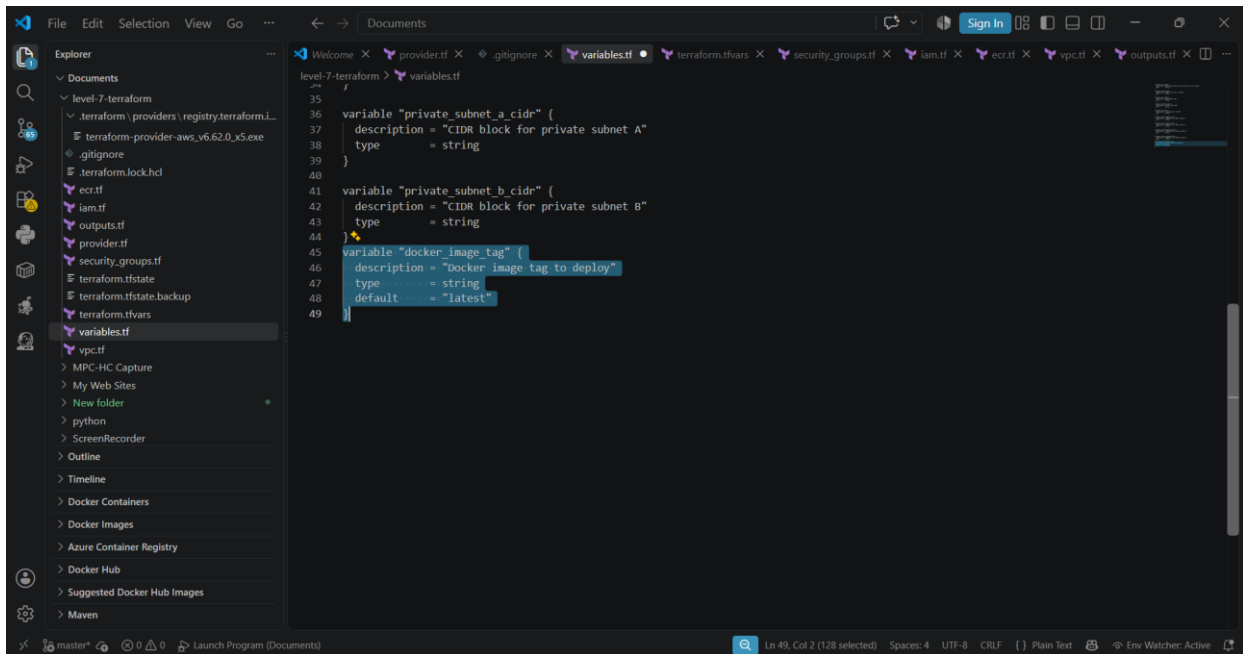


Figure 64. Set the application image URI.

Step 64 : Define EC2 user data

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.

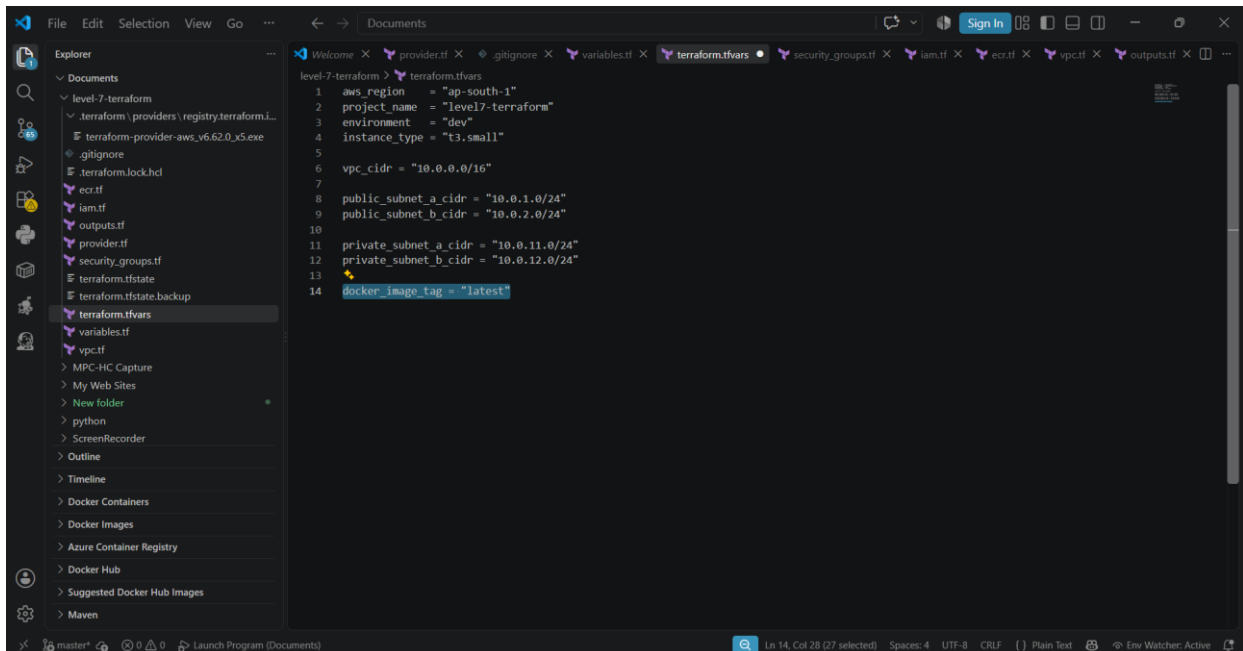
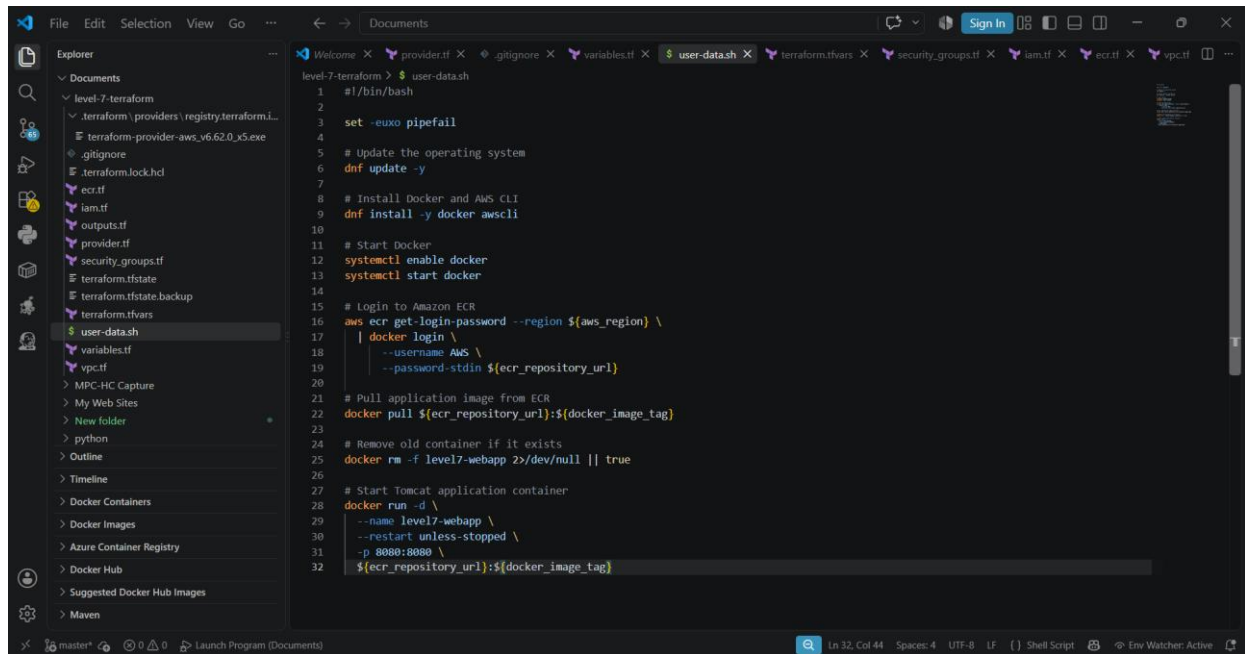


Figure 65. Define EC2 user data.

Step 65 : Configure ECR authentication in user data

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like provider.tf, user-data.sh, variables.tf, terraform.tfvars, security_groups.tf, iam.tf, ecr.tf, vpc.tf, outputs.tf, provider.tf, security_groups.tf, terraform.tfstate, terraform.tfstate.backup, terraform.tfvars, user-data.sh, variables.tf, vpc.tf, MPC-HC Capture, My Web Sites, New folder, python, Outline, Timeline, Docker Containers, Docker Images, Azure Container Registry, Docker Hub, Suggested Docker Hub Images, and Maven. The code editor shows the user-data.sh script with the following content:

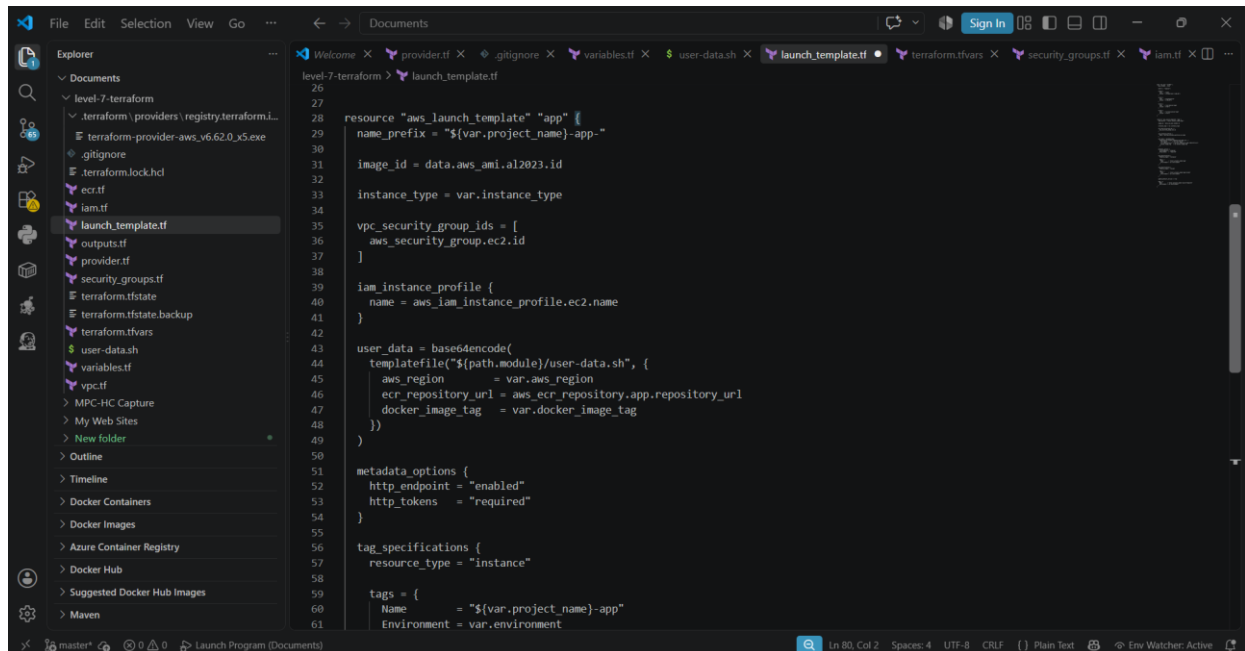
```
level-7-terraform > $ user-data.sh
1 #!/bin/bash
2
3 set -eoux pipefail
4
5 # Update the operating system
6 dnf update -y
7
8 # Install Docker and AWS CLI
9 dnf install -y docker awscli
10
11 # Start Docker
12 systemctl enable docker
13 systemctl start docker
14
15 # Login to Amazon ECR
16 aws ecr get-login-password --region ${aws_region} \
17 | docker login \
18   --username AWS \
19   --password-stdin ${ecr_repository_url}
20
21 # Pull application image from ECR
22 docker pull ${ecr_repository_url}:${docker_image_tag}
23
24 # Remove old container if it exists
25 docker rm -f level7-webapp 2>/dev/null || true
26
27 # Start Tomcat application container
28 docker run -d \
29   --name level7-webapp \
30   --restart unless-stopped \
31   -p 8080:8080 \
32   ${ecr_repository_url}:${docker_image_tag}
```

Figure 66. Configure ECR authentication in user data

Step 66 : Configure Docker/Tomcat startup

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like provider.tf, user-data.sh, variables.tf, terraform.tfvars, security_groups.tf, iam.tf, ecr.tf, vpc.tf, outputs.tf, provider.tf, security_groups.tf, terraform.tfstate, terraform.tfstate.backup, terraform.tfvars, user-data.sh, variables.tf, vpc.tf, MPC-HC Capture, My Web Sites, New folder, python, Outline, Timeline, Docker Containers, Docker Images, Azure Container Registry, Docker Hub, Suggested Docker Hub Images, and Maven. The code editor shows the launch_template.tf script with the following content:

```
level-7-terraform > $ launch_template.tf
26
27 resource "aws_launch_template" "app" {
28   name_prefix = "${var.project_name}-app-"
29
30   image_id = data.aws_ami.al2023.id
31
32   instance_type = var.instance_type
33
34   vpc_security_group_ids = [
35     aws_security_group.ec2.id
36   ]
37
38   iam_instance_profile {
39     name = aws_iam_instance_profile.ec2.name
40   }
41
42   user_data = base64encode(
43     templatefile("${path.module}/user-data.sh", {
44       aws_region = var.aws_region
45       ecr_repository_url = aws_ecr_repository.app.repository_url
46       docker_image_tag = var.docker_image_tag
47     })
48   )
49
50   metadata_options {
51     http_endpoint = "enabled"
52     http_tokens = "required"
53   }
54
55   tag_specifications {
56     resource_type = "instance"
57
58     tags = {
59       Name = "${var.project_name}-app"
60       Environment = var.environment
61     }
62   }
63 }
```

Figure 67. Configure Docker/Tomcat startup.

Step 67 : Complete Launch Template configuration

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.

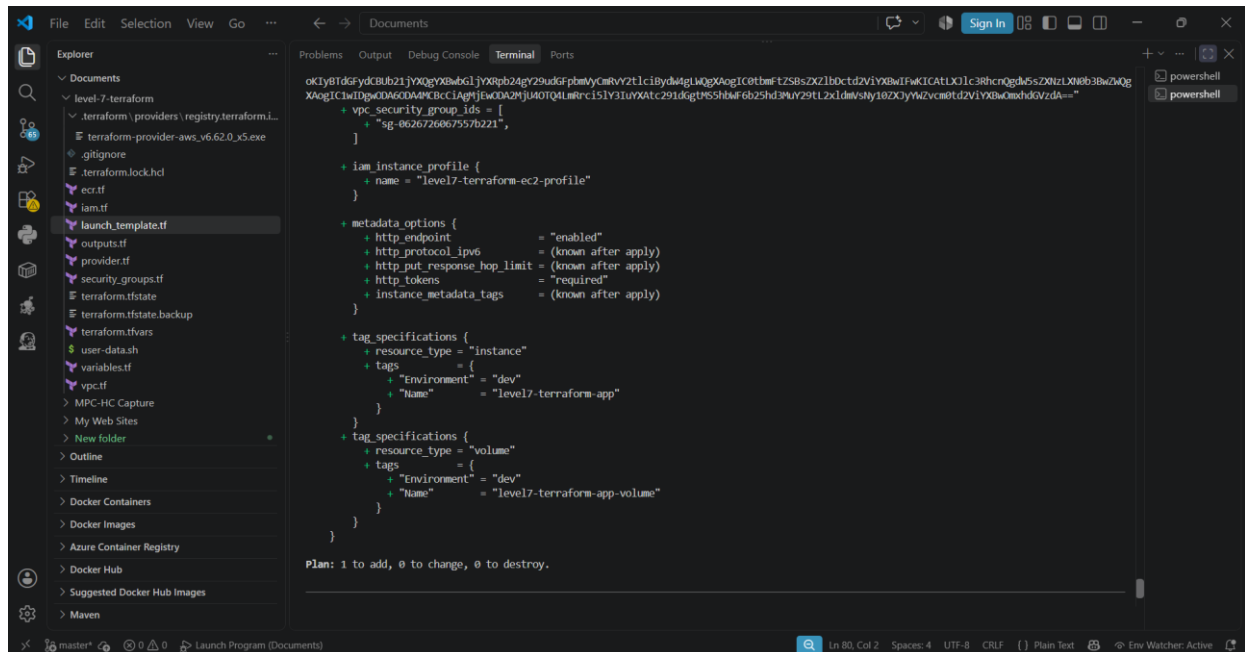


Figure 68. Complete Launch Template configuration.

Step 68 : Verify the Launch Template in AWS

Implementation Action / What I did: Configure the Launch Template or user-data component shown in the screenshot.

What the screenshot proves: The screenshot proves the EC2 startup configuration is defined or available in AWS.

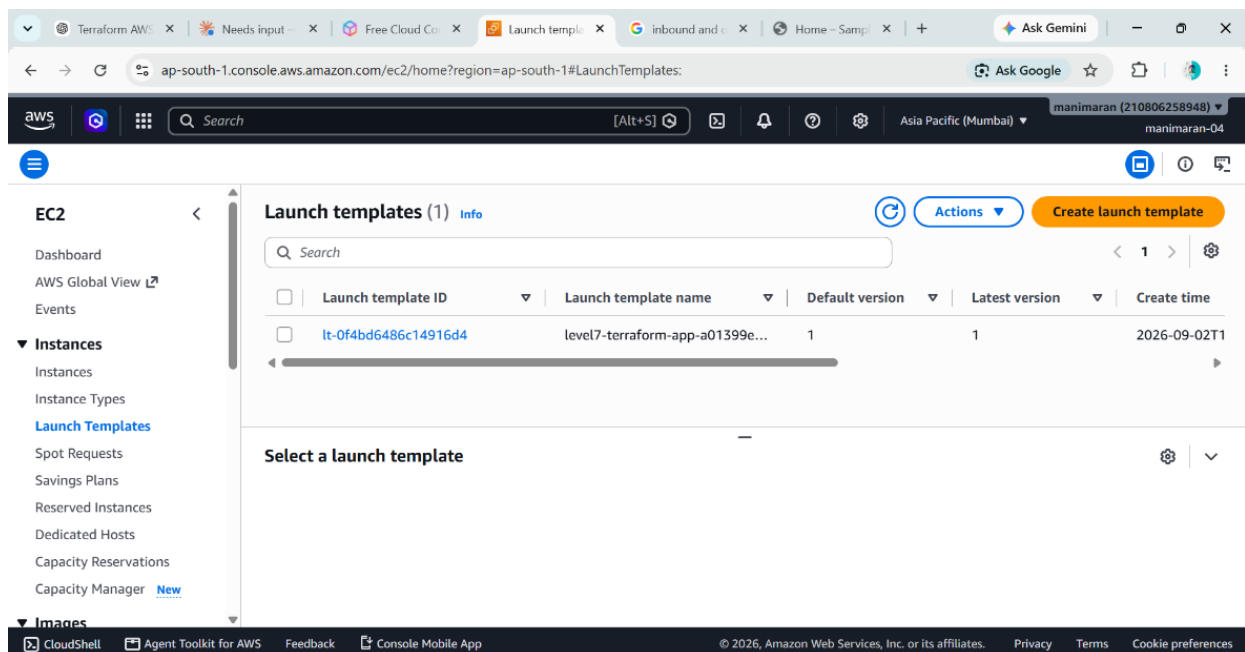


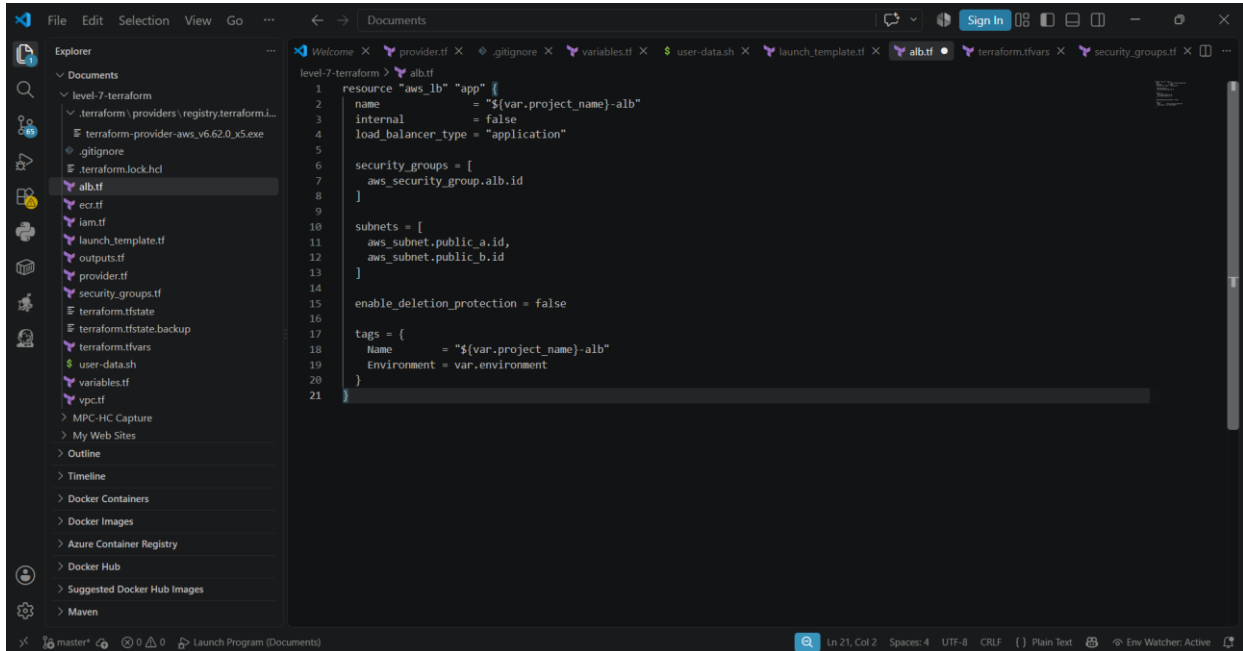
Figure 69. Verify the Launch Template in AWS.

Phase 8 – Application Load Balancer and Target Group

Step 69 : Define the Application Load Balancer

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



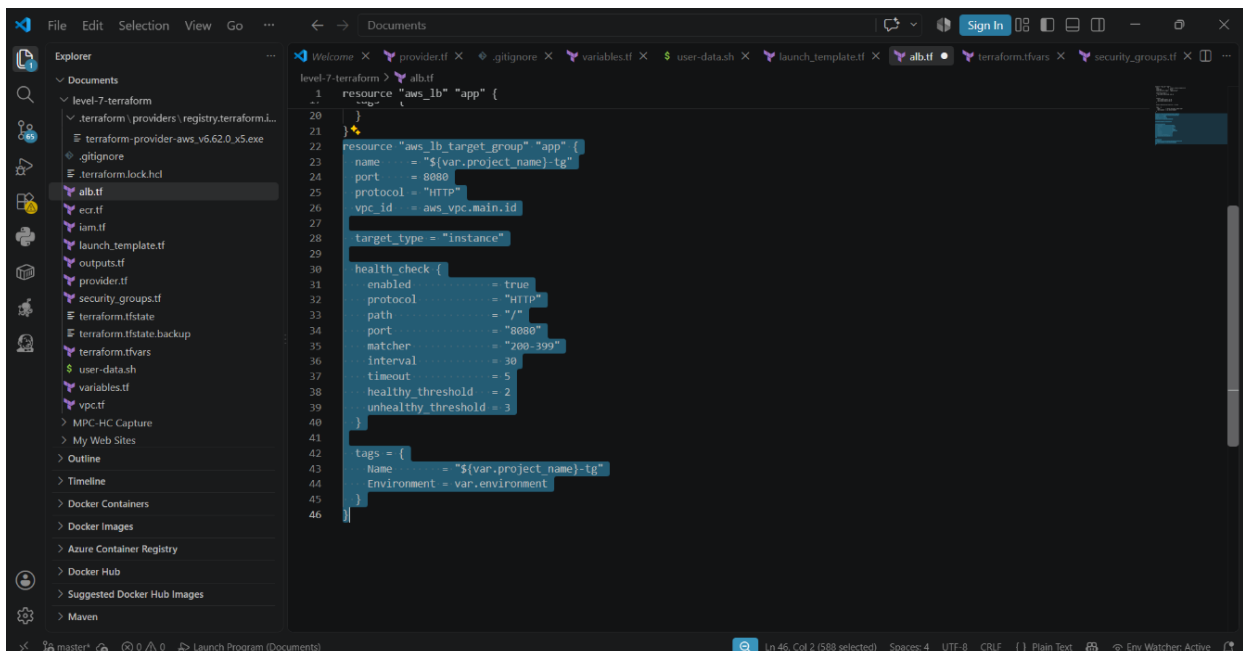
```
1 resource "aws_lb" "app" {
2   name = "${var.project_name}-alb"
3   internal = false
4   load_balancer_type = "application"
5
6   security_groups = [
7     aws_security_group.alb.id
8   ]
9
10  subnets = [
11    aws_subnet.public_a.id,
12    aws_subnet.public_b.id
13  ]
14
15  enable_deletion_protection = false
16
17  tags = {
18    Name = "${var.project_name}-alb"
19    Environment = var.environment
20  }
21 }
```

Figure 70. Define the Application Load Balancer.

Step 70 : Configure ALB subnet mapping

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



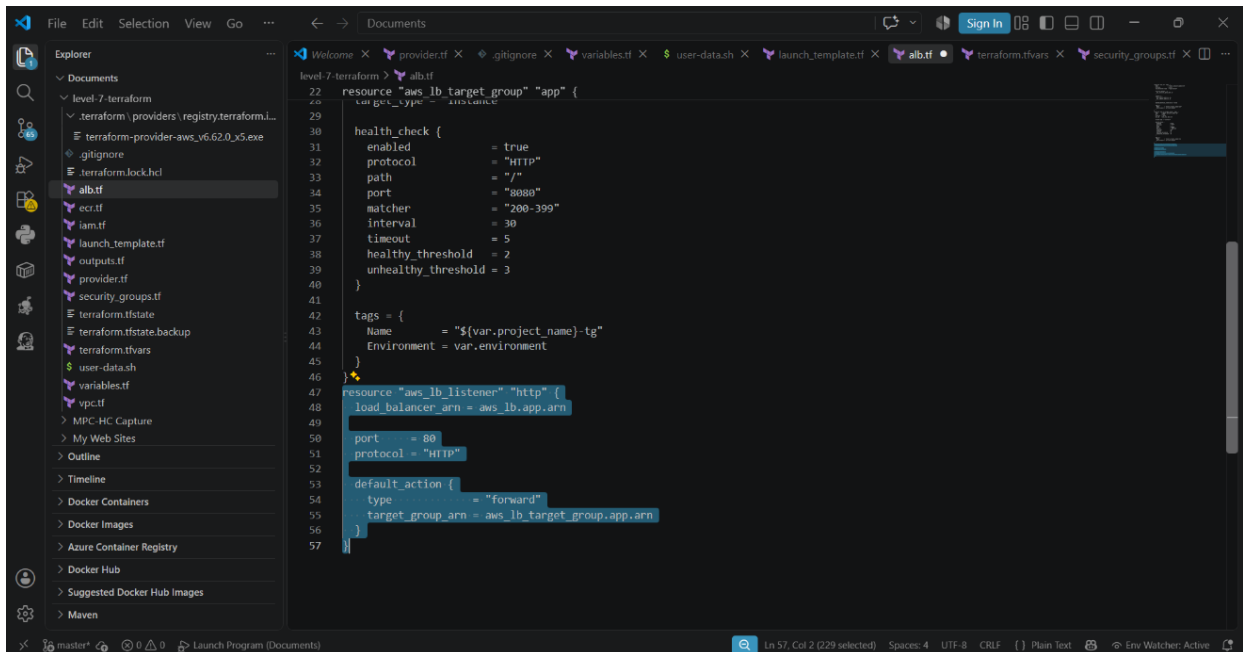
```
20 }
21
22 resource "aws_lb_target_group" "app" {
23   name = "${var.project_name}-tg"
24   port = 8080
25   protocol = "HTTP"
26   vpc_id = aws_vpc.main.id
27
28   target_type = "instance"
29
30   health_check {
31     enabled = true
32     protocol = "HTTP"
33     path = "/"
34     port = "8080"
35     matcher = "200-399"
36     interval = 30
37     timeout = 5
38     healthy_threshold = 2
39     unhealthy_threshold = 3
40   }
41
42   tags = {
43     Name = "${var.project_name}-tg"
44     Environment = var.environment
45   }
46 }
```

Figure 71 Configure ALB subnet mapping.

Step 71 : Define the target group

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like `level-7-terraform`, `terraform-provider-aws_v6.62.0_x5.exe`, `.gitignore`, `terraform.lock.hcl`, `alb.tf`, `ecr.tf`, `iam.tf`, `launch_template.tf`, `outputs.tf`, `provider.tf`, `security_groups.tf`, `terraform.tfstate`, `terraform.tfstate.backup`, `terraform.tvvars`, `user-data.sh`, `variables.tf`, `vpctf`, `MPC-HC Capture`, `My Web Sites`, `Outline`, `Timeline`, `Docker Containers`, `Docker Images`, `Azure Container Registry`, `Docker Hub`, `Suggested Docker Hub Images`, and `Maven`. The code editor shows the `alb.tf` file with the following content:

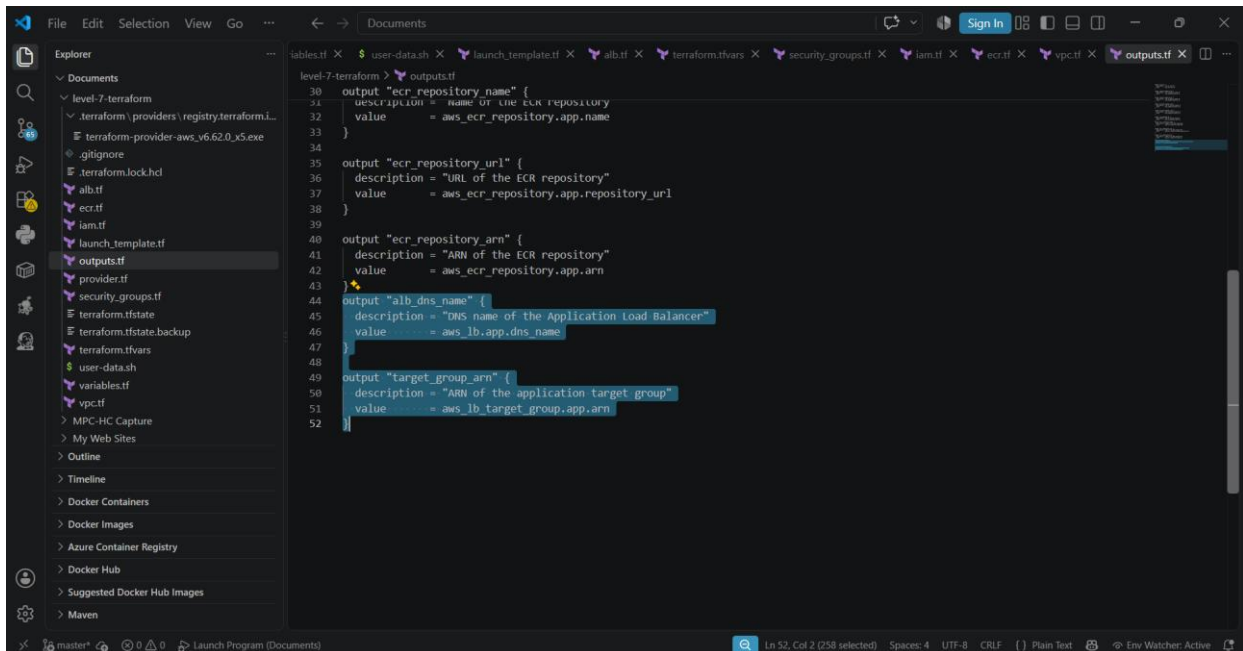
```
22 resource "aws_lb_target_group" "app" {
23   target_type = "instance"
24
25   health_check {
31     enabled = true
32     protocol = "HTTP"
33     path = "/"
34     port = "8080"
35     matcher = "200-399"
36     interval = 30
37     timeout = 5
38     healthy_threshold = 2
39     unhealthy_threshold = 3
40   }
41
42   tags = {
43     Name = "${var.project_name}-tg"
44     Environment = var.environment
45   }
46 }
47
48 resource "aws_lb_listener" "http" {
49   load_balancer_arn = aws_lb.app.arn
50
51   port = 80
52   protocol = "HTTP"
53
54   default_action {
55     type = "forward"
56     target_group_arn = aws_lb_target_group.app.arn
57   }
58 }
```

Figure 72. Define the target group.

Step 72 : Configure target health checks

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like `level-7-terraform`, `terraform-provider-aws_v6.62.0_x5.exe`, `.gitignore`, `terraform.lock.hcl`, `alb.tf`, `ecr.tf`, `iam.tf`, `launch_template.tf`, `outputs.tf`, `provider.tf`, `security_groups.tf`, `terraform.tfstate`, `terraform.tfstate.backup`, `terraform.tvvars`, `user-data.sh`, `variables.tf`, `vpctf`, `MPC-HC Capture`, `My Web Sites`, `Outline`, `Timeline`, `Docker Containers`, `Docker Images`, `Azure Container Registry`, `Docker Hub`, `Suggested Docker Hub Images`, and `Maven`. The code editor shows the `outputs.tf` file with the following content:

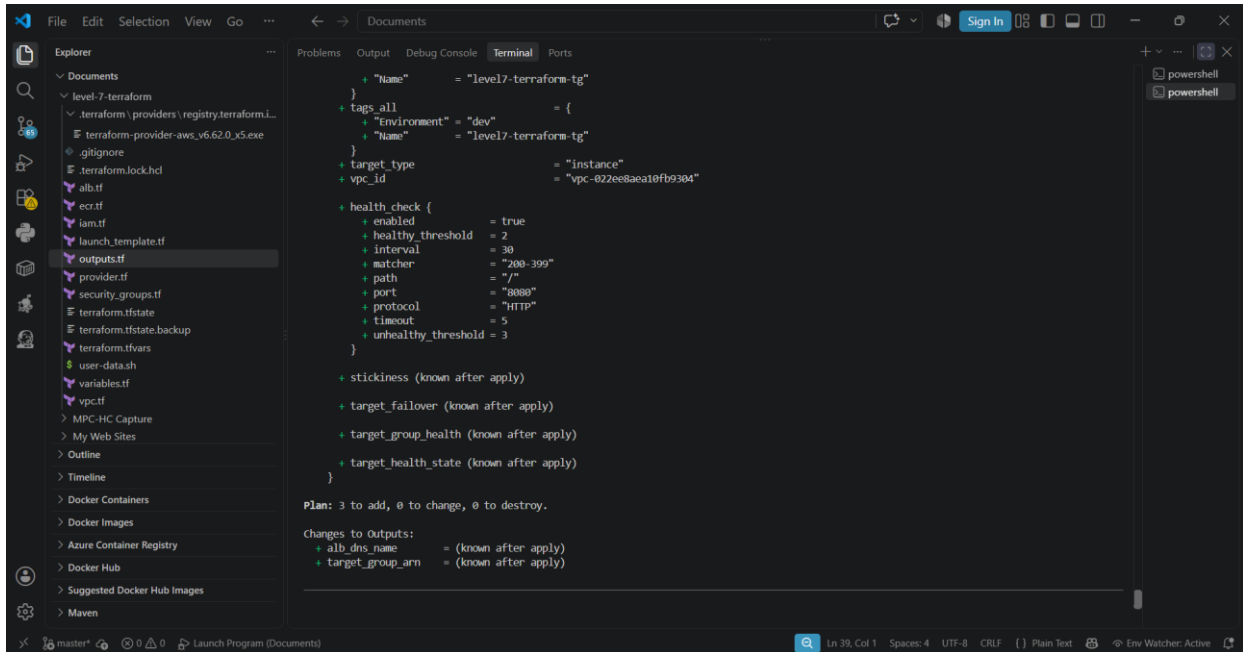
```
30 output "ecr_repository_name" {
31   description = "name of the ECR repository"
32   value = aws_ecr_repository.app.name
33 }
34
35 output "ecr_repository_url" {
36   description = "URL of the ECR repository"
37   value = aws_ecr_repository.app.repository_url
38 }
39
40 output "ecr_repository_arn" {
41   description = "ARN of the ECR repository"
42   value = aws_ecr_repository.app.arn
43 }
44
45 output "alb_dns_name" {
46   description = "DNS name of the Application load Balancer"
47   value = aws_lb.app.dns_name
48 }
49
50 output "target_group_arn" {
51   description = "ARN of the application target group"
52   value = aws_lb_target_group.app.arn
53 }
```

Figure 73. Configure target health checks.

Step 73 : Define the ALB listener

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



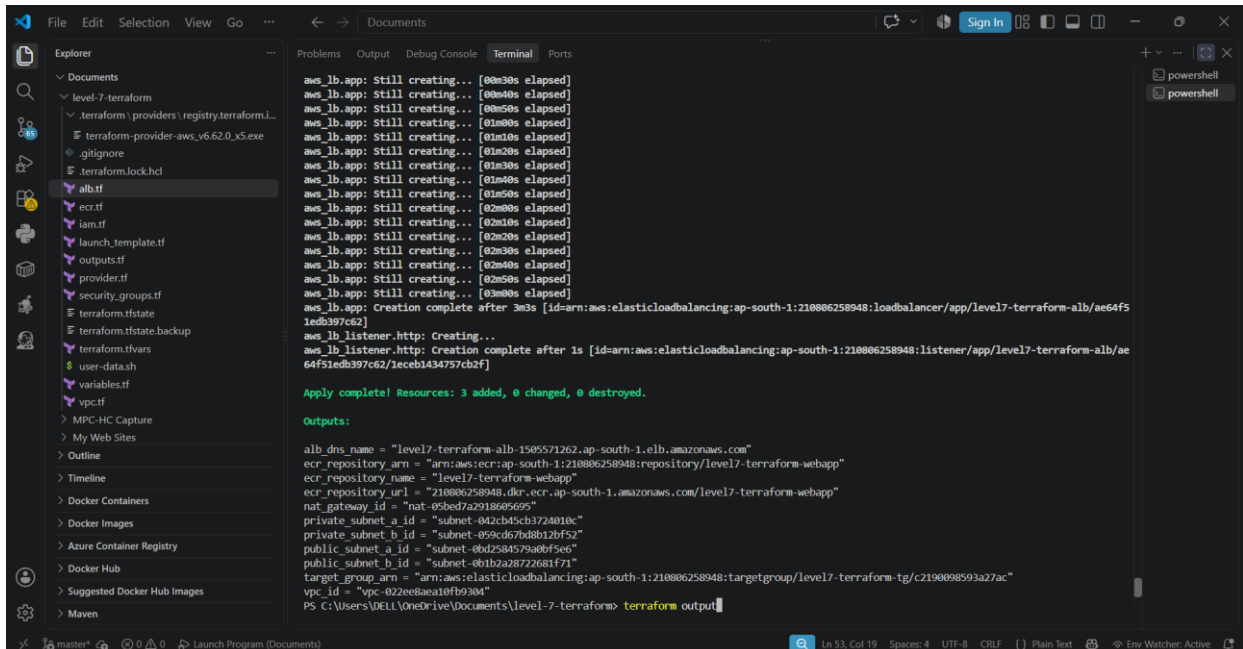
```
File Edit Selection View Go ... Documents
+ Sign In
Explorer
  Documents
  level-7-terraform
    terraform\providers\registry.terraform.I...
    terraform-provider-aws_v6.62.0_x5.exe
    .gitignore
    terraform.lock.hcl
    alb.tf
    ecr.tf
    iam.tf
    launch_template.tf
    outputs.tf
    provider.tf
    security_groups.tf
    terraform.tfstate
    terraform.tfstate.backup
    terraform.tfvars
    user-data.sh
    variables.tf
    vpc.tf
    MPC-HC Capture
    My Web Sites
    Outline
    Timeline
    Docker Containers
    Docker Images
    Azure Container Registry
    Docker Hub
    Suggested Docker Hub Images
    Maven
Problems Output Debug Console Terminal Ports
  + "Name" = "level7-terraform-tg"
  + tags.all = {
    + "Environment" = "dev"
    + "Name" = "level7-terraform-tg"
  }
  + target_type = "instance"
  + vpc_id = "vpc-022ee8aea10fb9304"
  + health_check {
    + enabled = true
    + healthy_threshold = 2
    + interval = 30
    + matcher = "200-399"
    + path = "/"
    + port = "8080"
    + protocol = "HTTP"
    + timeout = 5
    + unhealthy_threshold = 3
  }
  + stickiness (known after apply)
  + target_failover (known after apply)
  + target_group_health (known after apply)
  + target_health_state (known after apply)
}
Plan: 3 to add, 0 to change, 0 to destroy.
Changes to Outputs:
  + alb_dns_name = (known after apply)
  + target_group_arn = (known after apply)
```

Figure 74. Define the ALB listener.

Step 74 : Configure ALB outputs

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



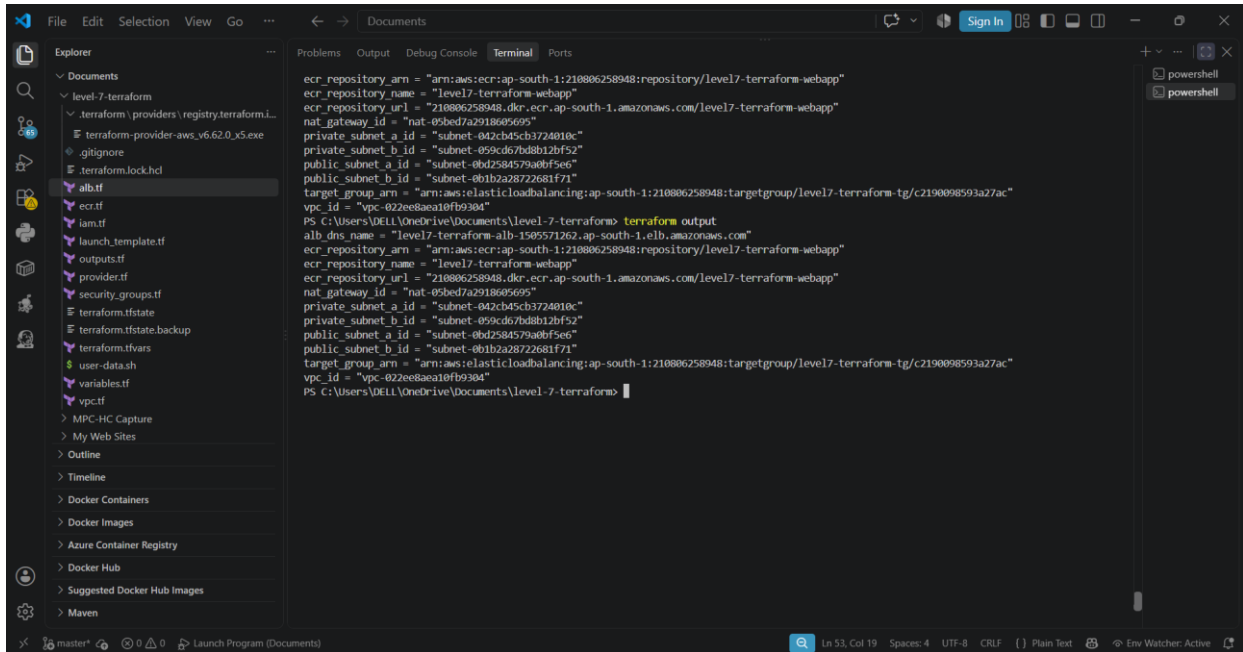
```
File Edit Selection View Go ... Documents
+ Sign In
Explorer
  Documents
  level-7-terraform
    terraform\providers\registry.terraform.I...
    terraform-provider-aws_v6.62.0_x5.exe
    .gitignore
    terraform.lock.hcl
    alb.tf
    ecr.tf
    iam.tf
    launch_template.tf
    outputs.tf
    provider.tf
    security_groups.tf
    terraform.tfstate
    terraform.tfstate.backup
    terraform.tfvars
    user-data.sh
    variables.tf
    vpc.tf
    MPC-HC Capture
    My Web Sites
    Outline
    Timeline
    Docker Containers
    Docker Images
    Azure Container Registry
    Docker Hub
    Suggested Docker Hub Images
    Maven
Problems Output Debug Console Terminal Ports
  aws_lb.app: Still creating... [00m30s elapsed]
  aws_lb.app: Still creating... [00m40s elapsed]
  aws_lb.app: Still creating... [00m50s elapsed]
  aws_lb.app: Still creating... [01m00s elapsed]
  aws_lb.app: Still creating... [01m10s elapsed]
  aws_lb.app: Still creating... [01m20s elapsed]
  aws_lb.app: Still creating... [01m30s elapsed]
  aws_lb.app: Still creating... [01m40s elapsed]
  aws_lb.app: Still creating... [01m50s elapsed]
  aws_lb.app: Still creating... [02m00s elapsed]
  aws_lb.app: Still creating... [02m10s elapsed]
  aws_lb.app: Still creating... [02m20s elapsed]
  aws_lb.app: Still creating... [02m30s elapsed]
  aws_lb.app: Still creating... [02m40s elapsed]
  aws_lb.app: Still creating... [02m50s elapsed]
  aws_lb.app: Still creating... [03m00s elapsed]
  aws_lb.app: Creation complete after 3m3s [id=arn:aws:elasticloadbalancing:ap-south-1:210806258948:loadbalancer/app/level7-terraform-alb/ae64f51ed397c62]
  aws_lb_listener.http: Creating...
  aws_lb_listener.http: Creation complete after 1s [id=arn:aws:elasticloadbalancing:ap-south-1:210806258948:listener/app/level7-terraform-alb/ae64f51ed397c62/1ecb1434757cb2f]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
Outputs:
alb_dns_name = "level7-terraform-alb-1505571262.ap-south-1.elb.amazonaws.com"
ecr_repository_arn = "arn:aws:ecr:ap-south-1:210806258948:repository/level7-terraform-webapp"
ecr_repository_name = "level7-terraform-webapp"
ecr_repository_url = "210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp"
nat_gateway_id = "nat-05bed7a2918605695"
private_subnet_a_id = "subnet-042cb5c3724010c"
private_subnet_b_id = "subnet-059c0670d8012bf52"
public_subnet_a_id = "subnet-0d2d847930d4f5e6"
public_subnet_b_id = "subnet-0b1b2a2872681f71"
target_group_arn = "arn:aws:elasticloadbalancing:ap-south-1:210806258948:targetgroup/level7-terraform-tg/c2190098593a27ac"
vpc_id = "vpc-022ee8aea10fb9304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform> terraform output
```

Figure 75. Configure ALB outputs.

Step 75 : Complete ALB/target-group dependencies

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.



```
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform> terraform output
alb_dns_name = "level7-terraform-alb-1595571262.ap-south-1.elb.amazonaws.com"
ecr_repository_arn = "arn:aws:ecr:ap-south-1:210806258948:repository/level7-terraform-webapp"
ecr_repository_name = "level7-terraform-webapp"
ecr_repository_url = "210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp"
nat_gateway_id = "nat-05bed7a2918605695"
private_subnet_a_id = "subnet-042cb45cb3724010c"
private_subnet_b_id = "subnet-059cd67bd8b12bf52"
public_subnet_a_id = "subnet-0bd2584579a0bf5e6"
public_subnet_b_id = "subnet-0bd2584579a0bf5e6"
target_group_arn = "arn:aws:elasticloadbalancing:ap-south-1:210806258948:targetgroup/level7-terraform-tg/c2190098593a27ac"
vpc_id = "vpc-022ee8a010fb9304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure Complete ALB/target-group dependencies.

Step 76 : Verify the load balancer in AWS

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.

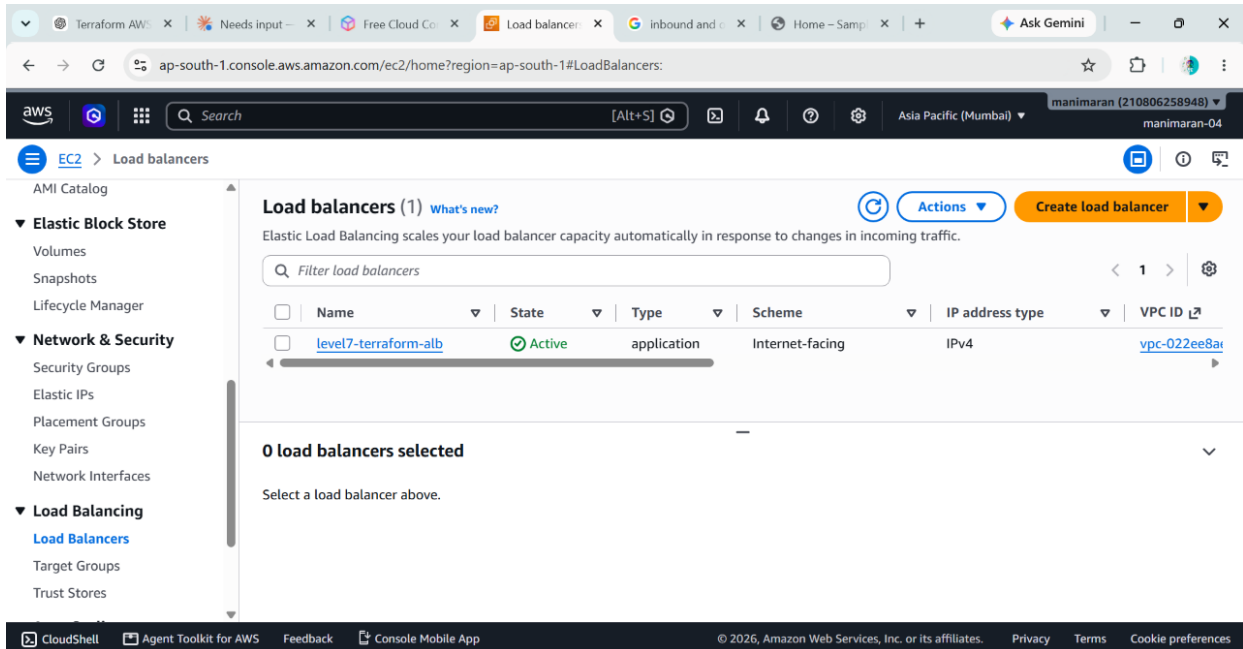


Figure 77. Verify the load balancer in AWS.

Step 77 : Verify the ALB listener and routing

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.

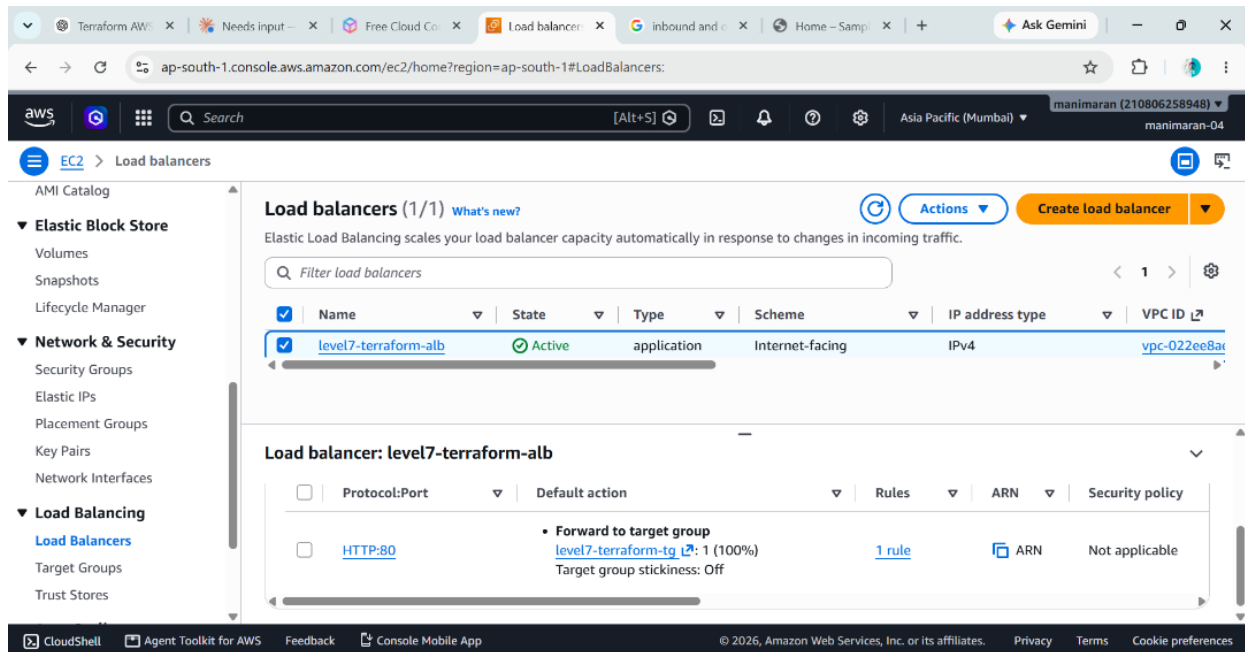


Figure 78. Verify the ALB listener and routing.

Step 78 : Verify the target group

Implementation Action / What I did: Configure or verify the ALB/target-group component shown in the screenshot.

What the screenshot proves: The screenshot proves the load-balancing layer is configured.

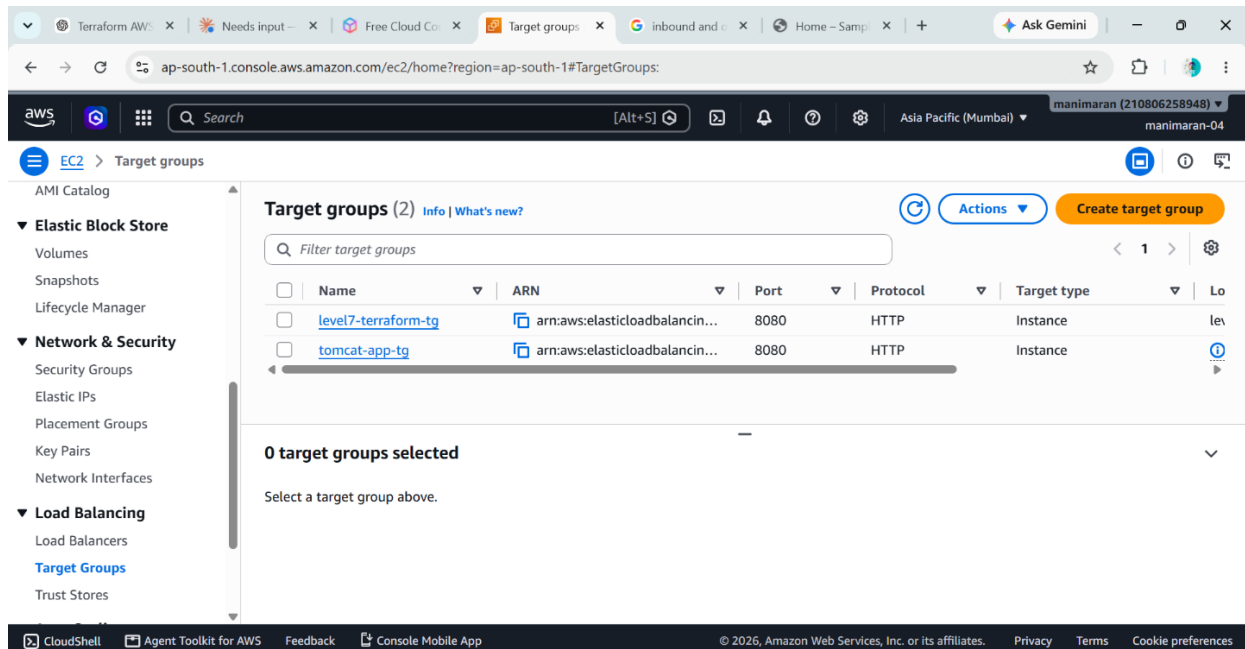


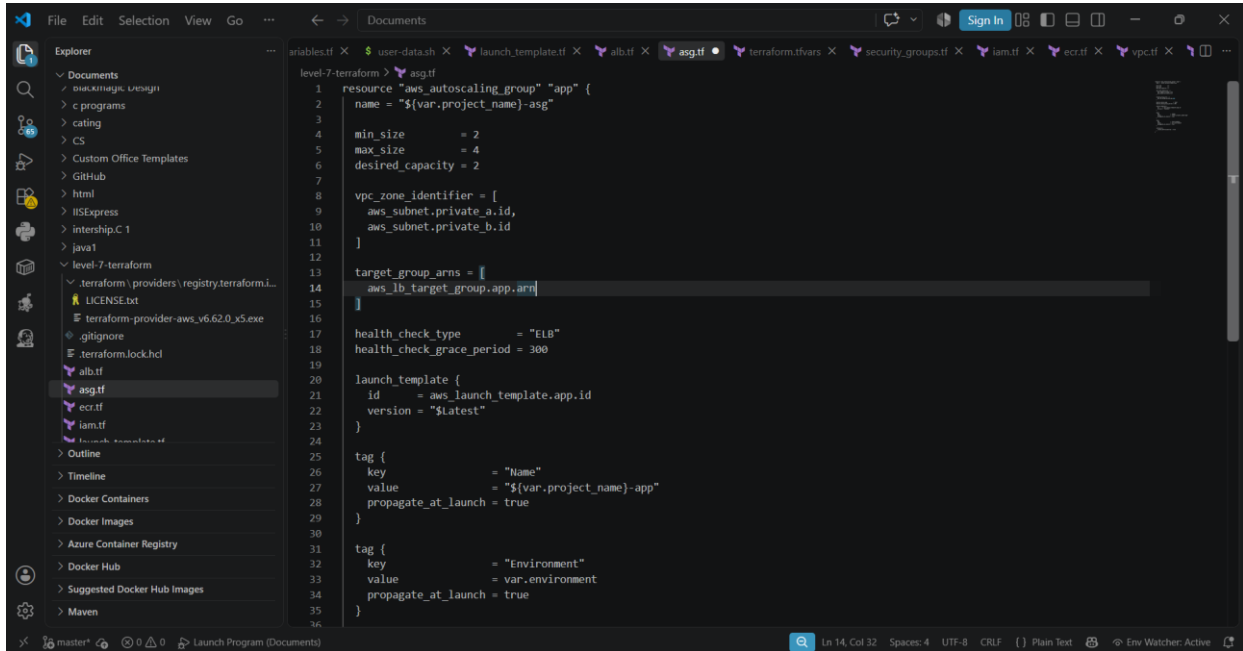
Figure 79. Verify the target group.

Phase 9 – Auto Scaling and Target Health

Step 79 : Define the Auto Scaling Group

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.



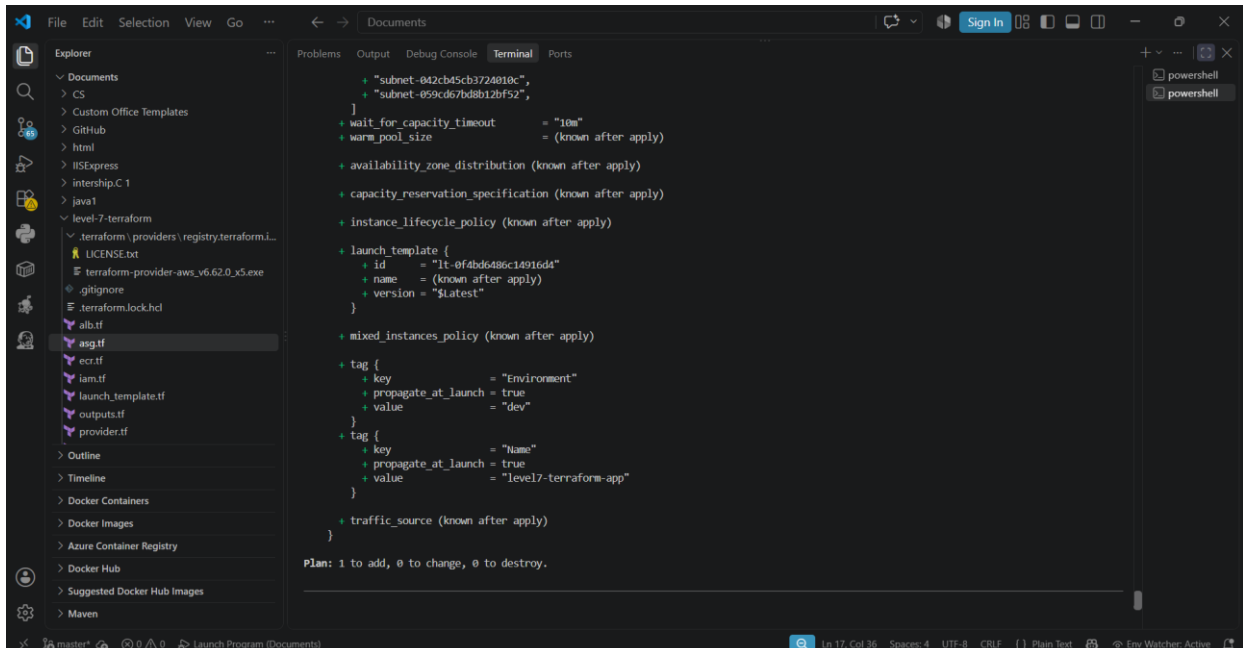
```
1 resource "aws_autoscaling_group" "app" {
2   name = "${var.project_name}-asg"
3
4   min_size         = 2
5   max_size         = 4
6   desired_capacity = 2
7
8   vpc_zone_identifier = [
9     aws_subnet.private_a_id,
10    aws_subnet.private_b_id
11  ]
12
13  target_group_arns = [
14    aws_lb_target_group.app.arn
15  ]
16
17  health_check_type           = "ELB"
18  health_check_grace_period = 300
19
20  launch_template {
21    id       = aws_launch_template.app.id
22    version = "$latest"
23  }
24
25  tag {
26    key                 = "Name"
27    value               = "${var.project_name}-app"
28    propagate_at_launch = true
29  }
30
31  tag {
32    key                 = "Environment"
33    value               = var.environment
34    propagate_at_launch = true
35  }
36}
```

Figure 80. Define the Auto Scaling Group.

Step 80 : Configure ASG subnets and target group

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.



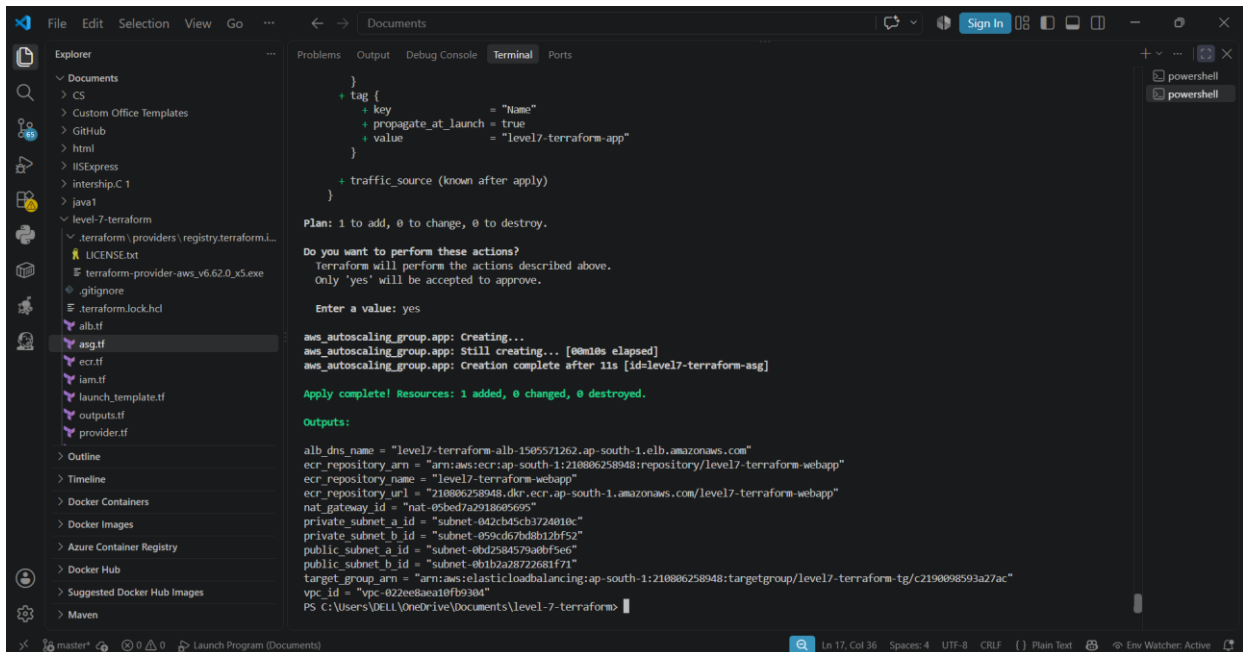
```
1 resource "aws_autoscaling_group" "app" {
2   name = "${var.project_name}-asg"
3
4   min_size         = 2
5   max_size         = 4
6   desired_capacity = 2
7
8   vpc_zone_identifier = [
9     "subnet-042cb45cb3724010c",
10    "subnet-059cd67bd8b12bf52",
11  ]
12
13  target_group_arns = [
14    aws_lb_target_group.app.arn
15  ]
16
17  health_check_type           = "ELB"
18  health_check_grace_period = 300
19
20  launch_template {
21    id       = "lt-0f4bd6486c14916d4"
22    name     = "(known after apply)"
23    version = "$latest"
24  }
25
26  mixed_instances_policy {
27    instances_distribution {
28      on_demand_allocation_strategy            = "Spread"
29      max_on_demand_percentage_in_protection = 100
30      exclude_instances_from_protection      = [
31        "i-04078c380d6618f52",
32        "i-0182c87b677c72627"
33      ]
34    }
35  }
36
37  tag {
38    key                 = "Environment"
39    value               = "dev"
40    propagate_at_launch = true
41  }
42
43  tag {
44    key                 = "Name"
45    value               = "level7-terraform-app"
46    propagate_at_launch = true
47  }
48
49  traffic_source {
50    type = "TrafficSource"
51  }
52}
```

Figure 81. Configure ASG subnets and target group.

Step 81 : Configure ASG capacity and health checks

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.



```
File Edit Selection View Go ... Documents
Explorer
  Documents
  CS
  Custom Office Templates
  GitHub
  html
  IISExpress
  intership.C.1
  java1
  level-7-terraform
    terraform\providers\registry.terraform.i...
    LICENSE.txt
    terraform-provider-aws_v6.62.0_x5.exe
    .gitignore
    terraform.lock.hcl
    alb.tf
    asg.tf
    ecr.tf
    iam.tf
    launch_template.tf
    outputs.tf
    provider.tf
  Outline
  Timeline
  Docker Containers
  Docker Images
  Azure Container Registry
  Docker Hub
  Suggested Docker Hub Images
  Maven
Problems Output Debug Console Terminal Ports
+ tag {
+   + key           = "Name"
+   + propagate_at_launch = true
+   + value          = "level7-terraform-app"
+ }
+ traffic_source (known after apply)
}
Plan: 1 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_autoscaling_group.app: Creating...
aws_autoscaling_group.app: Still creating... [00m10s elapsed]
aws_autoscaling_group.app: Creation complete after 11s [id=level7-terraform-asg]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
alb_dns_name = "level7-terraform-alb-1585571262-ap-south-1.elb.amazonaws.com"
ecr_repository_arn = "arn:aws:ecr:ap-south-1:210806258948:repository/level7-terraform-webapp"
ecr_repository_name = "level7-terraform-webapp"
ecr_repository_url = "210806258948.dkr.ecr.ap-south-1.amazonaws.com/level7-terraform-webapp"
nat_gateway_id = "nat-05bed7a2918605695"
private_subnet_a_id = "subnet-042cb5cb3724010c"
private_subnet_b_id = "subnet-059cd67bd8b12bf52"
public_subnet_a_id = "subnet-0bd2584579a0bf5e6"
public_subnet_b_id = "subnet-0b1b2a287226a1f71"
target_group_arn = "arn:aws:elasticloadbalancing:ap-south-1:210806258948:targetgroup/level7-terraform-tg/c2190098593a27ac"
vpc_id = "vpc-022ee8aea10f9a304"
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure 82. Configure ASG capacity and health checks.

Step 82 : Apply the Auto Scaling configuration

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.

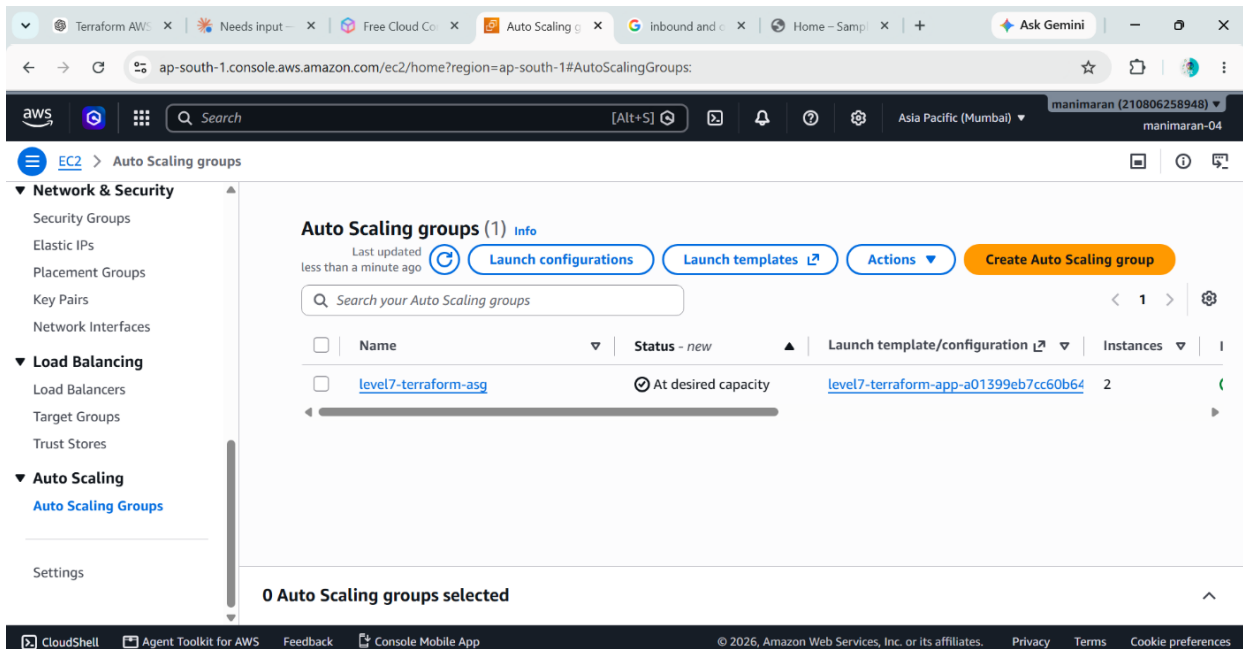


Figure 83. Apply the Auto Scaling configuration.

Step 84 : Verify Auto Scaling Group settings

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.

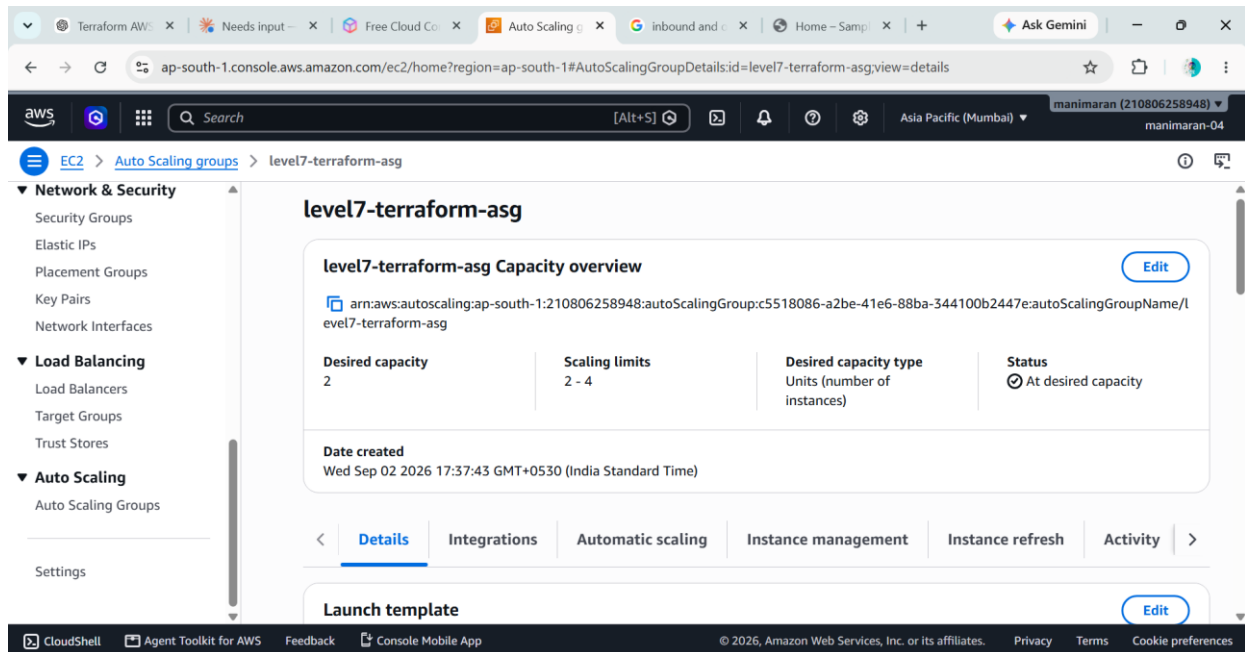


Figure 84. Verify Auto Scaling Group settings.

Step 85 : Verify running EC2 instances

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.

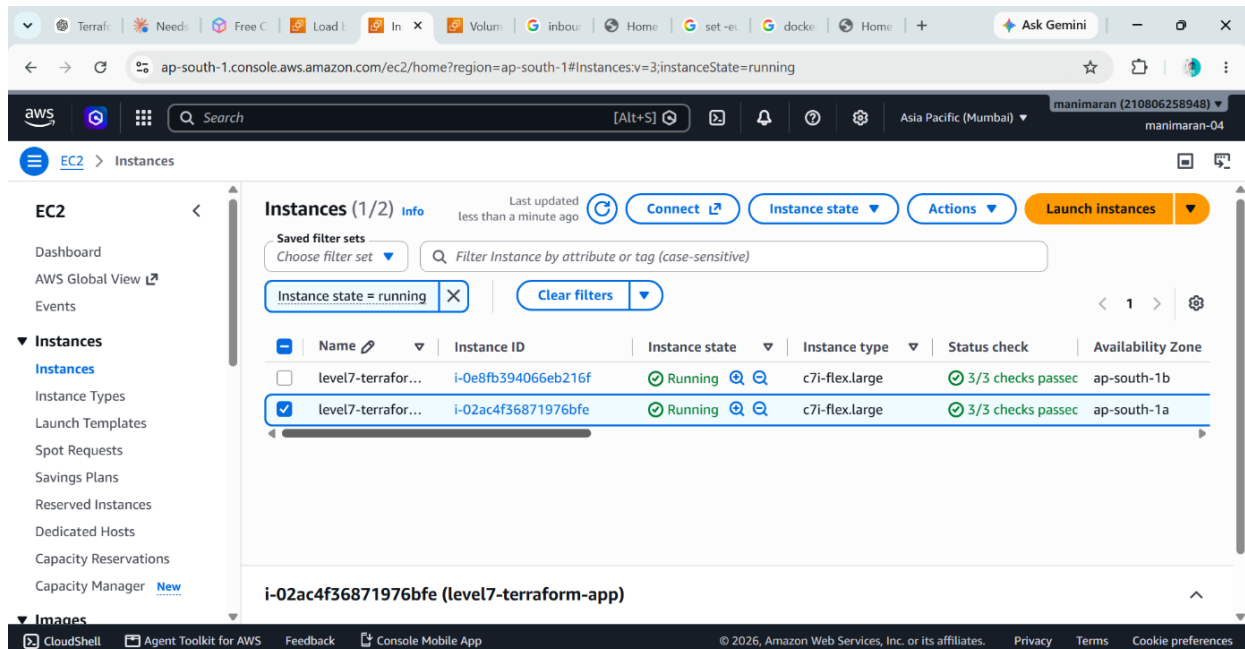


Figure 85. Verify running EC2 instances.

Step 86 : Verify target health

Implementation Action / What I did: Configure, apply or verify the Auto Scaling component shown in the screenshot.

What the screenshot proves: The screenshot proves the application fleet and load-balancing health are configured/verified.

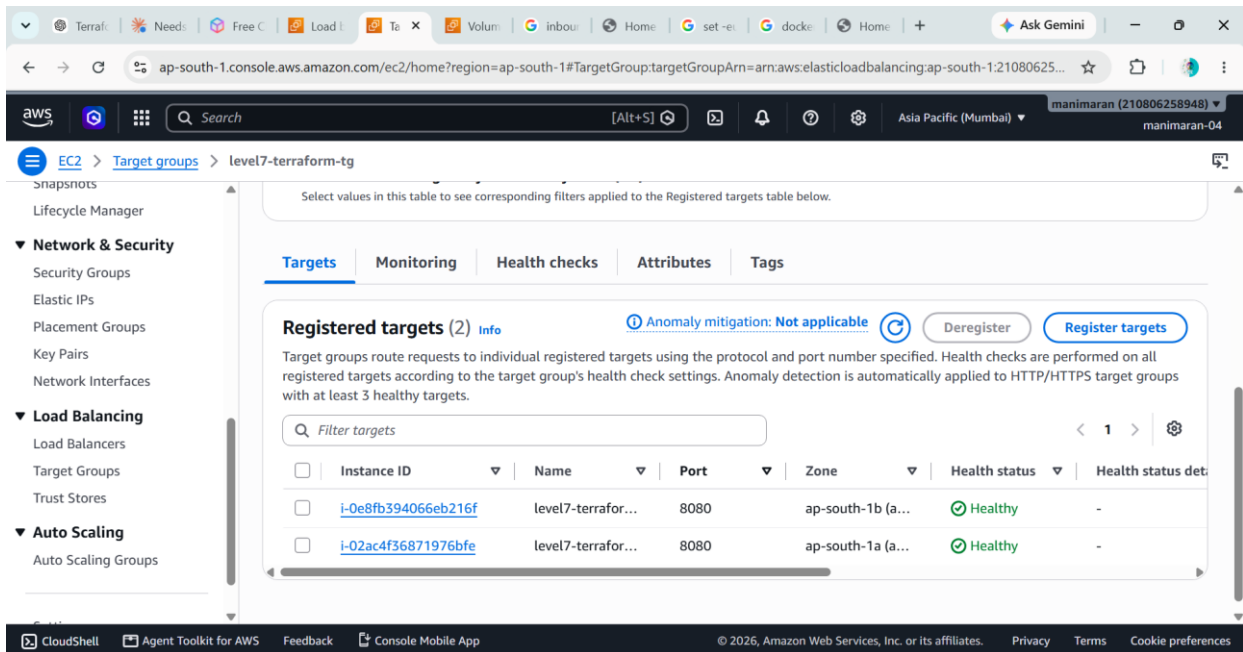


Figure 86. Verify target healthy

Phase 10 – Application Validation

Step 87 : Validate the SampleApp Home page

Implementation Action / What I did: Open the ALB endpoint and verify the Home page.

What the screenshot proves: The application is reachable end-to-end through the ALB.

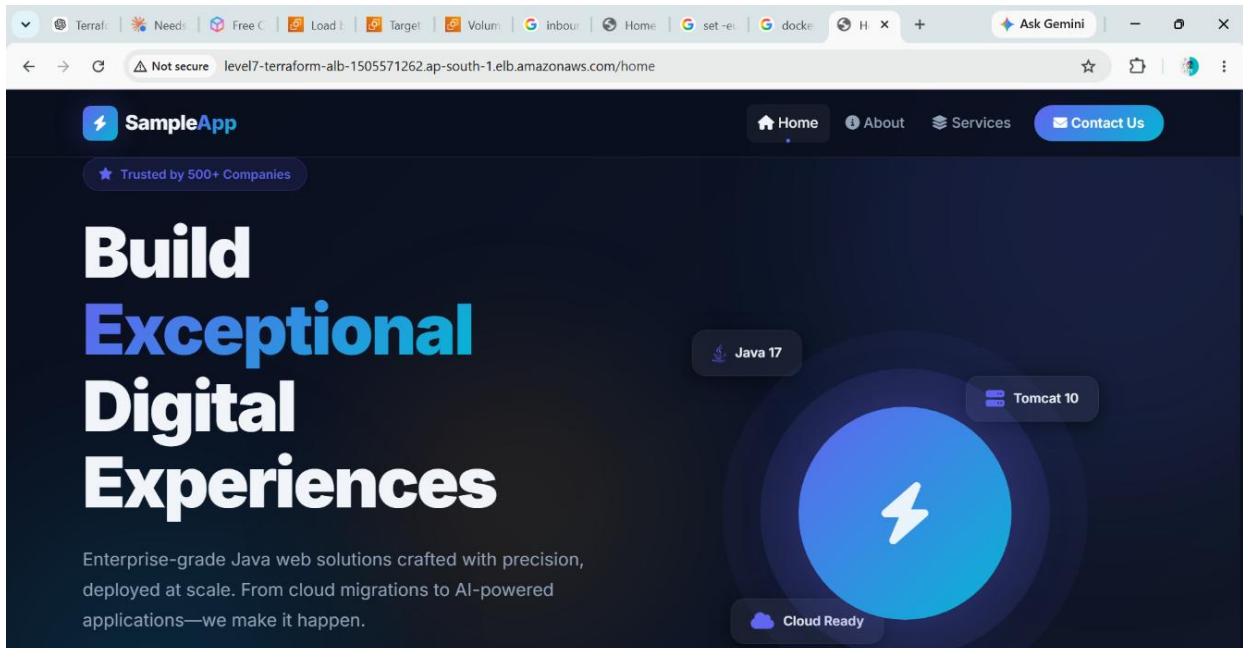


Figure 87. Validate the SampleApp Home page.

Step 88 : Validate the SampleApp About Us page

Implementation Action / What I did: Open the About Us page through the ALB endpoint.

What the screenshot proves: Another application route renders successfully.

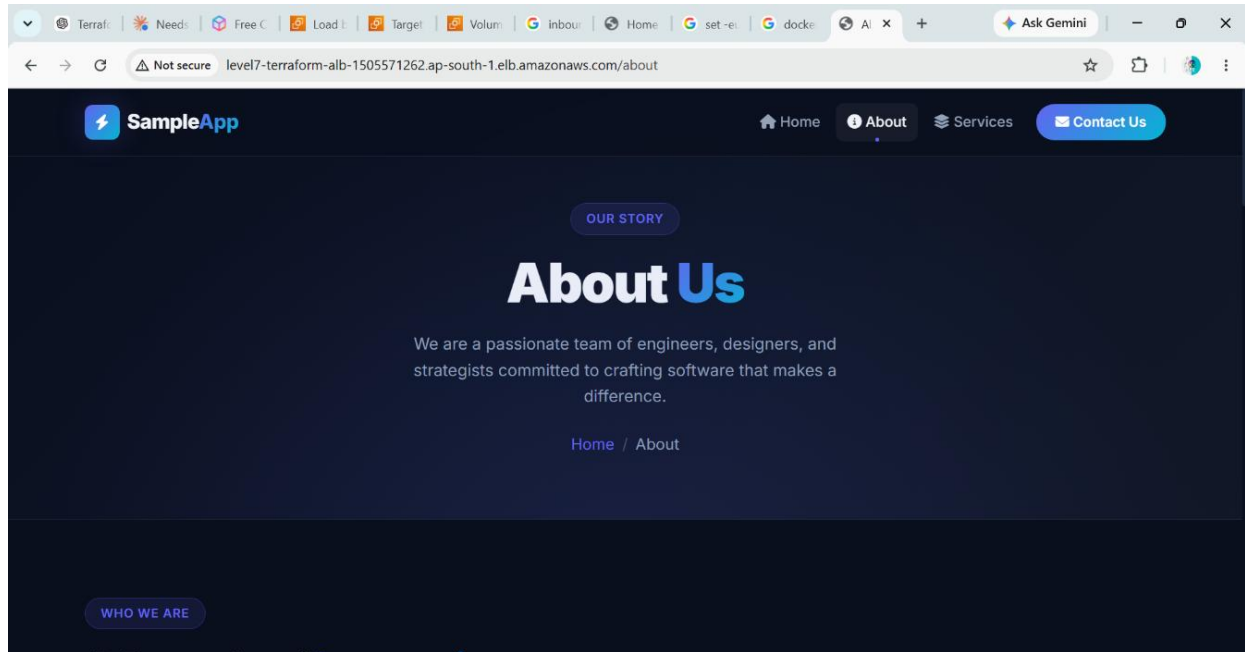


Figure 88. Validate the SampleApp About Us page.

Step 89 : Validate the SampleApp Services page

Implementation Action / What I did: Open the Services page through the ALB endpoint.

What the screenshot proves: The deployed application is functioning through the load-balanced path.

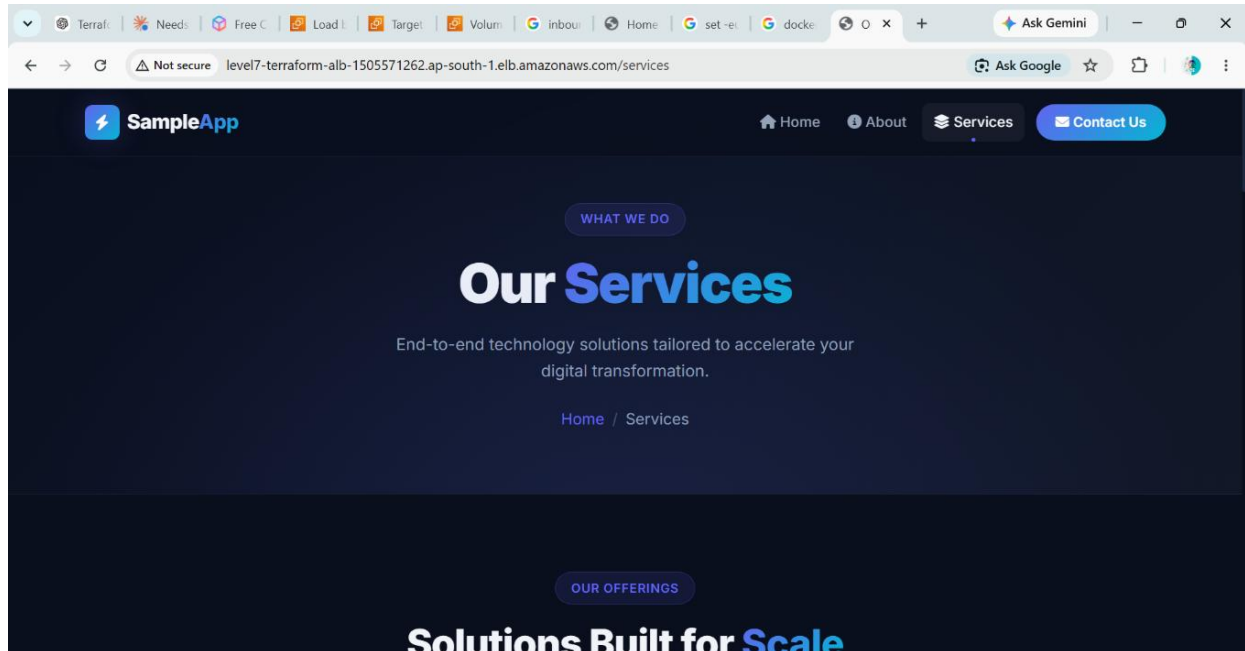


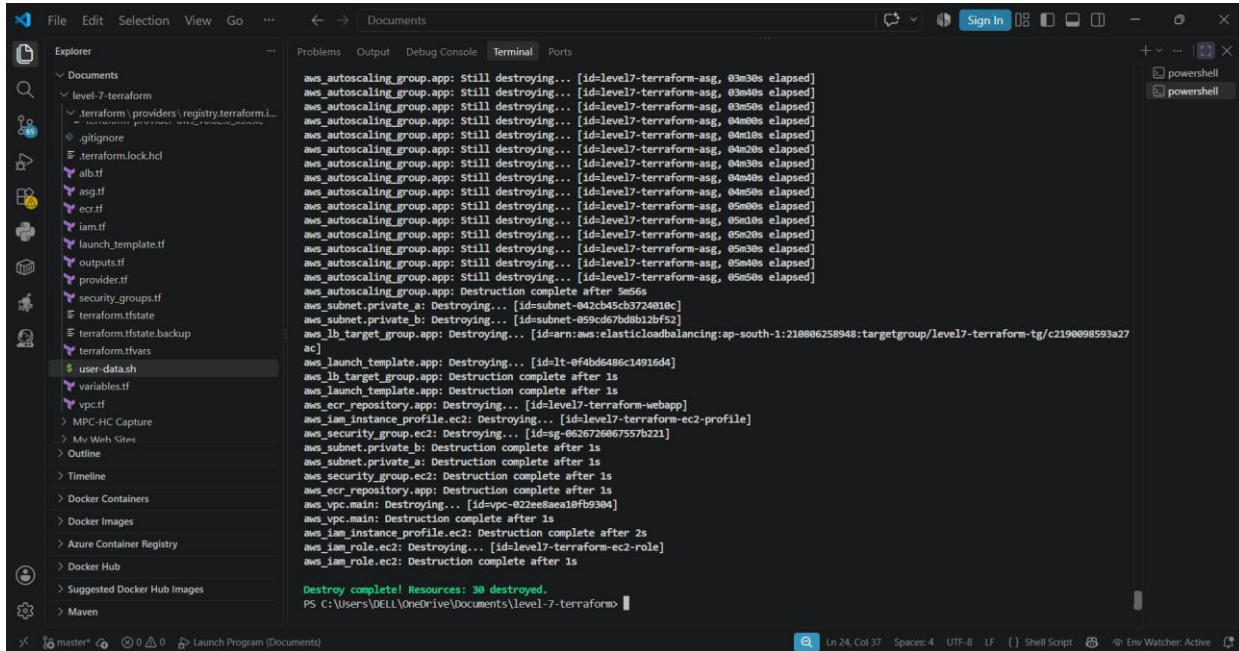
Figure 89. Validate the SampleApp Services page.

Phase 11 – Terraform Cleanup

Step 90 : Destroy the lab infrastructure

Implementation Action / What I did: Run terraform destroy after validation and confirm resource cleanup.

What the screenshot proves: The Terraform cleanup lifecycle is shown.



```
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 03m30s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 03m40s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 03m50s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 04m00s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 04m10s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 04m20s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 04m30s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 04m40s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 04m50s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 05m00s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 05m10s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 05m20s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 05m30s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 05m40s elapsed]
aws_autoscaling_group.app: Still destroying... [id=level7-terraform-asg, 05m50s elapsed]
aws_autoscaling_group.app: Destruction complete after 5m56s
aws_subnet.private_a: Destroying... [id=subnet-042cb45cb3724810c]
aws_subnet.private_b: Destroying... [id=subnet-059cd67bd8b12bf52]
aws_lb_target_group.app: Destroying... [id=arn:aws:elasticloadbalancing:ap-south-1:218806258948:targetgroup/level7-terraform-tg-c190098593a27
ec]
aws_launch_template.app: Destroying... [id=lt-0f4bd6486c14916d4]
aws_lb_target_group.app: Destruction complete after 1s
aws_launch_template.app: Destruction complete after 1s
aws_ecr_repository.app: Destroying... [id=level7-terraform-webapp]
aws_iam_instance_profile.ec2: Destroying... [id=level7-terraform-ec2-profile]
aws_security_group.ec2: Destroying... [id=sg-062672667557b221]
aws_subnet.private_b: Destruction complete after 1s
aws_subnet.private_a: Destruction complete after 1s
aws_security_group.ec2: Destruction complete after 1s
aws_ecr_repository.app: Destruction complete after 1s
aws_vpc.main: Destroying... [id=vpc-022ee8aa18fb9304]
aws_vpc.main: Destruction complete after 1s
aws_iam_instance_profile.ec2: Destruction complete after 2s
aws_iam_role.ec2: Destroying... [id=level7-terraform-ec2-role]
aws_iam_role.ec2: Destruction complete after 1s

Destroy complete! Resources: 30 destroyed.
PS C:\Users\DELL\OneDrive\Documents\level-7-terraform>
```

Figure 90. Destroy the lab infrastructure.

10. Terraform Configuration Overview

The project uses Terraform as the AWS control layer. The evidence shows provider and variable configuration, terraform.tfvars values, VPC resources, security groups, IAM, ECR, Launch Template, ALB, target group and Auto Scaling resources. The lifecycle commands demonstrated are terraform init, terraform validate, terraform plan, terraform apply and terraform destroy.

11. AWS Infrastructure Layers

Network: VPC, public/private subnets, Internet Gateway, NAT Gateway, Elastic IP and route tables. Security: separate ALB and EC2 security groups. IAM: EC2 role, policy and instance profile. Compute: Launch Template and EC2 instances. Load balancing: ALB and target group. Scaling: Auto Scaling Group.

12. Docker, Tomcat and Amazon ECR Integration

The Java web application is packaged into a Docker/Tomcat image, tested locally, tagged for the ECR repository and published to Amazon ECR. EC2 user data uses the instance role and ECR access to retrieve the image and start the application container.

13. Application Request Flow

User → ALB :80 → ALB listener → Target Group → healthy EC2 :8080 → Docker → Tomcat → Java Web Application
→ response through ALB.

17. Key Lessons Learned

Terraform makes AWS infrastructure reproducible and easier to manage as code. Separating public/private subnets improves the architecture. Security groups can enforce an ALB-to-EC2 traffic path. IAM instance profiles avoid long-lived credentials on EC2. ECR provides a managed private image registry. Launch Template user data automates bootstrap. ALB target health and Auto Scaling provide useful operational validation and capacity management.

19. Conclusion

The Level 7 project demonstrates a complete Terraform-managed AWS application environment. The supplied screenshots cover local setup, reusable variables, VPC and routing, security groups, IAM, ECR, Docker/Tomcat, Launch Template automation, ALB, Target Group, Auto Scaling, application validation and Terraform cleanup. The final application screenshots demonstrate successful access through the load-balanced architecture.

14. Validation and Verification Checklist

- Terraform initialization completed.
- Terraform validation completed without errors.
- Terraform plan was reviewed before apply.
- VPC and public/private subnets exist.
- Internet Gateway, NAT Gateway and routes are available.
- ALB security group permits required HTTP traffic.
- EC2 security group permits application traffic from the ALB.
- EC2 IAM role and ECR permissions exist.
- ECR contains the application image.
- Launch Template contains Docker/Tomcat bootstrap.
- ALB and target group are configured.
- Auto Scaling Group and EC2 instances are running.
- Registered targets are healthy.
- SampleApp pages load through the ALB.
- terraform destroy removes the lab resources after validation.

15. Troubleshooting and Operational Notes

Issue	Action
Terraform validation fails	Run terraform validate and inspect the referenced .tf file and line number.
Unexpected plan	Review terraform.tfvars, variable defaults, dependencies and planned changes.
Private EC2 cannot reach ECR	Verify NAT/network connectivity and EC2 ECR permissions.
Container does not start	Check user-data execution, Docker status, image URI/tag and logs.
ALB target unhealthy	Verify port 8080, container state, security-group path and health-check path.
ALB unreachable	Verify public subnets and ALB security-group HTTP access.
Image not found in ECR	Confirm repository URI, tag, region and authentication sequence.

16. Security Notes

- Do not store AWS access keys, passwords or private keys in Terraform source/GitHub.
- Use IAM roles/instance profiles for EC2 AWS permissions where possible.
- Keep application EC2 instances in private subnets.
- Allow public HTTP traffic at the ALB security group only.
- Allow application port 8080 from the ALB security group rather than directly from the Internet.
- Use only the IAM permissions required by the application.

18. Interview / Viva Explanation

I implemented an AWS Infrastructure as Code project using Terraform. I defined the AWS network, security, IAM, ECR, EC2 Launch Template, Application Load Balancer, target group and Auto Scaling Group as Terraform resources. The VPC contains public and private subnets. The ALB receives HTTP traffic on port 80 and forwards it to healthy EC2 targets on port 8080. The Launch Template automates Docker/Tomcat startup and ECR image retrieval. The Auto Scaling Group manages the application fleet. Finally, I verified the SampleApp through the ALB and used terraform destroy to clean up the lab.

Appendix – Complete Project Flow

Terraform Project → init / validate / plan → apply → VPC → Public/Private Subnets → Internet Gateway / NAT / Routing → Security Groups → IAM → ECR → Docker/Tomcat Image → Launch Template → ALB :80 → Target Group :8080 → Auto Scaling Group → Healthy EC2 Instances → SampleApp → terraform destroy